



Information Technology Institute (ITI)

RANPilotAI

AI-Powered RAN Performance
Optimization Assistant

Graduation Project • 2026

Development Team

Mahmoud Ashraf • Ibrahim Samy • Khaled Mogahed
Tasneem Amein • Mohab Tarek

Track Supervisor
Dr. Salma Sobhy

In Collaboration with



Abstract

Modern Radio Access Network (RAN) optimization relies heavily on manual inspection of vast volumes of performance data, expert engineering judgement, and reactive troubleshooting workflows. As LTE and 5G New Radio (NR) networks continue to grow in scale and complexity, this approach becomes increasingly inefficient and unscalable. This graduation project presents **RANPilotAI**, an integrated, AI-powered platform designed to automate and augment the daily workflows of Radio Frequency (RF) optimization engineers in 4G and 5G mobile networks.

The platform unifies four specialized analytical modules within a single web-based environment called the **Telecom Hub**, implemented using the Streamlit framework. Engineers can perform multiple optimization tasks through a common interface without switching between separate software tools.

The first module, the **KPI Degradation Analyzer**, automatically detects performance degradation across 13 Key Performance Indicator categories — including throughput, accessibility, retainability, mobility, availability, VoLTE, CSFB, and RRC re-establishment — using a three-layer analytical pipeline. The pipeline applies Welch’s t-test for statistical confidence scoring, RF-aware root cause analysis with configurable severity weighting, and a data quality framework that handles sentinel values, unit-aware validation, and same-weekday baseline imputation.

The second module, the **KPI Forecasting Engine**, predicts future network performance trends using two complementary time-series models: XGBoost with leakage-safe cross-KPI feature engineering, and Holt-Winters exponential smoothing. The engine provides backtesting metrics (MAE, RMSE, MAPE), STL-based seasonality analysis, recursive 7-day forward forecasts, and residual diagnostics that allow engineers to anticipate degradation before it manifests operationally.

The third module, the **Optimization Action Analyzer**, correlates RF configuration changes — such as antenna tilt and transmit power adjustments — with KPI impact using Dolt, a version-controlled SQL database. The module aligns configuration commit history with KPI time windows using weekday-matched baseline comparison, groups same-day commits into unified action hunks, and presents results through interactive timeline, before/after overlay, delta bar, and summary impact visualizations.

The fourth module, the **Telecom RAG Assistant**, is a Retrieval-Augmented Generation system that enables engineers to query official 3GPP Technical Specifications using natural language. The system combines section-aware PDF chunking, BGE-large dense embeddings, local Qdrant vector search, cross-encoder re-ranking, and a strict anti-hallucination prompt to produce traceable, citation-backed answers grounded directly in official standards documentation.

Experimental results demonstrate that the integrated platform substantially reduces the time required for network performance analysis, root cause identification, and standards consultation, while maintaining engineering accuracy and traceability. The RAN-PilotAI platform is implemented entirely in Python 3.11 and is accessible through a standard web browser without additional client-side installation.

Dedication

To our families,

*whose patience, encouragement, and unwavering belief sustained us
through every late night and every challenge this project brought.*

*To our instructors and supervisors at the Information Technology Institute,
and to Etisalat and (e&) for their mentorship and guidance,*

*whose support shaped not only this work
but the engineers we have become.*

*And to every RF engineer who spends hours
searching through performance counters and specification pages —
this platform was built for you.*

Acknowledgements

We would like to express our sincere gratitude to all those who contributed to the successful completion of this graduation project.

First and foremost, we extend our deepest appreciation to the **Information Technology Institute (ITI)**, whose rigorous academic programme and practical engineering environment provided the foundation upon which this work was built. We are also deeply grateful to **Etisalat and (e&)** for their mentorship, guidance, and industry support. The resources, facilities, and opportunities made available by the institute and our industry partners were essential to the realization of the RANPilotAI platform.

We are especially grateful to our **supervisor, DR: Salma Sobhy**, and our supervisors and instructors for their continuous guidance, technical insight, and encouragement throughout the project lifecycle. Their expertise in telecommunications engineering, software development, and artificial intelligence significantly influenced the direction and quality of this work.

We also thank our colleagues and peers at ITI for their constructive feedback, collaborative spirit, and shared knowledge throughout the duration of the programme.

On a personal level, each team member would like to acknowledge the support of their family and friends, whose patience and encouragement made it possible to devote the time and effort this project demanded.

Finally, we acknowledge the open-source communities behind the tools and libraries that made this platform possible, including Streamlit, XGBoost, Qdrant, Sentence Transformers, Dolt, and the broader Python scientific ecosystem. This project stands on the shoulders of the open-source work that preceded it.

The RANPilotAI Team

- Mahmoud Ashraf
- Ibrahim Samy
- Khaled Mogahed
- Tasneem Amein
- Mohab Tarek

Contents

Abstract	i
Dedication	ii
Acknowledgements	iii
1 Introduction and Motivation	1
1.1 Introduction	1
1.2 Motivation	2
1.3 Problem Statement	3
1.4 Background	4
1.4.1 Radio Access Network Performance Management	4
1.4.2 Network Optimization Workflow	4
1.5 Proposed Solution Overview	5
1.6 Platform Modules	5
1.6.1 KPI Degradation Analyzer	5
1.6.2 KPI Forecasting Engine	5
1.6.3 Optimization Action Analyzer	6
1.6.4 Telecom RAG Assistant	6
1.7 Summary	6
2 Literature Review	7
2.1 Introduction	7
2.2 Network KPI Monitoring and Performance Management	7
2.2.1 LTE and 5G NR Performance Architecture	7
2.2.2 Performance Degradation Detection	8
2.3 Statistical Methods for Performance Analysis	8
2.3.1 Hypothesis Testing for Degradation Confidence	8
2.3.2 Baseline Computation Methods	9
2.4 Machine Learning for Time Series Forecasting	9
2.4.1 Classical Time Series Methods	9
2.4.2 Gradient Boosting for Non-linear Temporal Modelling	9
2.5 Root Cause Analysis in Telecommunications	10
2.5.1 Rule-Based Root Cause Inference	10
2.5.2 Data Quality and Artefact Handling	10
2.6 Retrieval-Augmented Generation for Engineering Domains	10
2.6.1 Information Retrieval Foundations	10
2.6.2 Neural Re-ranking and Anti-hallucination	10
2.7 Software Architecture for Integrated Platforms	11

2.7.1	Modular Architecture Patterns	11
2.7.2	Web-Based Deployment Architectures	11
2.8	Summary	11
3	Overall System Design	13
3.1	Introduction	13
3.2	Design Objectives and Constraints	14
3.2.1	Functional Objectives	14
3.2.2	Non-Functional Objectives	14
3.2.3	Technical Constraints	15
3.3	Overall Platform Architecture	15
3.4	Software Architecture and Design Patterns	16
3.4.1	Layered Architecture	16
3.4.2	Design Patterns	17
3.4.3	Data Flow Architecture	17
3.5	Technology Stack	17
3.5.1	Backend Framework	17
3.5.2	Data Processing	17
3.5.3	Machine Learning	18
3.5.4	Retrieval-Augmented Generation	18
3.5.5	Data Persistence	18
3.6	Data Management Strategy	18
3.6.1	KPI Data Schema	18
3.6.2	Configuration Change Tracking	19
3.6.3	Vector Index Management	19
3.7	Module Interfaces and Integration	19
3.7.1	Telecom Hub Interface	19
3.7.2	Session State Management	19
3.7.3	Data Sharing Between Modules	20
3.8	Deployment Architecture	20
3.8.1	Single-Workstation Deployment	20
3.8.2	Scalability Considerations	20
3.9	Summary	20
4	KPI Degradation Analyzer	22
4.1	Introduction	22
4.2	Motivation	23
4.3	Problem Statement	24
4.4	Background	24
4.4.1	KPI Categories and Performance Pillars	24
4.4.2	Welch's t-test for Degradation Confidence	25
4.4.3	RF-Aware Root Cause Scoring	25
4.5	Analysis Configuration	26
4.6	Analysis Results and KPI Summary	27
4.7	Main Dashboard and Charts	28
4.8	Root Cause Analysis	29
4.8.1	Root Cause Dashboard	29
4.8.2	Root Cause Recommendations	29
4.8.3	Severity and Confidence Metrics	31

4.9	Trend Analysis	31
4.10	Anomaly Detection	33
4.11	Export Options	34
4.12	Summary	36
5	KPI Forecasting & Predictive Analytics Module	37
5.1	Introduction & Motivation	37
5.2	Theoretical Background	38
5.2.1	Time-Series Forecasting in Telecom Networks	38
5.2.2	Holt-Winters Exponential Smoothing	38
5.2.3	Gradient Boosting for Time-Series (XGBoost)	38
5.2.4	Residual Diagnostics & Model Validation	39
5.3	System Requirements & Design	39
5.3.1	Functional Requirements	39
5.3.2	Module Architecture & Data Flow	40
5.3.3	Interface with Adjacent Modules	40
5.4	Data Pipeline & Preprocessing	41
5.4.1	Input Data Format	41
5.4.2	Seasonality Detection	41
5.4.3	Cross-KPI Feature Engineering	42
5.5	Forecasting Models	43
5.5.1	Baseline Models	43
5.5.2	Holt-Winters Exponential Smoothing	43
5.5.3	XGBoost Regressor with Recursive Multi-Step Forecasting	43
5.5.4	Model Selection & Fallback Logic	45
5.6	Model Evaluation & Diagnostics	45
5.6.1	Accuracy Metrics	45
5.6.2	Baseline Comparison	46
5.6.3	Residual Analysis	46
5.6.4	Feature Importance Interpretation	48
5.7	Chapter Summary	48
6	Optimization Action Analyzer	50
6.1	Introduction	50
6.2	Motivation and Problem Statement	50
6.3	System Architecture	51
6.4	Data Model and Sources	51
6.5	Core Algorithms	52
6.5.1	Action Hunks: Grouping Commits by Calendar Day	52
6.5.2	Rolling Evaluation Window	52
6.5.3	Weekday-Matched Baseline Construction	53
6.5.4	Optimization Polarity	53
6.6	Visualization Layer	53
6.7	User Workflow	54
6.8	Technology Stack	57
6.9	Design Highlights	57
6.10	Limitations and Future Work	57
6.11	Summary	58

7	Telecom RAG Assistant	59
7.1	Introduction	59
7.2	Motivation	61
7.3	Problem Statement	61
7.4	Background	62
7.4.1	Large Language Models (LLMs)	62
7.4.2	Retrieval-Augmented Generation (RAG)	63
7.4.3	Embedding Models	63
7.4.4	Vector Databases	64
7.4.5	Cross Encoder Re-ranking	64
7.5	Overall System Architecture	64
7.6	Software Architecture	66
7.7	Query Processing Workflow	67
7.8	PDF Processing	68
7.9	Section-aware Chunking	68
7.10	Embedding Generation	69
7.11	Vector Database (Qdrant)	69
7.12	Metadata Filtering	70
7.13	Hybrid Retrieval Pipeline	71
7.14	Cross Encoder Re-ranking	72
7.15	Prompt Construction	73
7.16	LLM Response Generation	73
7.17	User Interface	74
7.18	Performance Evaluation	77
7.19	Discussion	78
7.20	Limitations	78
7.21	Future Improvements	79
7.22	Chapter Summary	79
8	System Integration	80
8.1	Introduction	80
8.2	Overall Platform Architecture	80
8.3	Integration Objectives	81
8.4	Layered Software Architecture	82
8.5	Module Integration	83
8.6	Shared Components	85
8.7	Data Flow Between Modules	86
8.8	User Navigation Workflow	87
8.9	Deployment Environment	88
8.10	Platform Scalability	89
8.11	Integration Challenges	90
8.12	Future Extensions	90
8.13	Chapter Summary	91
9	Testing Strategy	92
9.1	Introduction	92
9.1.1	Purpose	92
9.2	Scope	92
9.3	Testing Levels	92

9.3.1	Unit Testing	92
9.3.2	Integration Testing	93
9.3.3	System Testing	93
9.4	Functional Test Cases	95
9.5	Boundary Testing	97
9.6	Negative Testing	97
9.7	Regression Testing	97
9.8	Acceptance Criteria	98
9.8.1	Test Coverage Summary	98
9.9	Testing Framework Architecture	98
9.9.1	Workflow Diagram	99
9.9.2	Modular Subsystem Map	100
9.10	Test Execution Observations	100
9.10.1	Test Case: TC-001	100
9.10.2	Test Case: TC-002	101
10	Conclusion and Future Work	102
10.1	Conclusion	102
10.2	Limitations	104
10.3	Future Work	104
10.4	Closing Remarks	105
11	User Manual	107
11.1	System Requirements	107
11.2	Installation	107
11.2.1	Step 1 — Install Python	107
11.2.2	Step 2 — Install Dependencies	107
11.2.3	Step 3 — Configure the RAG Assistant (First Time Only)	108
11.2.4	Step 4 — Configure the Optimization Action Analyzer (If Required)	108
11.3	Launching the Platform	108
11.4	Navigating the Telecom Hub	109
11.5	Tool 1 — KPI Degradation Analyzer	109
11.5.1	Purpose	109
11.5.2	Input Data Format	109
11.5.3	Step-by-Step Usage	109
11.5.4	Understanding the Output	110
11.6	Tool 2 — KPI Forecasting Engine	110
11.6.1	Purpose	110
11.6.2	Input Data Format	110
11.6.3	Step-by-Step Usage	111
11.7	Tool 3 — Optimization Action Analyzer	111
11.7.1	Purpose	111
11.7.2	Prerequisites	111
11.7.3	Step-by-Step Usage	112
11.8	Tool 4 — Telecom RAG Assistant	112
11.8.1	Purpose	112
11.8.2	Prerequisites	112
11.8.3	Step-by-Step Usage	112
11.8.4	Performance Tuning	113

11.9 Troubleshooting	113
References	114

List of Figures

1.1	Home interface of the RANPilotAI platform presenting the Telecom Hub with the four selectable analytical modules: KPI Degradation Analyzer, KPI Forecasting Engine, Optimization Action Analyzer, and Telecom RAG Assistant.	2
3.1	Navigation interface of the RANPilotAI Telecom Hub presenting the four selectable analytical modules. The interface provides consistent access to KPI Degradation Analyzer, KPI Forecasting Engine, Optimization Action Analyzer, and Telecom RAG Assistant through a unified module selection paradigm.	14
4.1	Initial interface of the KPI Degradation Analyzer module showing the tool header, sidebar data source and analysis settings panel, and the four capability cards: Degradation Detection, Dashboard & Charts, Trend Analysis, and Anomaly Detection.	23
4.2	KPI Degradation Analyzer configuration panel after loading the New Cairo KPI dataset (37,684 rows, 308 columns). The sidebar shows the selected analysis settings: All KPIs, 4-day observation window, 30% degradation threshold, Historical Weekday Median baseline mode, and 4 lookback weeks.	26
4.3	KPI Summary tab showing the analysis results for the New Cairo dataset (37,684 rows, 308 columns, All KPIs, 30% threshold, Historical Weekday Median baseline). The table lists each KPI category with its degraded cell count and maximum degradation percentage.	27
4.4	KPI Degradation Analyzer main dashboard presenting KPI-level degradation metrics, cell count distributions, and interactive charts for the New Cairo dataset analysis.	28
4.5	Root cause analysis dashboard showing the distribution of RCA pattern classifications, severity scores, and correlated diagnostic counter evidence across degraded cells in the New Cairo dataset.	29
4.6	Root cause recommendation panel for a degraded cell, showing the identified RCA pattern, supporting correlated counter evidence, RF severity classification, and recommended next investigation steps.	30
4.7	Severity and confidence metric panels showing the distribution of rf_severity classifications and analysis_confidence levels across all degraded cells detected in the New Cairo dataset analysis.	31
4.8	Trend analysis view showing the KPI time series for a selected degraded cell, with the historical baseline period and recent observation window indicated. The before-and-after overlay enables visual assessment of degradation onset and trajectory.	32

4.9	Additional trend analysis drill-down showing multi-day KPI trajectory comparison between the baseline and recent periods for a second degraded cell, demonstrating the enhancement potential and before-after overlay visualisation capability.	33
4.10	Anomaly detection panel showing zero-value and spike anomalies identified in the KPI time series across the analysed cell set, using a 4-week rolling baseline for comparison.	34
4.11	Primary export options panel of the KPI Degradation Analyzer, providing download buttons for Excel and Word format exports of the degradation analysis results.	35
4.12	Additional export options providing CSV download and individual chart export capabilities for integration with external reporting and ticketing systems.	35
5.1	High-level data-flow diagram of the KPI Forecasting Module. Solid arrows indicate primary data flow; dashed arrows indicate metadata or control signals.	40
5.2	Streamlit sidebar exposing XGBoost hyperparameters: <code>n_estimators=100</code> , <code>learning_rate=0.05</code> , <code>max_depth=3</code> , and <code>subsample=0.80</code>	44
5.3	Dashboard header for cell LCAIN10022-6 forecasting DL Traffic Volume. The seasonality strength badge ($F_{\text{seasonal}} = 0.87$) and baseline comparison panel confirm that Holt-Winters (MAE 6.352) substantially outperforms the naive baseline (MAE 16.384), validating model deployment for this cell.	45
5.4	Forecast chart for DL Traffic Volume (GBytes) on cell LCAIN10022-6. The strong weekly seasonality is accurately tracked by Holt-Winters (purple), while XGBoost (red) exhibits larger deviations during the hold-out period. The naive baseline (gray dotted) flatlines at the last training value.	46
5.5	Residual summary statistics for XGBoost on the training set. Mean residual = 0.0027, standard deviation = 0.2397, Durbin-Watson = 1.638 (within acceptable range), and Ljung-Box p -value = 0.923 (no significant autocorrelation).	47
5.6	Four-panel residual diagnostic plot for XGBoost. Top-left: residuals over time with zero reference; top-right: residual histogram; bottom-left: ACF with 95% confidence bands; bottom-right: residuals versus fitted values. The absence of structure supports the white-noise hypothesis.	47
5.7	Top-15 XGBoost feature importance by gain for DL Traffic Volume forecasting. <code>lag7_norm</code> and <code>lag_14</code> dominate, confirming strong weekly periodicity. Cross-KPI features (<code>Inter_HO_SR</code> , <code>Avg_UE_Number</code>) and momentum terms contribute non-linear explanatory power.	48
6.1	High-level module architecture of the Optimization Action Analyzer. The Streamlit application (<code>app.py</code>) orchestrates calls into four independent layers: database access, data wrangling, visualization, and configuration persistence.	51
6.2	Action Timeline HUD showing five Action Hunks with per-commit hover detail.	54
6.3	Before/After KPI trend comparison for a selected metric, with commit annotations.	55
6.4	Polarity-aware interval delta bar chart.	55

6.5	KPI impact summary table with Improved/Degraded/No-Change status counts.	55
6.6	The five-step analyst workflow implemented in <code>app.py</code>	56
7.1	Main interface of the Telecom RAG Assistant integrated into the RAN-PilotAI platform. The interface provides access to the Telecom RAG module, allows filtering by specific 3GPP Technical Specifications, and supports interactive engineering question answering using Retrieval-Augmented Generation.	60
7.2	Overall architecture of the proposed Telecom RAG Assistant.	65
7.3	Real-time execution stages of the Retrieval-Augmented Generation pipeline during query processing.	67
7.4	Specification Filter allowing users to restrict semantic retrieval to selected 3GPP Technical Specifications.	71
7.5	Execution stages of the hybrid Retrieval-Augmented Generation pipeline.	72
7.6	Generated engineering response for the LTE Event A3 measurement event.	74
7.7	Main interface of the Telecom RAG Assistant.	75
7.8	Context-aware response generated by the Telecom RAG Assistant using information from previous user interactions within the same conversation.	76
7.9	Structured engineering summary generated by the Telecom RAG Assistant.	76
7.10	Generated response explaining the Packet Data Convergence Protocol (PDCP).	77
7.11	Structured engineering summary for the PDCP protocol.	77
7.12	Pipeline profiling and execution statistics generated during query processing.	78
8.1	Overall architecture of the integrated RANPilotAI platform. The platform groups all analytical modules within a unified engineering environment while providing common platform services shared across the developed applications.	81
8.2	Layered software architecture of the RANPilotAI platform. The platform is organized into four logical layers separating presentation, application logic, engineering modules, and data management.	83
8.3	Module interaction within the RANPilotAI platform. The Telecom Hub coordinates user requests while common software services are shared among all analytical modules.	84
8.4	Component organization of the RANPilotAI platform showing the relationship between the Telecom Hub, shared software components, platform resources, and engineering modules.	86
8.5	Overall data flow within the RANPilotAI platform. All requests are coordinated by the Telecom Hub before reaching the selected engineering module.	87
8.6	Typical user navigation workflow within the RANPilotAI platform.	88

List of Tables

1.1	Standard RAN performance pillars and associated KPI categories monitored by the RANPilotAI platform.	4
3.1	Platform components and their primary responsibilities within the RAN-PilotAI architecture.	16
4.1	KPI categories monitored by the KPI Degradation Analyzer.	25
5.1	Supported LTE KPIs with canonical internal names and performance categories.	41
5.2	Leakage-safe feature engineering taxonomy for the XGBoost regressor. . . .	42
6.1	Module responsibilities	52
6.2	Technologies used in the Optimization Action Analyzer	57

Chapter 1

Introduction and Motivation

Modern Radio Access Network (RAN) optimization has traditionally relied on manual inspection of vast performance datasets, reactive troubleshooting workflows, and fragmented software tooling. As LTE and 5G New Radio (NR) networks continue to expand in scale and operational complexity, this approach becomes increasingly inefficient, inconsistent, and incapable of delivering timely engineering decisions. This graduation project presents **RANPilotAI**, an integrated, AI-powered platform designed to automate and augment the daily workflows of Radio Frequency (RF) optimization engineers in 4G LTE and 5G mobile networks.

The platform unifies four specialized analytical modules within a single web-based environment called the **Telecom Hub**, implemented using the Streamlit framework. The developed modules address the complete RAN optimization cycle: the **KPI Degradation Analyzer** detects and classifies performance degradations using statistically rigorous methods; the **KPI Forecasting Engine** predicts future network performance trajectories using ensemble time-series models; the **Optimization Action Analyzer** correlates configuration changes with KPI impact using version-controlled data management; and the **Telecom RAG Assistant** enables natural-language consultation of official 3GPP Technical Specifications through Retrieval-Augmented Generation.

1.1 Introduction

Modern telecommunications networks generate extraordinary volumes of performance management data. A single LTE macro cell produces thousands of performance counter records every hour, spanning dozens of Key Performance Indicator (KPI) categories including accessibility, retainability, mobility, integrity, and availability metrics [1]. For a metropolitan network comprising thousands of cells, this results in datasets containing millions of rows and hundreds of columns that must be analysed daily to maintain acceptable subscriber quality of service.

The traditional approach to network optimization relies on human engineers manually inspecting these datasets, applying domain expertise to identify degraded cells, determining probable root causes, and recommending corrective actions. While effective for small networks, this workflow does not scale to modern network sizes. An engineer manually reviewing a weekly KPI report for a city-scale deployment may spend hours identifying cells requiring attention, and the consistency of the analysis depends entirely on the individual engineer's experience and concentration [2].

Figure 1.1 presents the home interface of the RANPilotAI platform. The interface

provides centralized access to all four analytical modules through the Telecom Hub, presenting each module as a selectable capability card alongside platform information and navigation controls.

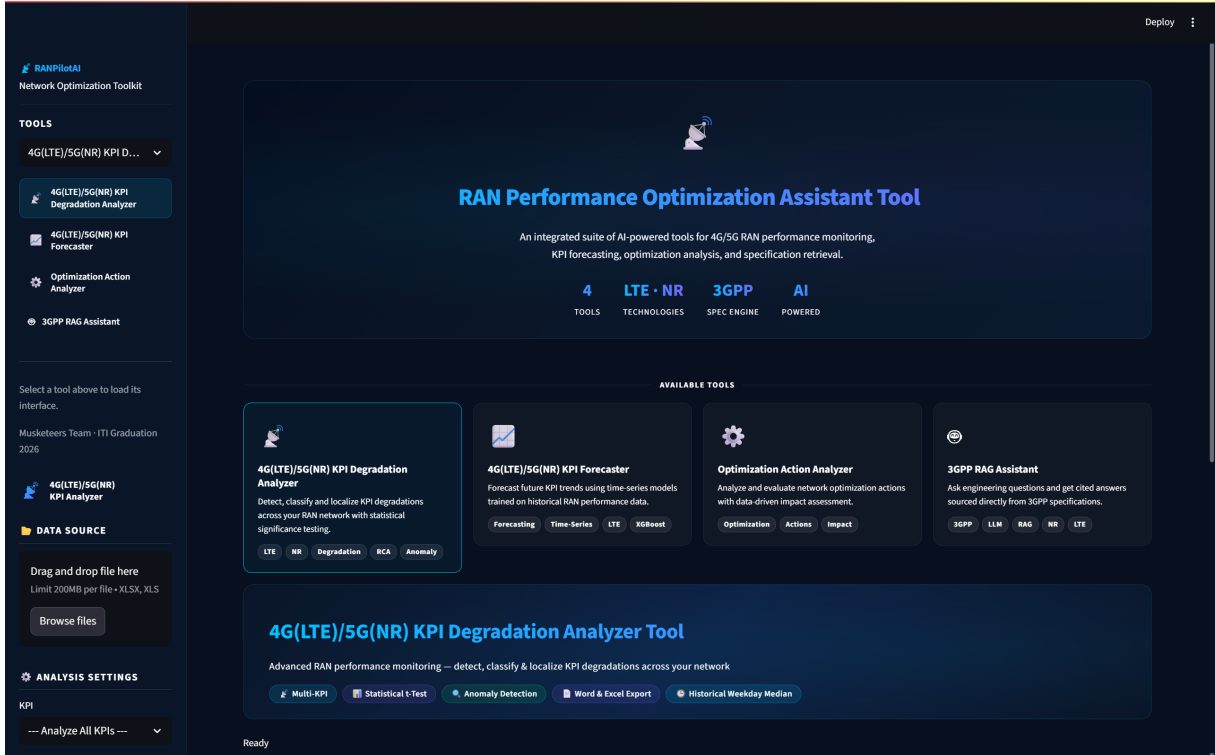


Figure 1.1: Home interface of the RANPilotAI platform presenting the Telecom Hub with the four selectable analytical modules: KPI Degradation Analyzer, KPI Forecasting Engine, Optimization Action Analyzer, and Telecom RAG Assistant.

The platform architecture follows a modular design in which each analytical module encapsulates its own processing logic while sharing common infrastructure services provided by the Telecom Hub. Engineers interact exclusively with the hub, which manages application routing, session handling, and data persistence. This unified environment eliminates the need to switch between separate software tools during the optimization workflow, reducing context-switching overhead and ensuring that data and analysis artefacts remain accessible across modules.

1.2 Motivation

The motivation for developing the RANPilotAI platform arises from five converging pressures in modern network operations.

First, data volume growth has outpaced manual analysis capacity. LTE and 5G NR networks produce performance counters at intervals as short as 15 minutes, generating datasets that are impractical to review manually at network scale. A mid-sized operator managing ten thousand cells across multiple frequency layers may collect over one hundred thousand KPI observations per day, a volume that defies spreadsheet-based inspection.

Second, operational tempo has accelerated. Subscriber expectations for uninterrupted service, combined with competitive pressure and regulatory service-level obligations, require network issues to be identified and resolved within hours rather than days. Reactive

workflows that detect degradation only after subscriber complaints or quality-of-service breaches are no longer acceptable.

Third, domain expertise is becoming scarce. The retirement of experienced RF engineers who accumulated decades of hands-on network knowledge creates a skills gap that manual troubleshooting workflows cannot bridge. Systematic, knowledge-encoded automation is required to preserve and transfer this expertise to new engineering teams.

Fourth, existing tooling is fragmented. Network optimization currently requires engineers to interact with multiple vendor-specific network management systems, spreadsheet applications, statistical packages, and documentation repositories. The absence of a unified environment increases cognitive load, introduces data transfer errors, and fragments the audit trail of engineering decisions.

Fifth, artificial intelligence technologies have matured to the point where they can reliably assist with domain-specific engineering tasks. Advances in statistical learning, gradient boosting, retrieval-augmented generation, and version-controlled data management provide the technical foundation for building intelligent assistance tools that are trustworthy enough for professional engineering use.

RANPilotAI was therefore conceived as a response to these pressures: a unified platform that encodes RF optimization knowledge into automated analytical modules while preserving the engineer’s role as the final decision-maker. Rather than replacing engineering judgement, the platform accelerates the path from raw data to actionable insight.

1.3 Problem Statement

The central problem addressed by this graduation project can be decomposed into four interrelated technical challenges.

Scalable degradation detection: Network operators require automated methods for identifying performance degradations across thousands of cells and dozens of KPI categories without manual threshold tuning or cell-by-cell inspection. The detection method must account for the natural variability of each cell’s historical performance, handle data quality artefacts introduced by vendor systems, and assign statistically meaningful confidence levels to detected degradations.

Predictive performance modelling: Reactive maintenance — addressing network issues only after they manifest as subscriber-facing incidents — is costly and operationally undesirable. Operators need reliable forecasts of future KPI trajectories to enable proactive interventions. Forecasting must accommodate non-linear temporal dependencies, multi-KPI interactions, and varying degrees of seasonality across different cells and KPIs.

Optimization impact quantification: RF configuration changes — including antenna tilt adjustments, transmit power modifications, and handover parameter updates — produce effects that are difficult to isolate from normal network variability. Engineers require systematic methods for correlating configuration commits with KPI changes while controlling for confounding factors such as traffic patterns, weather, and concurrent network events.

Standards consultation efficiency: 3GPP Technical Specifications contain thousands of pages of protocol definitions, signaling procedures, and configuration parameters distributed across hundreds of documents. Engineers require efficient mechanisms for locating and interpreting relevant standards content without spending hours navigating PDF files or relying on potentially inaccurate memory.

1.4 Background

1.4.1 Radio Access Network Performance Management

A Radio Access Network connects user equipment — mobile phones, IoT devices, and other wireless terminals — to the mobile core network through base stations known as evolved Node Bs (eNodeBs) in LTE and gNodeBs (gNBs) in 5G NR. Each base station serves one or more cells, and each cell is characterised by a set of Key Performance Indicators that quantify its operational health from multiple perspectives [3].

The three primary performance pillars used in RAN assessment are accessibility, retainability, and mobility. Accessibility measures the ability of user equipment to establish connections, quantified through indicators such as RRC Setup Success Rate and E-RAB Setup Success Rate. Retainability measures the ability to maintain established connections, quantified through E-RAB Drop Rate and RRC Re-establishment Success Rate. Mobility measures the success of handover procedures between cells, quantified through Handover Success Rate and RRC Re-establishment delay [1].

In addition to these core pillars, modern networks monitor throughput KPIs (DL/UL Traffic Volume and Throughput), availability metrics (Cell Availability and Downtime), and service-specific indicators for VoLTE and CSFB services. Table 1.1 summarises the five standard performance pillars and their associated KPI categories.

Table 1.1: Standard RAN performance pillars and associated KPI categories monitored by the RANPilotAI platform.

Pillar	Associated KPI Categories
Accessibility	RRC Setup Success Rate, E-RAB Setup Success Rate, RACH Success Rate, CSFB KPIs
Retainability	E-RAB Drop Rate, RRC Re-establishment Success Rate
Mobility	Handover Success Rate
Integrity	DL Throughput, UL Throughput, VoLTE KPIs
Availability	Cell Availability, Downtime

1.4.2 Network Optimization Workflow

The standard network optimization workflow comprises four phases: data collection, performance analysis, root cause identification, and corrective action implementation. In the data collection phase, network management systems export performance counter data at regular intervals, typically every 15 minutes for LTE networks and every 5 minutes for 5G NR networks. This data is stored in proprietary vendor formats and must be transformed into analysis-ready tabular structures before statistical processing can begin.

The performance analysis phase involves identifying cells whose KPI values have degraded relative to their historical baselines. Engineers apply thresholds — often fixed values inherited from vendor recommendations — to flag underperforming cells. Threshold-based alerting suffers from high false-positive rates because it ignores the natural variability and baseline performance level of each individual cell [4].

The root cause identification phase requires the engineer to examine the diagnostic counters associated with each flagged cell and apply domain knowledge to determine the

probable cause of degradation. Common root causes include coverage problems (insufficient signal strength or excessive interference), capacity congestion (insufficient resources relative to traffic demand), hardware faults, and configuration errors [2].

The corrective action phase involves implementing configuration changes — such as adjusting antenna tilt, modifying handover parameters, or adding capacity — and then monitoring the resulting KPI impact to validate the intervention.

1.5 Proposed Solution Overview

RANPilotAI addresses the limitations of manual network optimization by encoding each phase of the workflow into dedicated analytical modules implemented within a unified web-based platform. The platform operates as a centralized engineering environment accessible through a standard web browser, eliminating the need for client-side software installation and enabling engineers to perform complex analytical tasks from any workstation.

The platform architecture follows a modular design philosophy in which the Telecom Hub acts as the central coordination point. Rather than exposing individual module interfaces directly to the user, the hub provides a consistent navigation paradigm, shared session management, and common data services. Each module communicates with the hub through standardized interfaces, preserving module autonomy while ensuring data consistency across the platform.

The technical implementation adopts Python 3.11 as the primary development language, with Streamlit providing the web-based user interface framework. Data processing leverages the Pandas and NumPy scientific computing ecosystem, while machine learning components are built upon Scikit-learn, XGBoost, and Statsmodels. Retrieval-augmented generation capabilities are provided through sentence-transformers, Qdrant vector database, and OpenRouter language model APIs. Version-controlled data management is provided through Dolt, a SQL database with Git-like versioning semantics.

1.6 Platform Modules

The RANPilotAI platform integrates four analytical modules, each addressing a distinct phase of the RAN optimization workflow.

1.6.1 KPI Degradation Analyzer

The KPI Degradation Analyzer is the primary performance analysis module of the platform. It processes raw KPI counter exports and applies a three-layer analytical pipeline comprising data quality validation, statistical degradation detection, and RF-aware root cause analysis. The module monitors 13 KPI categories across five performance pillars and produces structured degradation reports including severity classifications, confidence scores, root cause pattern identifications, and recommended next investigation steps.

1.6.2 KPI Forecasting Engine

The KPI Forecasting Engine predicts future network performance using an ensemble of statistical and machine learning models. The module implements Holt-Winters exponential smoothing for classical seasonal pattern capture and XGBoost gradient boosting

with leakage-safe cross-KPI feature engineering for non-linear multivariate modelling. A unified evaluation framework compares all candidate models against hold-out data and automatically recommends the most reliable predictor on a per-cell, per-KPI basis. The engine generates recursive 7-day forward forecasts accompanied by residual diagnostics and seasonality quantification.

1.6.3 Optimization Action Analyzer

The Optimization Action Analyzer correlates RF configuration changes with KPI impact using Dolt as a version-controlled data backend. The module tracks antenna tilt, transmit power, and other configuration parameters through commit history, aligns configuration events with KPI time windows using weekday-matched baseline comparison, and presents results through interactive timeline, before/after overlay, delta bar, and summary impact visualizations.

1.6.4 Telecom RAG Assistant

The Telecom RAG Assistant enables engineers to query official 3GPP Technical Specifications using natural language. The system combines PDF processing, section-aware chunking, BGE-large dense embeddings, local Qdrant vector search, cross-encoder re-ranking, and strict anti-hallucination prompt engineering to produce traceable, citation-backed answers grounded directly in official standards documentation.

1.7 Summary

This chapter has introduced the RANPilotAI platform as an integrated, AI-powered solution for Radio Access Network optimization. The chapter has motivated the development of the platform through an analysis of the limitations inherent in current manual network optimization workflows, formulated the technical challenges addressed by the developed modules, and provided a high-level overview of the platform architecture and its four constituent analytical modules.

The subsequent chapters develop the platform components in detail. Chapter 2 surveys the relevant research literature spanning network performance management, statistical learning, time-series forecasting, root cause analysis, retrieval-augmented generation, and software architecture. Chapter 3 presents the detailed system design, including the platform architecture, technology stack, data management strategy, and module integration interfaces. Chapters 4 through 7 describe each analytical module in depth, following the structure established in this chapter: module introduction, operational motivation, technical problem formulation, background theory, configuration and results presentation, and summary.

Chapter 2

Literature Review

This chapter surveys the research literature and established engineering practice relevant to the development of the RANPilotAI platform. The review is organised thematically, progressing from foundational concepts in mobile network performance management through advanced machine learning and information retrieval techniques to software architecture considerations for integrated engineering platforms. Each section establishes the theoretical and practical context for the corresponding module of the RANPilotAI platform, identifies gaps in existing tooling, and justifies the technical approaches adopted in this project.

2.1 Introduction

The convergence of mobile telecommunications, statistical computing, and artificial intelligence has produced a rich body of research spanning network performance management, time-series forecasting, root cause analysis, and retrieval-augmented generation. This literature review synthesises the most relevant contributions across these domains and maps them to the four analytical modules developed within the RANPilotAI platform.

The review begins with the fundamentals of LTE and 5G NR performance management, establishing the KPI taxonomy and measurement methodology that underpin all subsequent analysis. It then examines statistical methods for degradation detection, machine learning approaches to time-series forecasting, and knowledge-engineering techniques for root cause attribution. The review continues with information retrieval and natural language processing research relevant to the Telecom RAG Assistant, and concludes with software architecture literature that informed the design of the integrated platform.

2.2 Network KPI Monitoring and Performance Management

2.2.1 LTE and 5G NR Performance Architecture

Modern LTE and 5G NR networks implement a hierarchical performance management architecture defined by the Third Generation Partnership Project (3GPP) in Technical Specification 32.401. This specification establishes the standard set of Key Performance Indicators, measurement procedures, and reporting formats used by network operators

worldwide [1]. The performance management architecture operates through three logical entities: the Network Element (NE), which generates performance counters; the Operations Support System (OSS), which collects and processes these counters; and the Network Management System (NMS), which presents processed results to engineering personnel.

The standard defines three measurement types: count-type measurements, which accumulate events over a measurement period; gauge-type measurements, which capture instantaneous values such as buffer occupancy; and status-type measurements, which indicate the presence or absence of specific conditions. Each measurement is associated with a measurement scope — cell, eNodeB, or network level — and a reporting period that typically ranges from 15 minutes to 24 hours depending on the KPI criticality.

Dahlman et al. provide a comprehensive treatment of the 5G NR physical layer and system architecture, establishing the performance characteristics and operational constraints that inform the KPI schema developed for the RANPilotAI platform [3]. Their work describes the relationship between radio resource configuration, scheduling decisions, and resulting throughput and latency metrics, providing the domain knowledge required to interpret degradation patterns and assign meaningful root cause classifications.

2.2.2 Performance Degradation Detection

Automated performance degradation detection in cellular networks has been an active research area since the adoption of LTE technology. Mishra provides foundational coverage of cellular network planning and optimisation, including threshold-based alerting methodologies and the limitations of fixed-threshold approaches in heterogeneous network environments [4]. The primary limitation identified in this literature is that threshold crossing alone provides no information about the probable root cause of the degradation and cannot account for cell-specific baseline performance variations.

Holma and Toskala’s treatment of LTE-Advanced systems examines the performance management requirements of evolved networks, including the introduction of carrier aggregation, coordinated multi-point transmission, and heterogeneous network topologies [2]. Their analysis demonstrates that these advanced features increase the dimensionality of performance analysis, making manual inspection increasingly impractical and motivating the development of automated, statistically rigorous detection methods.

2.3 Statistical Methods for Performance Analysis

2.3.1 Hypothesis Testing for Degradation Confidence

Welch’s two-sample t-test is the primary statistical method employed by the RANPilotAI KPI Degradation Analyzer for assigning confidence levels to detected degradations. The test is preferred over the standard Student’s t-test because it does not assume equal variance between the recent observation window and the baseline window — an assumption commonly violated in KPI degradation scenarios where the degradation period is characterised by increased variance [5].

The test statistic is computed as:

$$t = \frac{\bar{x}_{\text{recent}} - \bar{x}_{\text{baseline}}}{\sqrt{\frac{s_{\text{recent}}^2}{n_{\text{recent}}} + \frac{s_{\text{baseline}}^2}{n_{\text{baseline}}}}} \quad (2.1)$$

The resulting p-value is used to assign a statistical confidence label — High, Medium, or Low — to each detected degradation. Critically, the p-value is treated as advisory evidence rather than as a hard gate: cells with operationally significant degradation magnitudes are retained in the output even when statistical confidence is Low due to insufficient sample count, reflecting the engineering requirement to avoid missing potentially critical network issues.

2.3.2 Baseline Computation Methods

The choice of baseline computation method significantly affects the sensitivity and specificity of degradation detection. The RANPilotAI platform implements two baseline modes: Same Weekdays – Last Week, which computes the baseline from the corresponding weekday in the immediately preceding week; and Historical Weekday Median, which computes the baseline from the median of the same weekday across a configurable number of lookback weeks.

The Historical Weekday Median mode provides greater robustness against week-to-week variability by filtering out transient anomalies and special events that may have affected a single baseline week. This mode is particularly valuable in operational environments where traffic patterns exhibit strong weekly periodicity but may be disrupted by holidays, promotional events, or maintenance activities.

2.4 Machine Learning for Time Series Forecasting

2.4.1 Classical Time Series Methods

Holt-Winters exponential smoothing represents the classical approach to seasonal time series forecasting and serves as the statistical benchmark in the RANPilotAI KPI Forecasting Engine. The method decomposes a time series into level, trend, and seasonal components, updating each component via exponential smoothing as new observations become available [6]. The additive and multiplicative seasonal variants accommodate different growth patterns in the underlying series, while the damped trend variant prevents unrealistic extrapolation for long-horizon forecasts.

Hyndman and Athanasopoulos provide a comprehensive treatment of forecasting principles, including the evaluation metrics adopted in the RANPilotAI platform: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE) [6]. Their analysis of baseline forecasting methods establishes the naive and seasonal naive benchmarks against which more sophisticated models must justify their complexity.

2.4.2 Gradient Boosting for Non-linear Temporal Modelling

XGBoost gradient boosting represents the machine learning component of the RANPilotAI forecasting ensemble. Gradient boosting constructs an additive model by sequentially fitting decision trees to the residuals of the current ensemble, with each tree correcting the errors of its predecessors [7]. The method handles non-linear relationships, feature interactions, and heterogeneous feature types without requiring explicit feature scaling or transformation.

The critical challenge in applying gradient boosting to network KPI forecasting is preventing data leakage — the inadvertent use of future information during model training. The RANPilotAI platform addresses this through leakage-safe feature engineering in which all predictive features are computed using only observations available at the forecast origin, ensuring that model performance estimates are valid for operational deployment.

2.5 Root Cause Analysis in Telecommunications

2.5.1 Rule-Based Root Cause Inference

Root cause analysis (RCA) in telecommunications networks has traditionally relied on rule-based expert systems that encode domain knowledge as conditional inference rules. The RANPilotAI platform implements an RF-aware RCA engine that applies over 140 correlated detection rules to classify degraded cells into eight operational patterns: Outage, Congestion, Radio Quality, Coverage, Interference, Mobility, Demand, and Unknown.

The composite scoring mechanism combines four additive components: base severity weight assigned to the diagnostic counter type, RF-layer priority bonus reflecting higher diagnostic priority of physical layer counters, threshold score increment applied when counter values exceed predefined engineering thresholds, and magnitude score component proportional to the absolute deviation of the counter from its expected value.

2.5.2 Data Quality and Artefact Handling

Real-world KPI exports contain numerous data quality artefacts that must be detected and handled before statistical analysis. The RANPilotAI platform implements a data quality framework that detects sentinel values — large integer constants used by Huawei systems to indicate counter unavailability — negative values in non-negative counters, values exceeding theoretical bounds for percentage KPIs, and cells with incomplete observation windows. These artefacts are filtered or imputed before analysis to prevent incorrect degradation magnitude computation and false root cause assignments.

2.6 Retrieval-Augmented Generation for Engineering Domains

2.6.1 Information Retrieval Foundations

The Telecom RAG Assistant builds upon classical information retrieval principles described by Manning et al. [8]. The fundamental challenge in technical document retrieval is the vocabulary mismatch between user queries and indexed document content. Semantic embedding models address this by mapping both queries and documents into a shared vector space where semantically related text segments are positioned near each other regardless of exact term overlap.

2.6.2 Neural Re-ranking and Anti-hallucination

The RANPilotAI RAG pipeline employs cross-encoder re-ranking to improve retrieval precision by scoring query-document pairs with a bidirectional transformer model. This

two-stage retrieval architecture — coarse semantic retrieval followed by precise re-ranking — balances computational efficiency with retrieval accuracy. Strict anti-hallucination prompt engineering ensures that generated responses are grounded exclusively in retrieved document excerpts, preventing the language model from introducing unsupported technical claims [9].

2.7 Software Architecture for Integrated Platforms

2.7.1 Modular Architecture Patterns

The RANPilotAI platform adopts a modular architecture in which the Telecom Hub functions as a central coordination point while individual analytical modules operate as independent subsystems. Bass et al. describe the architectural principles underlying such modular designs, including the separation of concerns, information hiding, and the management of cross-cutting concerns such as logging, authentication, and data persistence through shared infrastructure components [10].

Gamma et al.’s catalog of design patterns informs the implementation of module communication through standardized interfaces, enabling new analytical modules to be integrated without modifying existing platform infrastructure [11]. The Strategy pattern is applied to model selection in the forecasting engine, allowing new prediction models to be incorporated through a common evaluation interface. The Observer pattern supports real-time progress reporting during long-running analyses, and the Facade pattern simplifies module interaction by presenting a consistent API surface to the Telecom Hub.

2.7.2 Web-Based Deployment Architectures

Sommerville’s treatment of software engineering establishes the principles guiding the platform’s web-based deployment architecture [12]. The decision to implement the platform as a browser-accessible web application rather than a desktop application addresses several operational requirements: platform independence across Windows, macOS, and Linux workstations; simplified deployment through a single Python process rather than per-workstation installation; and the ability to serve multiple engineers from a shared server instance.

Pressman and Maxim’s practitioner-oriented approach to software engineering reinforces the importance of maintainability, testability, and scalability in the platform design [13]. The adoption of type-annotated Python, comprehensive docstrings, and separation of analytical logic from presentation logic ensures that the platform can be extended and maintained by future development teams without requiring architectural redesign.

2.8 Summary

This chapter has surveyed the research literature relevant to the development of the RANPilotAI platform. The review has established the theoretical foundations for statistical degradation detection through Welch’s t-test and baseline computation methodologies, machine learning-based time-series forecasting using Holt-Winters exponential smoothing and XGBoost gradient boosting, rule-based root cause analysis with RF-aware severity

weighting, retrieval-augmented generation for technical document consultation, and modular software architecture patterns for integrated engineering platforms.

The literature review identifies a clear gap in existing tooling: the absence of an integrated platform that combines statistically rigorous degradation detection, predictive forecasting, configuration impact analysis, and standards consultation within a unified engineering environment. The RANPilotAI platform addresses this gap by implementing each of these capabilities as a dedicated module within a modular, web-based architecture guided by established software engineering principles.

Chapter 3 presents the detailed system design of the RANPilotAI platform, translating the requirements and theoretical foundations established in this chapter into concrete software architecture, technology selections, data management strategies, and module integration interfaces.

Chapter 3

Overall System Design

The RANPilotAI platform adopts a modular software architecture in which four independent analytical modules operate within a unified web-based engineering environment. Rather than implementing a monolithic application, the platform separates domain-specific analytical processing from common infrastructure services, ensuring that each module can be developed, tested, and deployed independently while presenting a consistent user experience through the central Telecom Hub interface.

The platform is implemented entirely in Python 3.11 using the Streamlit framework for web-based user interaction. Data processing leverages the Pandas and NumPy scientific computing ecosystem, while machine learning components are built upon Scikit-learn, XGBoost, and Statsmodels. Retrieval-augmented generation capabilities are provided through sentence-transformers, Qdrant vector database, and OpenRouter language model APIs. Version-controlled data management is provided through Dolt, a SQL database with Git-like versioning semantics. The modular architecture ensures that new analytical modules can be incorporated without redesigning the platform from the ground up.

3.1 Introduction

The design of the RANPilotAI platform was guided by four principal objectives: modularity, maintainability, usability, and extensibility. Modularity requires that each analytical module encapsulate its own processing logic and data dependencies, enabling independent development and testing. Maintainability demands that the codebase follow consistent engineering conventions, include comprehensive documentation, and separate analytical logic from presentation logic. Usability requires that the graphical interface present complex analytical capabilities through an intuitive workflow that minimises the learning curve for RF engineers. Extensibility ensures that new analytical modules, data sources, and machine learning models can be incorporated without architectural redesign.

Figure 3.1 presents the navigation interface of the Telecom Hub, illustrating how the platform exposes its four analytical modules through a unified selection mechanism. The interface enables engineers to switch between modules without losing analysis context, with each module maintaining its own session state and data artefacts.

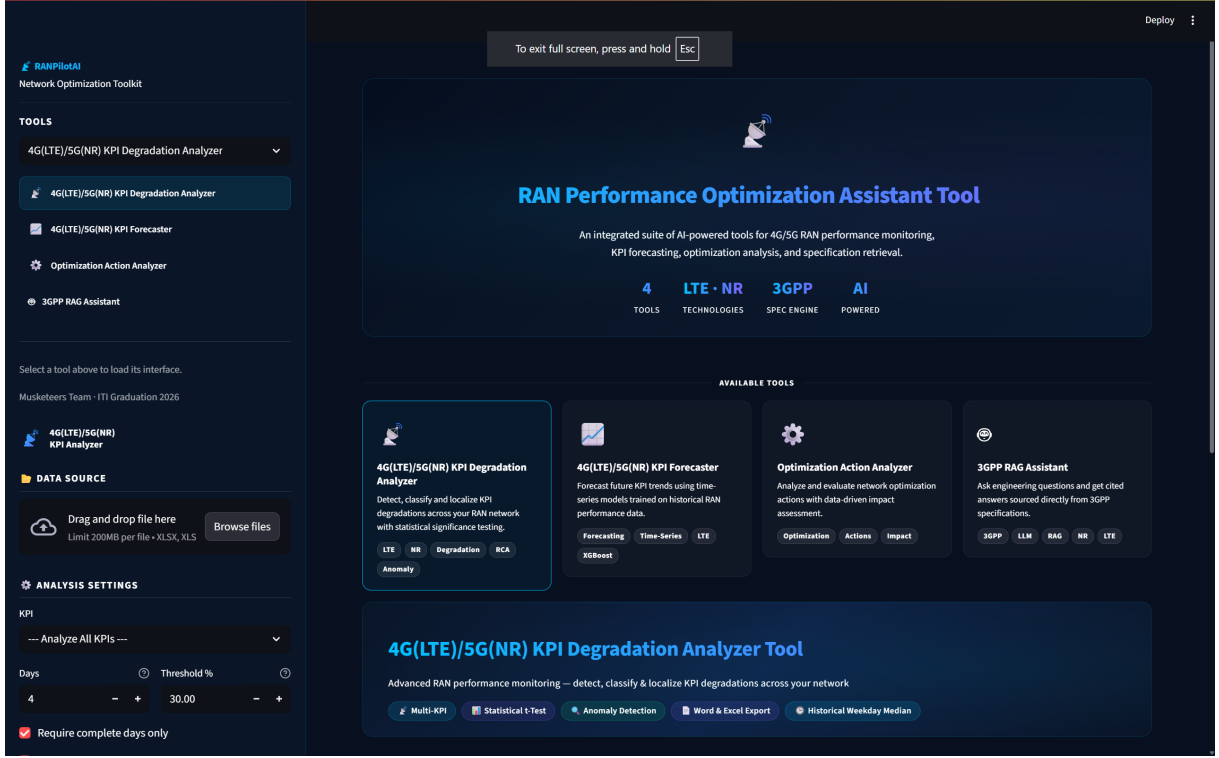


Figure 3.1: Navigation interface of the RANPilotAI Telecom Hub presenting the four selectable analytical modules. The interface provides consistent access to KPI Degradation Analyzer, KPI Forecasting Engine, Optimization Action Analyzer, and Telecom RAG Assistant through a unified module selection paradigm.

3.2 Design Objectives and Constraints

3.2.1 Functional Objectives

The platform must support the complete RAN optimization workflow within a single environment. This includes uploading and validating KPI counter exports, configuring analysis parameters, executing degradation detection and forecasting computations, visualising results through interactive charts and tables, correlating configuration changes with KPI impact, and consulting 3GPP Technical Specifications using natural language queries. Each analytical module must operate independently while sharing common data services and presenting results through a consistent interface paradigm.

3.2.2 Non-Functional Objectives

The platform must be accessible through a standard web browser without requiring client-side software installation, operate on a single workstation without distributed computing infrastructure, and maintain interactive response times for UI operations. Analysis computations that require significant processing time must provide progress indication and allow cancellation. Data persistence must ensure that analysis results are retained across sessions, and export functionality must support integration with existing operational reporting workflows.

3.2.3 Technical Constraints

The platform is constrained to Python 3.11 for implementation consistency across all modules. Third-party dependencies are limited to packages available through standard Python package repositories with permissive licensing. The deployment target is a single Windows or Linux workstation with 16 GB RAM and a multi-core processor, sufficient to process datasets containing up to one million rows and five hundred columns without requiring cloud infrastructure or distributed computing frameworks.

3.3 Overall Platform Architecture

The RANPilotAI platform follows a layered modular architecture comprising four principal components: the Telecom Hub, which coordinates user interaction and module routing; the Engineering Modules, which implement the four analytical capabilities; the Shared Components, which provide common functionality reused across modules; and the Platform Resources, which manage data persistence, configuration, and external service integration.

The Telecom Hub functions as the central access point. Rather than interacting directly with individual processing engines, users communicate exclusively with the hub, which manages application routing, session handling, and interaction with underlying analytical components. This centralised execution model simplifies software maintenance, reduces coupling between modules, and establishes a consistent execution model throughout the platform.

The Engineering Modules operate as independent subsystems connected through the Telecom Hub via standardized interfaces. Each module encapsulates its own processing logic, data models, and visualisation components while relying on shared components for common operations such as file upload handling, progress indication, and result export.

Table 3.1: Platform components and their primary responsibilities within the RANPilotAI architecture.

Component	Responsibilities
Telecom Hub	Module routing, session management, unified navigation, shared UI components
KPI Degradation Analyzer	Data quality validation, degradation detection, RF-aware root cause analysis, export generation
KPI Forecasting Engine	Time-series decomposition, feature engineering, model training and evaluation, forecast generation
Optimization Action Analyzer	Configuration tracking, action hunk generation, KPI impact quantification, timeline visualisation
Telecom RAG Assistant	PDF processing, semantic indexing, vector retrieval, re-ranking, LLM prompt construction
Shared Components	File handling, progress indication, data export utilities, logging and error management
Platform Resources	Dolt database, vector store, configuration files, exported results persistence

Table 3.1 summarises the platform components and their primary responsibilities. The separation between engineering modules and shared infrastructure ensures that analytical logic can evolve independently of platform services, while the centralised hub ensures that users experience a consistent interface across all modules.

3.4 Software Architecture and Design Patterns

3.4.1 Layered Architecture

The platform adopts a layered architecture that separates user interaction, application logic, engineering modules, and data management into distinct software layers [12]. The presentation layer is implemented through Streamlit components and custom CSS styling, providing responsive layout management and interactive visualisations. The application logic layer contains the Telecom Hub orchestration code, session management, and module routing logic. The engineering module layer contains the analytical processing pipelines for each module. The data management layer abstracts database access, file system operations, and external service communication.

This separation improves maintainability by ensuring that each layer is responsible for a well-defined set of functionalities while communicating only with adjacent layers. Changes to the user interface do not affect analytical processing logic, and modifications to data storage mechanisms do not require changes to module orchestration code.

3.4.2 Design Patterns

The platform implements several established design patterns to improve code organisation and extensibility [11].

The Strategy pattern is applied to model selection in the KPI Forecasting Engine, allowing new forecasting models to be incorporated through a common evaluation interface. Each model implements the standard interface methods — `train`, `predict`, and `evaluate` — enabling the model selection protocol to compare arbitrary candidate models without conditional logic.

The Observer pattern supports real-time progress reporting during long-running analyses. Each analytical module emits progress events during computation, which the Telecom Hub captures and forwards to the Streamlit UI for display. This decouples analysis progress tracking from the specific implementation of each module.

The Facade pattern simplifies module interaction by presenting a consistent API surface to the Telecom Hub. Each engineering module exposes a standard set of entry points — `load_data`, `configure`, `run_analysis`, and `get_results` — which the hub invokes without requiring knowledge of the module’s internal processing pipeline.

3.4.3 Data Flow Architecture

The overall data flow within the RANPilotAI platform follows a unidirectional pipeline pattern. User inputs are captured by the Telecom Hub and forwarded to the selected engineering module. The module processes the input through its analytical pipeline, persisting intermediate results to the platform data layer. Completed results are returned to the hub for presentation through the Streamlit interface. Export operations are triggered by user request and routed to the appropriate export generator.

This architecture ensures that data flows in a single direction, simplifying debugging and ensuring that analysis state is fully reproducible from the original input data and configuration parameters.

3.5 Technology Stack

3.5.1 Backend Framework

The platform is implemented in Python 3.11, selected for its extensive scientific computing ecosystem, strong type annotation support, and widespread adoption in data engineering and machine learning communities. The Streamlit framework provides the web-based user interface, offering rapid development of interactive data applications through a declarative programming model. Streamlit’s automatic reloading, session state management, and component library accelerate frontend development while maintaining acceptable performance for the platform’s analytical workloads.

3.5.2 Data Processing

Pandas provides the primary data manipulation infrastructure, supporting the loading, transformation, and aggregation of KPI datasets containing hundreds of columns and millions of rows. NumPy underpins numerical computations including statistical tests, rolling window operations, and matrix algebra required for feature engineering and model

evaluation. The platform leverages Pandas’ datetime indexing and groupby operations extensively for time-series analysis and weekday-matched baseline computation.

3.5.3 Machine Learning

Scikit-learn provides preprocessing utilities, evaluation metrics, and baseline model implementations used across multiple modules. XGBoost implements the gradient boosting regression models used in the KPI Forecasting Engine, providing efficient training through histogram-based algorithm variants and GPU acceleration support. Statsmodels provides the Holt-Winters exponential smoothing implementation and statistical test functions including Welch’s t-test.

3.5.4 Retrieval-Augmented Generation

Sentence-transformers provides the BGE-large embedding model used to convert text chunks and user queries into dense vector representations. Qdrant serves as the local vector database, storing indexed document embeddings and supporting semantic similarity search with metadata filtering. The CrossEncoder from the sentence-transformers library implements the neural re-ranking stage, rescoring initial retrieval candidates to improve precision. OpenRouter provides unified API access to large language models including Llama and Gemma variants, enabling the platform to generate engineering responses grounded in retrieved specification excerpts.

3.5.5 Data Persistence

Dolt provides version-controlled SQL database functionality, enabling the Optimization Action Analyzer to track configuration commit history, query historical KPI snapshots, and compute before/after comparisons using Git-like branching and diff semantics. SQLite serves as the default persistence backend for smaller datasets and configuration state, while Parquet files provide efficient columnar storage for large KPI exports. The platform uses Apache ECharts through the Streamlit native charting interface for interactive visualisations.

3.6 Data Management Strategy

3.6.1 KPI Data Schema

The platform defines a canonical KPI data schema that maps vendor-specific counter names to standardised internal identifiers. This schema normalises column naming conventions, units, and measurement directions across different network management system exports, enabling the analytical modules to operate on heterogeneous data sources without modification. The schema includes metadata for each KPI category specifying the performance pillar, measurement direction, theoretical bounds, and associated root cause detection rules.

3.6.2 Configuration Change Tracking

The Optimization Action Analyzer implements a configuration change tracking system built on Dolt’s versioning semantics. Each configuration modification — including antenna tilt adjustments, transmit power changes, and handover parameter updates — is recorded as a Dolt commit with associated metadata specifying the cell identifier, parameter name, previous value, new value, timestamp, and author. This commit history enables the module to align configuration events with KPI time windows and compute the causal impact of individual changes or coordinated change campaigns.

3.6.3 Vector Index Management

The Telecom RAG Assistant maintains a persistent vector index of processed 3GPP specification documents. The indexing pipeline processes PDF files through text extraction, section-aware chunking, embedding generation, and vector storage. The index is rebuilt incrementally when specification documents are updated, ensuring that the retrieval corpus remains current with the latest 3GPP releases. Metadata associated with each chunk — including specification number, release version, section identifier, and page number — is stored alongside the embedding vector to support citation generation and specification filtering.

3.7 Module Interfaces and Integration

3.7.1 Telecom Hub Interface

The Telecom Hub defines a standard module interface contract that each analytical module must implement. The contract specifies four principal methods: `load_data` for ingesting user-uploaded files, `configure` for applying user-specified analysis parameters, `run_analysis` for executing the analytical pipeline, and `get_results` for retrieving completed analysis outputs. Additional optional methods support progress reporting, cancellation, and result export.

This interface contract enables the hub to orchestrate module execution through a uniform mechanism regardless of the specific analytical processing implemented within each module. New modules can be integrated by implementing the standard interface without modifying hub orchestration logic.

3.7.2 Session State Management

Streamlit’s session state mechanism provides the foundation for multi-module workflow continuity. Each module maintains its own session state namespace, storing loaded data, configuration parameters, analysis results, and visualisation selections. The Telecom Hub coordinates module switching by preserving session state across module transitions, enabling engineers to perform degradation analysis, switch to the forecasting module to predict future trajectories, and return to the degradation module with analysis context intact.

3.7.3 Data Sharing Between Modules

The platform supports controlled data sharing between modules through the platform’s shared data layer. The KPI Degradation Analyzer can export degraded cell lists that serve as input to the KPI Forecasting Engine, enabling engineers to focus forecasting computation on cells that have already been identified as degraded. Similarly, the Optimization Action Analyzer can reference degraded cell identifiers from the degradation module to focus configuration impact analysis on cells experiencing active performance issues.

3.8 Deployment Architecture

3.8.1 Single-Workstation Deployment

The platform is designed for deployment on a single workstation without distributed computing infrastructure. The Streamlit web server runs locally, serving the user interface through a web browser on the same machine or on connected devices within the local network. This deployment model prioritises simplicity, portability, and data locality — all KPI data and analysis results remain on the operator’s workstation, addressing data security and regulatory compliance requirements.

3.8.2 Scalability Considerations

While the current deployment model operates on a single workstation, the modular architecture enables horizontal scaling without architectural redesign. Each analytical module can be deployed as an independent service communicating with the Telecom Hub through network APIs, allowing compute-intensive modules to be hosted on dedicated servers while maintaining the unified user interface. The version-controlled data backend provided by Dolt supports multi-user collaboration through its branching and merging semantics, enabling teams of engineers to work on parallel analysis investigations without interfering with each other’s data.

3.9 Summary

This chapter has presented the overall system design of the RANPilotAI platform. The platform adopts a modular, layered architecture in which four independent analytical modules operate within a unified web-based engineering environment coordinated by the Telecom Hub. The design prioritises modularity, maintainability, usability, and extensibility, ensuring that the platform can evolve to incorporate new analytical capabilities without architectural redesign.

The technology stack combines established open-source libraries for data processing, machine learning, retrieval-augmented generation, and web-based user interface development. The data management strategy provides persistent, version-controlled storage for KPI datasets, configuration histories, and document indices. The module integration architecture defines standardised interfaces that enable the Telecom Hub to orchestrate analytical workflows across heterogeneous processing engines.

The subsequent chapters describe each analytical module in detail. Chapter 4 presents the KPI Degradation Analyzer, Chapter 5 presents the KPI Forecasting Engine, Chap-

ter 6 presents the Optimization Action Analyzer, and Chapter 7 presents the Telecom RAG Assistant. Each chapter follows the structure established in Chapter 1: module introduction, operational motivation, technical problem formulation, background theory, configuration and results presentation, and summary.

Chapter 4

KPI Degradation Analyzer

The KPI Degradation Analyzer is the first and most comprehensive analytical module of the RANPilotAI platform. It is designed to automatically detect, classify, and localise Key Performance Indicator degradations across LTE and 5G NR Radio Access Networks using a statistically-aware, three-layer analytical pipeline. The module processes tabular KPI counter exports, applies data quality validation, computes degradation magnitudes against historical baselines, and performs RF-aware root cause analysis to produce actionable engineering output for each degraded cell.

The module supports 13 KPI categories spanning all five standard pillars of RAN performance: accessibility, retainability, mobility, integrity, and availability. The root cause analysis engine applies over 140 correlated detection rules encoding domain-specific RF engineering knowledge, producing structured output that includes degradation severity classification, statistical confidence scoring, root cause pattern identification, and recommended next investigation steps.

4.1 Introduction

Modern LTE and 5G NR base stations generate performance management counter data at regular intervals, typically every 15 minutes or hourly. For a network of even moderate size, this results in datasets containing tens of thousands of rows and hundreds of columns, with each cell contributing observations across dozens of KPI categories [2]. Identifying which cells are degraded, quantifying the severity of the degradation, and determining the probable root cause requires correlating the degraded KPI against a set of related diagnostic counters using engineering expertise that is difficult to systematise and impossible to apply consistently at network scale through manual methods.

The KPI Degradation Analyzer addresses this challenge by implementing a fully automated three-layer analytical pipeline that processes raw KPI counter exports and produces structured degradation reports without requiring manual cell-by-cell inspection. The module is implemented as a Streamlit-based interactive application that enables engineers to configure analysis parameters, upload data, initiate analysis, and explore results through an interactive tabular and graphical interface.

Figure 4.1 presents the initial state of the KPI Degradation Analyzer interface before data is loaded. The interface displays the tool header with its four capability tags, the sidebar data source upload panel, and the four analytical capability cards describing the available analysis functions.

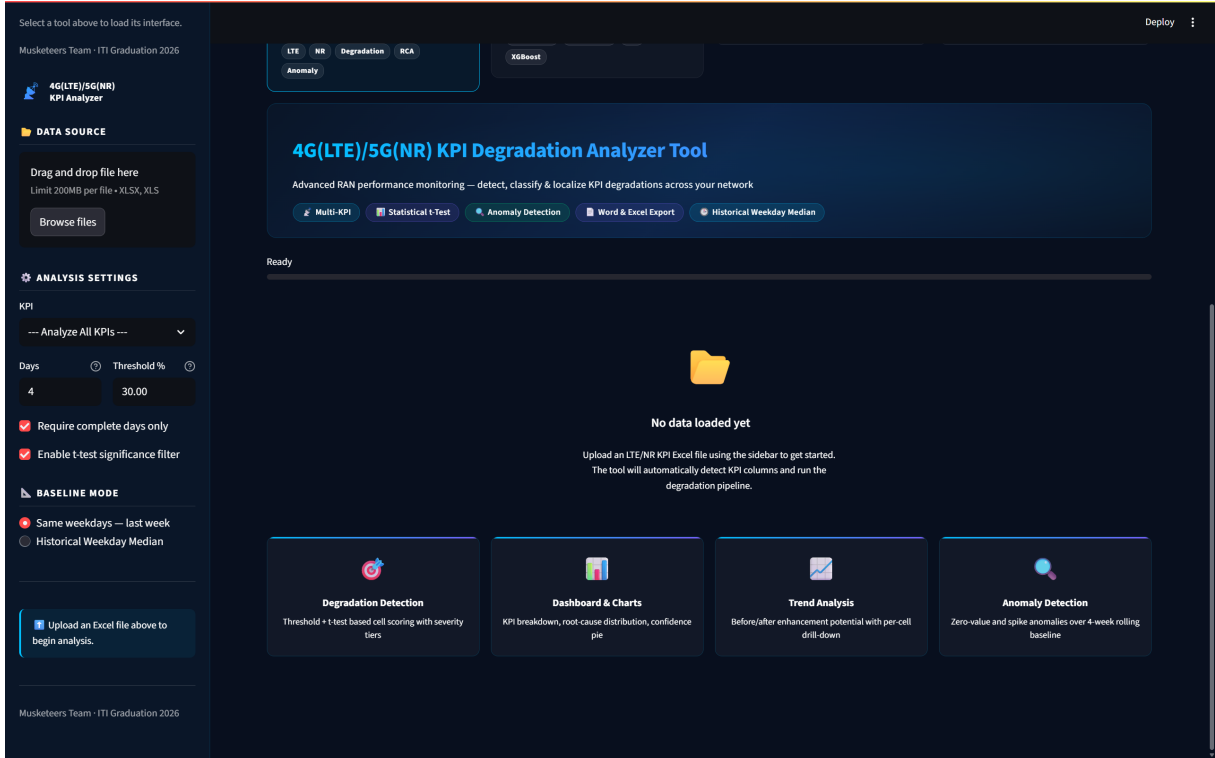


Figure 4.1: Initial interface of the KPI Degradation Analyzer module showing the tool header, sidebar data source and analysis settings panel, and the four capability cards: Degradation Detection, Dashboard & Charts, Trend Analysis, and Anomaly Detection.

4.2 Motivation

The operational motivation for developing an automated KPI degradation detection module arises directly from the limitations of existing manual analysis workflows in commercial network operations centres.

In a typical manual workflow, an RF engineer downloads a weekly KPI report from the network management system, opens it in a spreadsheet application, applies filters and conditional formatting to identify cells exceeding predefined thresholds, and then manually examines the counter values for each flagged cell to determine the root cause. This process is time-consuming, requires sustained concentration, and relies entirely on the individual engineer’s domain knowledge and experience to produce consistent root cause classifications.

Furthermore, threshold-based alerting — the primary automated mechanism available in most commercial network management systems — suffers from two fundamental limitations. First, a fixed threshold applied uniformly across all cells ignores the natural variability and baseline performance level of each individual cell. A threshold of 95% for RRC Setup Success Rate may correctly flag a degraded urban macro cell while missing a structural degradation in a rural small cell with a historically lower baseline. Second, threshold crossing alone provides no information about the probable root cause of the degradation, leaving the full diagnostic burden on the engineer.

The KPI Degradation Analyzer replaces this manual workflow with an automated pipeline that applies cell-specific historical baselines, computes statistically-aware degradation magnitudes, and correlates degraded KPIs against RF-aware root cause patterns

to produce a complete, structured analysis report in seconds.

4.3 Problem Statement

The technical challenge addressed by the KPI Degradation Analyzer encompasses three distinct sub-problems.

Data quality: Real-world KPI exports from vendor network management systems contain numerous data quality artefacts that must be detected and handled before statistical analysis can be applied. These include sentinel values — large integer constants such as 4,294,967,295 and 4,294,967,294 used by Huawei systems to indicate counter unavailability — negative values in counters that should be non-negative, values exceeding theoretical bounds for percentage KPIs, cells with incomplete observation windows, and mixed units within a single column due to vendor export configurations. Failure to detect and remove these artefacts before analysis produces incorrect degradation magnitudes and false root cause assignments.

Degradation quantification: Different KPI types require different mathematical treatment when computing degradation magnitude. Counter KPIs such as traffic volume and throughput are best expressed as relative percentage changes from baseline: $(B - R)/B \times 100$, where B is the baseline mean and R is the recent mean. Ratio KPIs such as RRC Setup Success Rate, E-RAB Drop Rate, and Handover Success Rate are better expressed as signed percentage-point differences: $R - B$, which avoids divide-by-zero conditions when the baseline approaches zero and provides a physically more meaningful magnitude for binary success rate metrics.

Root cause attribution: Determining the probable root cause of a detected KPI degradation requires correlating the degraded KPI against a specific set of related diagnostic counters. The appropriate set of diagnostic counters, the directionality of the correlation, and the severity weighting assigned to each correlated counter depend on the KPI category, the direction of degradation, the RF technology, and the deployment context. Encoding this domain knowledge as a systematic, extensible rule set that produces consistent results across all cells and KPI categories is the central engineering challenge of the root cause analysis engine.

4.4 Background

4.4.1 KPI Categories and Performance Pillars

The KPI Degradation Analyzer monitors 13 KPI categories aligned with the five standard performance pillars used in LTE and 5G NR network assessment. Table 4.1 summarises the monitored KPI categories, their associated performance pillars, and the direction of degradation for each.

Table 4.1: KPI categories monitored by the KPI Degradation Analyzer.

#	KPI Category	Pillar	Bad Direction
1	DL Traffic Volume	Traffic	Low
2	UL Traffic Volume	Traffic	Low
3	DL Throughput	Integrity	Low
4	UL Throughput	Integrity	Low
5	RRC Setup SR	Accessibility	Low
6	E-RAB Setup SR	Accessibility	Low
7	E-RAB Drop Rate	Retainability	High
8	HO Success Rate	Mobility	Low
9	Cell Availability	Availability	Low
10	RACH Success Rate	Accessibility	Low
11	CSFB KPIs	Accessibility	Low
12	VoLTE KPIs	Integrity	Varies
13	RRC Re-establishment	Retainability	High

4.4.2 Welch’s t-test for Degradation Confidence

The module employs Welch’s two-sample t-test as an advisory statistical confidence measure when comparing recent KPI observations against the historical baseline [5]. Welch’s test is preferred over the standard Student’s t-test because it does not assume equal variance between the recent observation window and the baseline window, which is a commonly violated assumption in KPI degradation scenarios where the degradation period is characterised by increased variance.

The test statistic is computed as:

$$t = \frac{\bar{x}_{\text{recent}} - \bar{x}_{\text{baseline}}}{\sqrt{\frac{s_{\text{recent}}^2}{n_{\text{recent}}} + \frac{s_{\text{baseline}}^2}{n_{\text{baseline}}}}} \quad (4.1)$$

The resulting p-value is used to assign a statistical confidence label — High, Medium, or Low — to each detected degradation. Critically, the p-value is treated as advisory evidence rather than as a hard gate: cells with operationally significant degradation magnitudes are retained in the output even when the statistical confidence is Low due to insufficient sample count in the observation window.

4.4.3 RF-Aware Root Cause Scoring

The root cause analysis engine assigns a composite score to each candidate root cause for a degraded cell using four additive components:

$$\text{Score} = w_{\text{severity}} + w_{\text{RF_priority}} + w_{\text{threshold}} + \min(w_{\text{magnitude}}, w_{\text{cap}}) \quad (4.2)$$

where w_{severity} is the base severity weight assigned to the diagnostic counter type, $w_{\text{RF_priority}}$ is a bonus applied to RF-layer counters reflecting their higher diagnostic priority, $w_{\text{threshold}}$ is a score increment applied when the counter value exceeds a predefined RF engineering threshold, $w_{\text{magnitude}}$ is a score component proportional to the absolute deviation of the counter from its expected value, and w_{cap} is an upper bound on the

magnitude component to prevent a single extremely deviant counter from dominating the composite score.

The eight operational RCA patterns produced by the engine are: Outage, Congestion, Radio Quality, Coverage, Interference, Mobility, Demand, and Unknown. Each pattern has a KPI-specific triage order that prioritises checking higher-confidence root cause hypotheses before less likely ones.

4.5 Analysis Configuration

Before initiating analysis, the engineer configures the analysis parameters through the sidebar panel. Figure 4.2 illustrates the analysis configuration state after a KPI data file has been uploaded and the analysis settings have been configured.

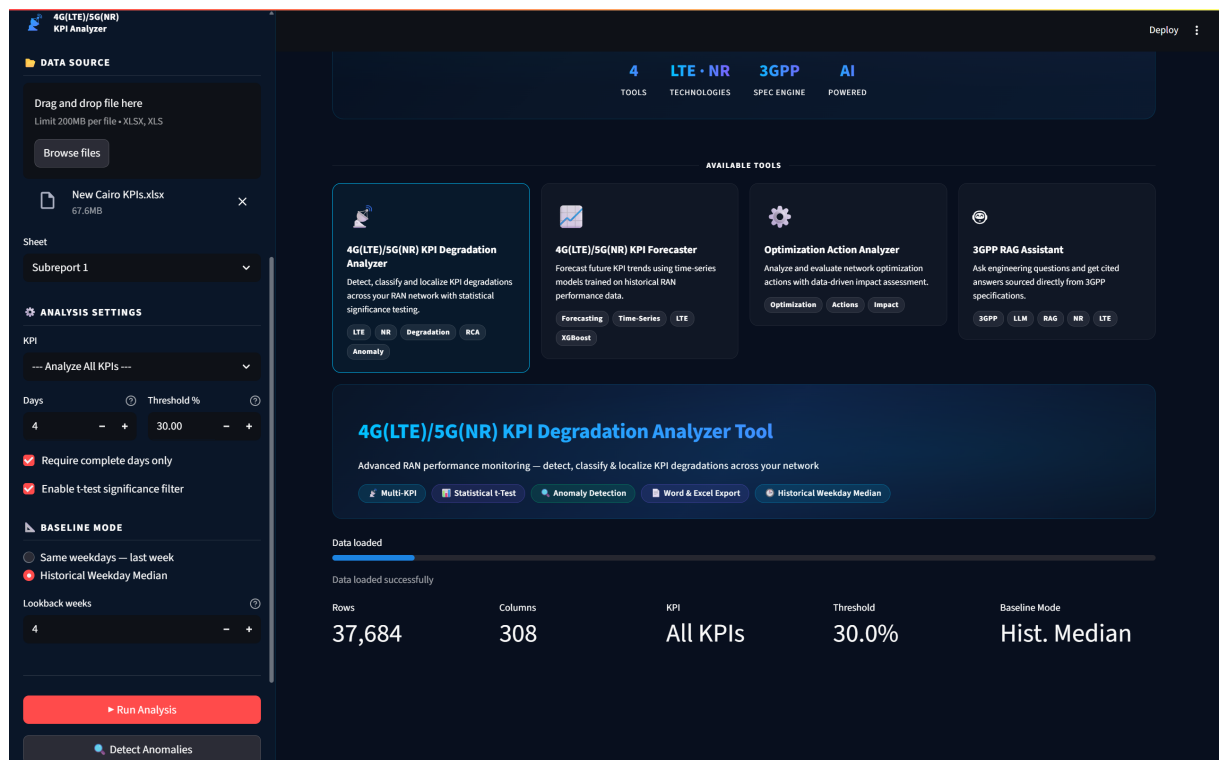


Figure 4.2: KPI Degradation Analyzer configuration panel after loading the New Cairo KPI dataset (37,684 rows, 308 columns). The sidebar shows the selected analysis settings: All KPIs, 4-day observation window, 30% degradation threshold, Historical Weekday Median baseline mode, and 4 lookback weeks.

The configurable analysis parameters are as follows:

- **KPI selection:** Engineers can choose to analyse all supported KPI categories simultaneously or restrict analysis to a specific KPI category.
- **Observation window:** The number of days in the recent observation window used for comparison against the baseline. A shorter window detects acute degradations while a longer window provides more stable statistical estimates.

- **Degradation threshold:** The minimum percentage degradation magnitude required to classify a cell-KPI combination as degraded. The default threshold of 30% balances detection sensitivity against false positive rate.
- **Complete days filter:** When enabled, cells with incomplete observation records for the specified window are excluded from analysis to prevent artefactual degradation detection due to missing data.
- **t-test significance filter:** When enabled, the Welch's t-test p-value is applied as an additional filter, retaining only degradations that pass a specified significance threshold.
- **Baseline mode:** Engineers select between two baseline computation methods. *Same Weekdays – Last Week* computes the baseline from the corresponding weekday in the immediately preceding week, providing sensitivity to acute changes. *Historical Weekday Median* computes the baseline from the median of the same weekday across the most recent N weeks (configurable via the lookback weeks parameter), providing a more stable long-term baseline that is robust to week-to-week variability.

4.6 Analysis Results and KPI Summary

Upon completion of the analysis pipeline, the module presents a structured summary of the detected degradations. Figure 4.3 shows the KPI Summary tab after a full analysis of the New Cairo dataset, which identified 784 total degraded cells across 13 KPI categories.

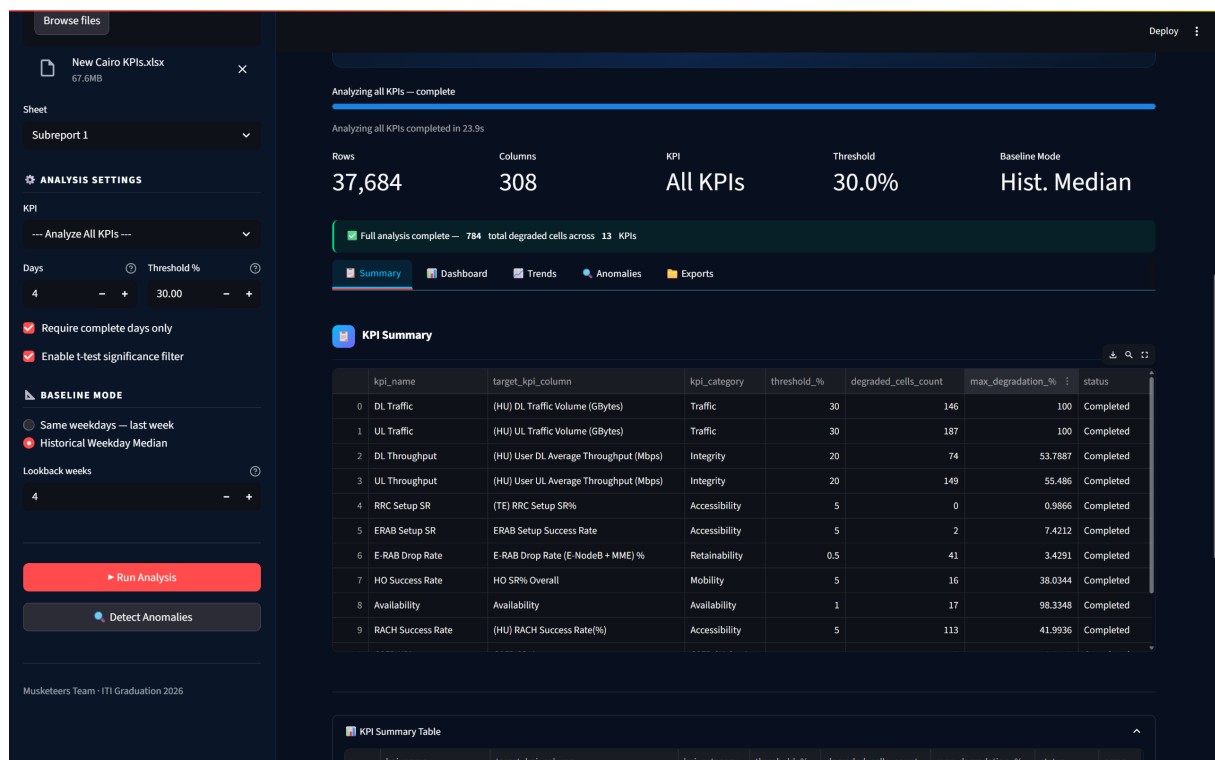


Figure 4.3: KPI Summary tab showing the analysis results for the New Cairo dataset (37,684 rows, 308 columns, All KPIs, 30% threshold, Historical Weekday Median baseline). The table lists each KPI category with its degraded cell count and maximum degradation percentage.

The KPI Summary table presents each monitored KPI category with the following fields:

- **kpi_name**: The human-readable name of the KPI category.
- **target_kpi_column**: The specific counter column name used for degradation computation.
- **kpi_category**: The performance pillar classification (Traffic, Integrity, Accessibility, Retainability, Mobility, Availability).
- **threshold_%**: The degradation threshold applied to this KPI category.
- **degraded_cells_count**: The number of cells classified as degraded for this KPI.
- **max_degradation_%**: The maximum observed degradation percentage across all degraded cells for this KPI.
- **status**: Completion status of the analysis for this KPI (Completed or Error).

4.7 Main Dashboard and Charts

The Dashboard tab presents an aggregated visual summary of the degradation analysis results. Figure 4.4 illustrates the main dashboard view.

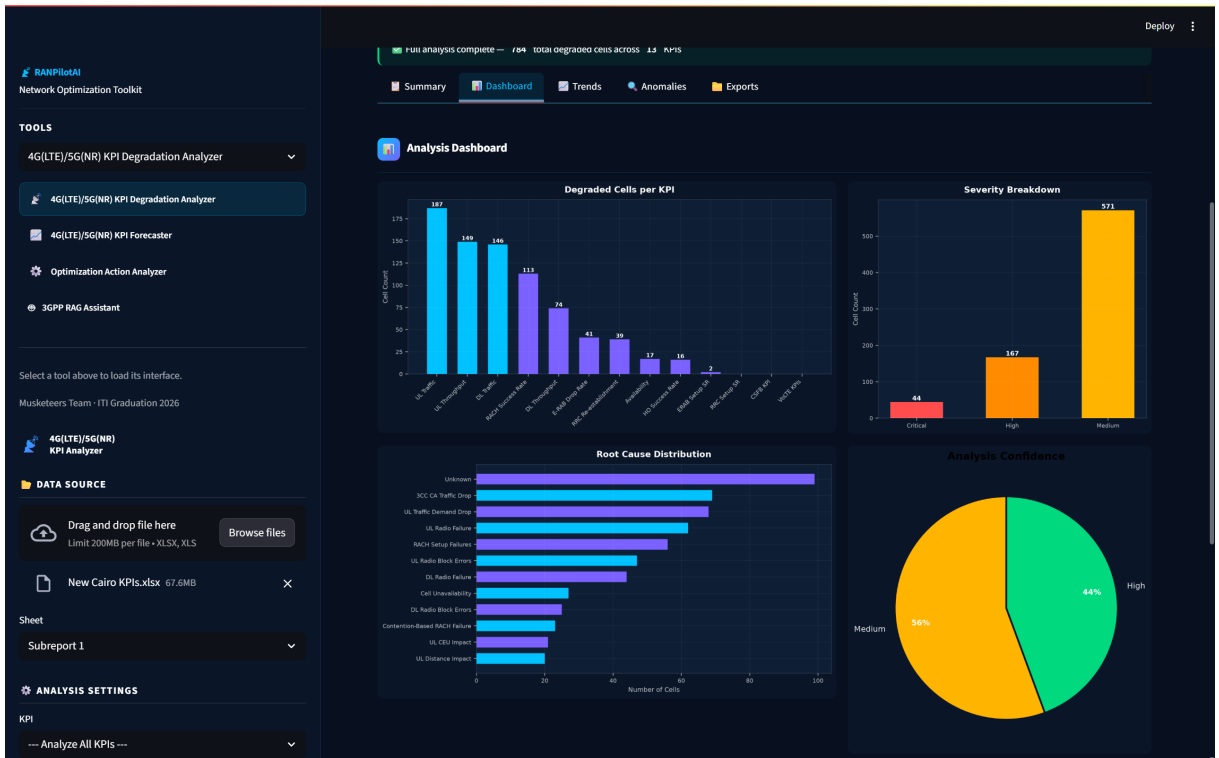


Figure 4.4: KPI Degradation Analyzer main dashboard presenting KPI-level degradation metrics, cell count distributions, and interactive charts for the New Cairo dataset analysis.

The dashboard provides multiple interactive chart types including:

- KPI-level degraded cell count bar charts enabling engineers to identify which KPI categories are most severely affected.
- Degradation magnitude distribution histograms showing the distribution of degradation percentages across degraded cells.
- Root cause pattern distribution pie charts illustrating the relative frequency of different RCA classifications across all degraded cells.
- Confidence level distribution charts showing the proportion of High, Medium, and Low confidence degradation detections.

4.8 Root Cause Analysis

4.8.1 Root Cause Dashboard

The root cause analysis output is presented through dedicated visualisation panels that enable engineers to drill down into specific cells and examine the RCA evidence in detail. Figure 4.5 presents the root cause analysis dashboard.

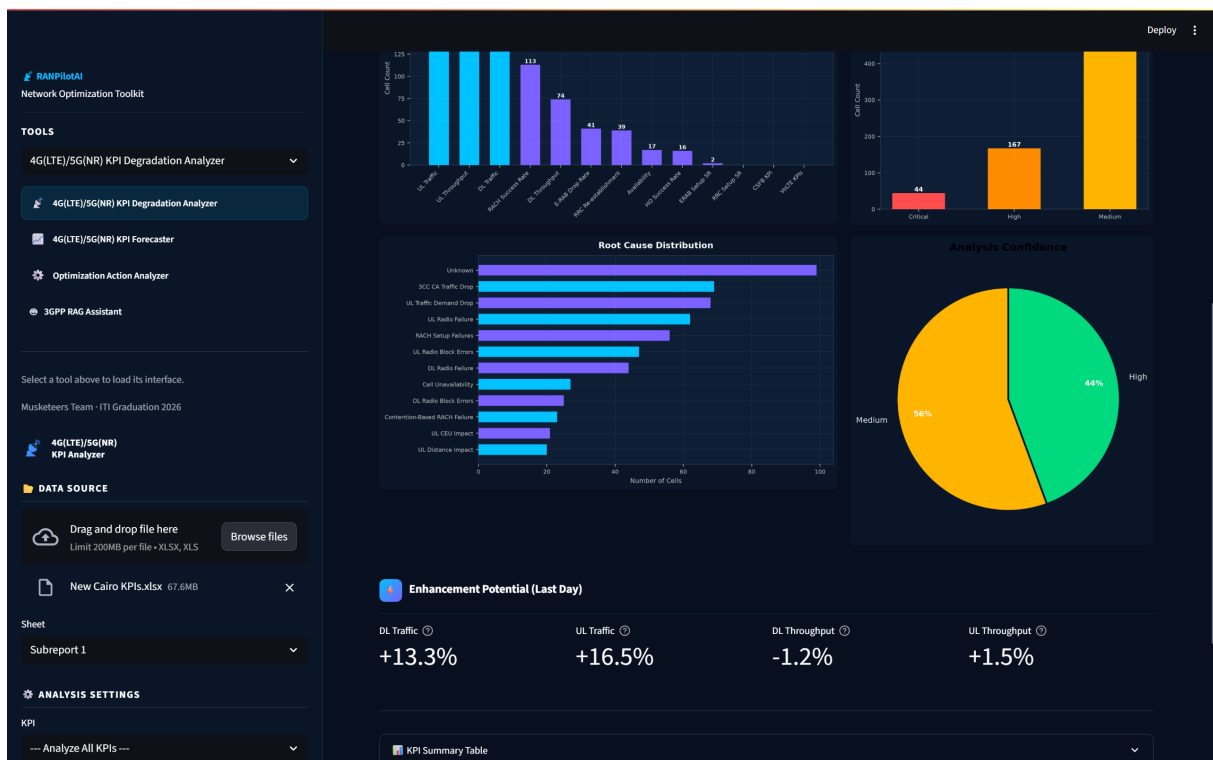


Figure 4.5: Root cause analysis dashboard showing the distribution of RCA pattern classifications, severity scores, and correlated diagnostic counter evidence across degraded cells in the New Cairo dataset.

4.8.2 Root Cause Recommendations

For each degraded cell, the RCA engine generates a structured recommendation that includes the primary root cause pattern, the supporting diagnostic evidence, and a set of

recommended next investigation steps. Figure 4.6 illustrates the root cause recommendation output for a representative degraded cell.

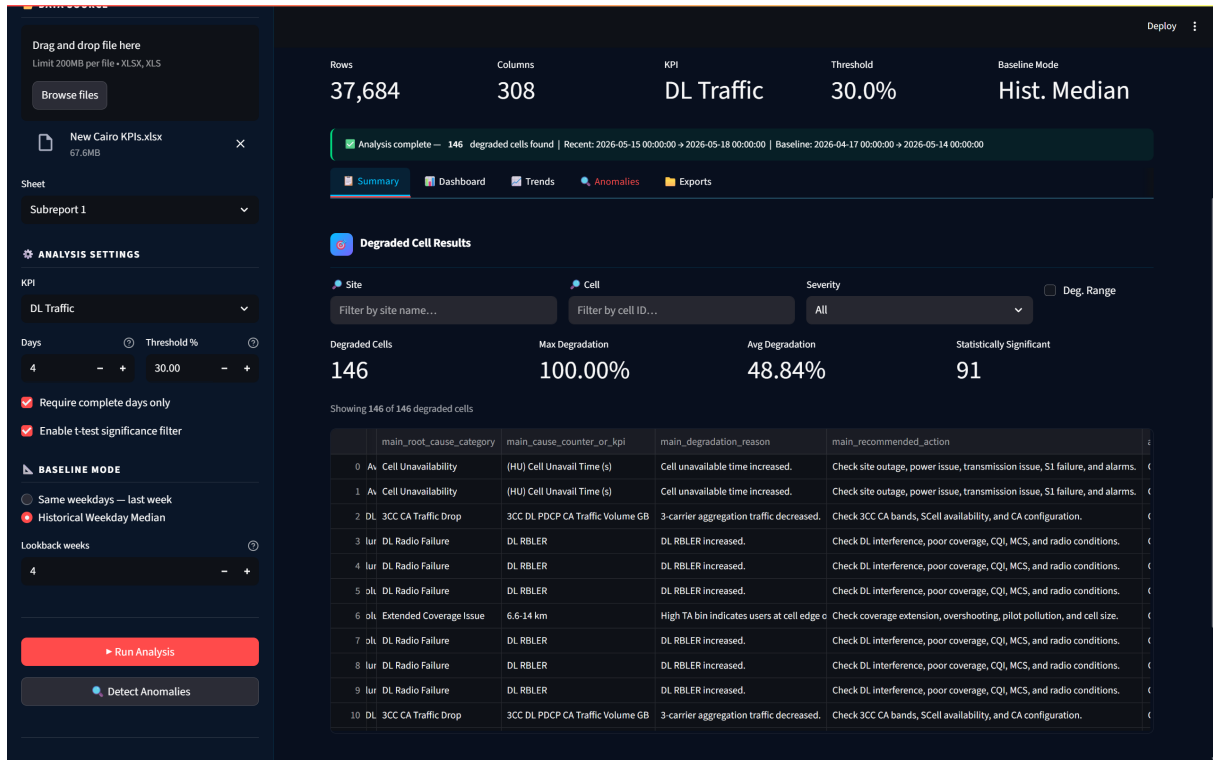


Figure 4.6: Root cause recommendation panel for a degraded cell, showing the identified RCA pattern, supporting correlated counter evidence, RF severity classification, and recommended next investigation steps.

The root cause recommendation output includes the following structured fields:

- **rca_pattern**: The classified root cause pattern (Outage, Congestion, Radio Quality, Coverage, Interference, Mobility, Demand, or Unknown).
- **main_root_cause_category**: A human-readable label for the primary root cause category.
- **rf_severity**: A severity classification (Critical, High, Medium, or Low) based on the composite RCA score.
- **analysis_confidence**: The statistical confidence level of the degradation detection (High, Medium, or Low).
- **confidence_reason**: A textual explanation of the factors contributing to the assigned confidence level.
- **next_investigation_steps**: A list of specific counter values and diagnostic checks recommended for the engineer to perform as the next step in the troubleshooting workflow.

4.8.3 Severity and Confidence Metrics

Figure 4.7 presents the severity and confidence metric panels, which provide an aggregated view of the RCA output quality and the distribution of severity levels across all degraded cells.

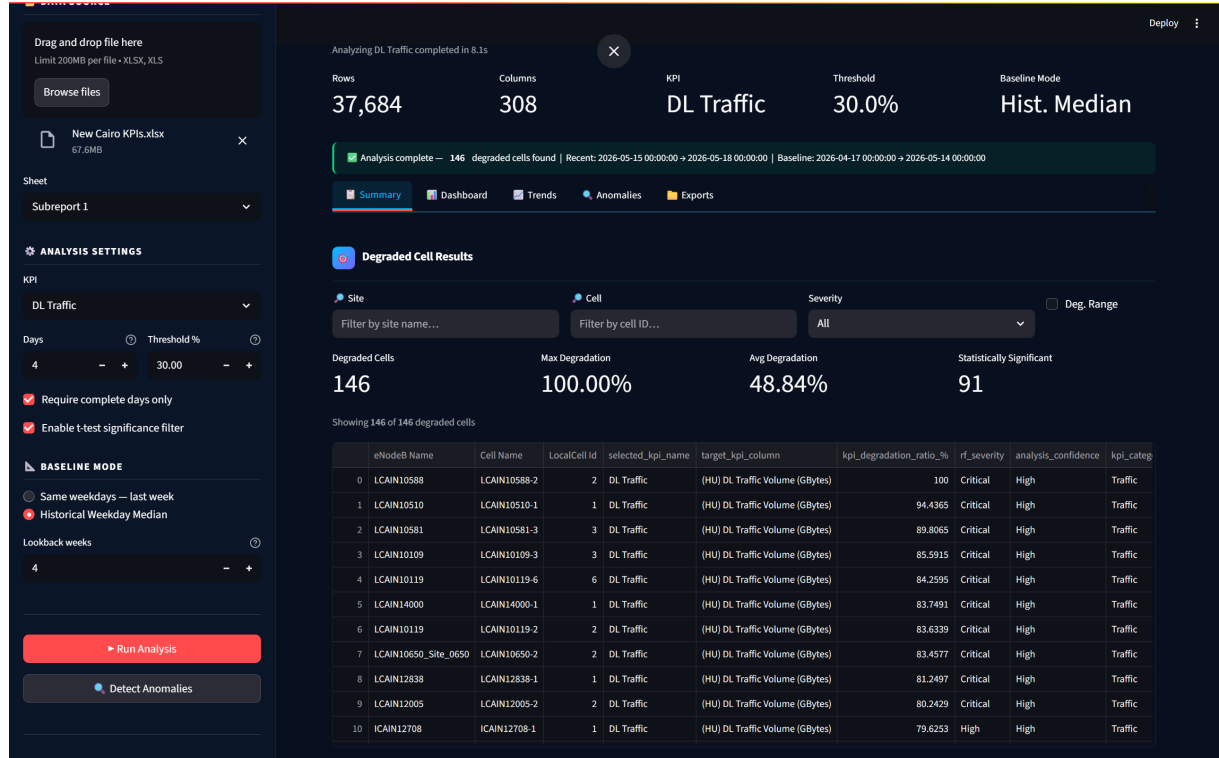


Figure 4.7: Severity and confidence metric panels showing the distribution of rf_severity classifications and analysis_confidence levels across all degraded cells detected in the New Cairo dataset analysis.

4.9 Trend Analysis

The Trends tab enables engineers to visualise the historical KPI time series for individual degraded cells, comparing the recent observation window against the historical baseline period. This visualisation supports engineering judgement about the trajectory of the degradation and assists in distinguishing acute step changes from gradual long-term trends.

Figure 4.8 presents the trend analysis view for a representative degraded cell, showing the KPI time series with baseline and recent window highlighted.

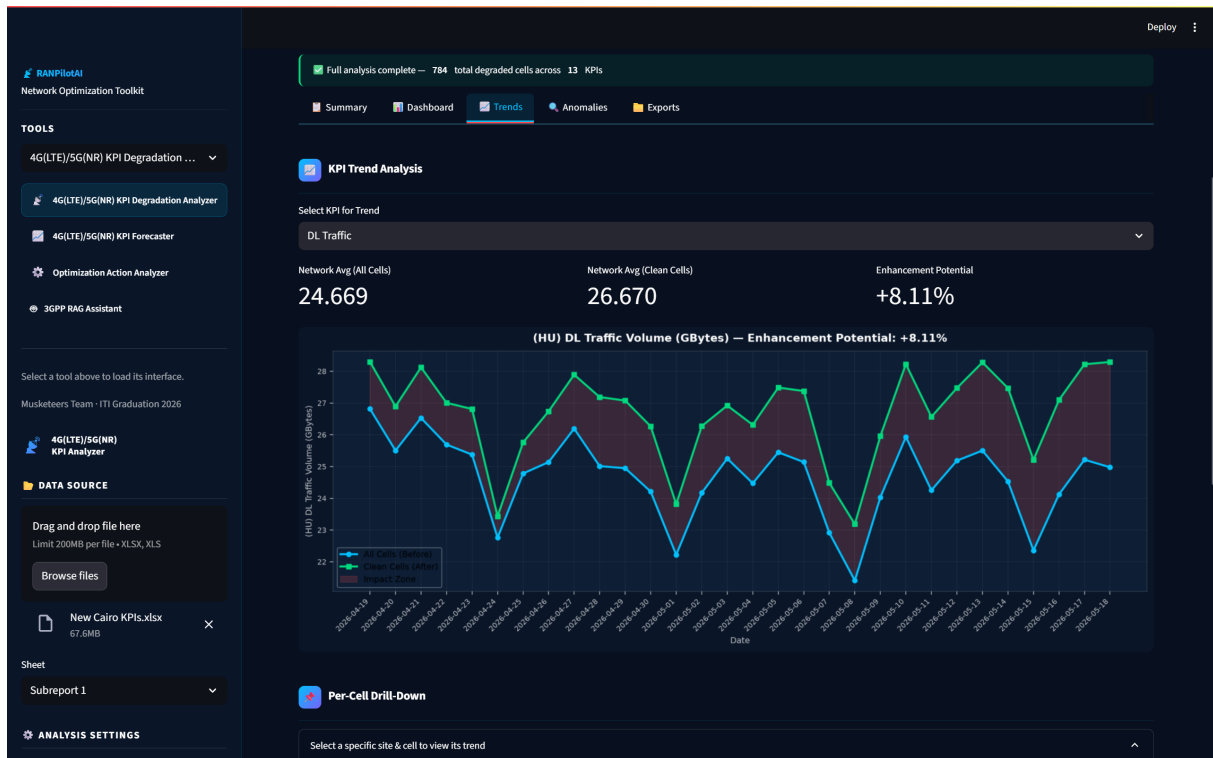


Figure 4.8: Trend analysis view showing the KPI time series for a selected degraded cell, with the historical baseline period and recent observation window indicated. The before-and-after overlay enables visual assessment of degradation onset and trajectory.

Figure 4.9 presents an additional trend analysis example illustrating the per-cell drill-down capability for a different KPI category.

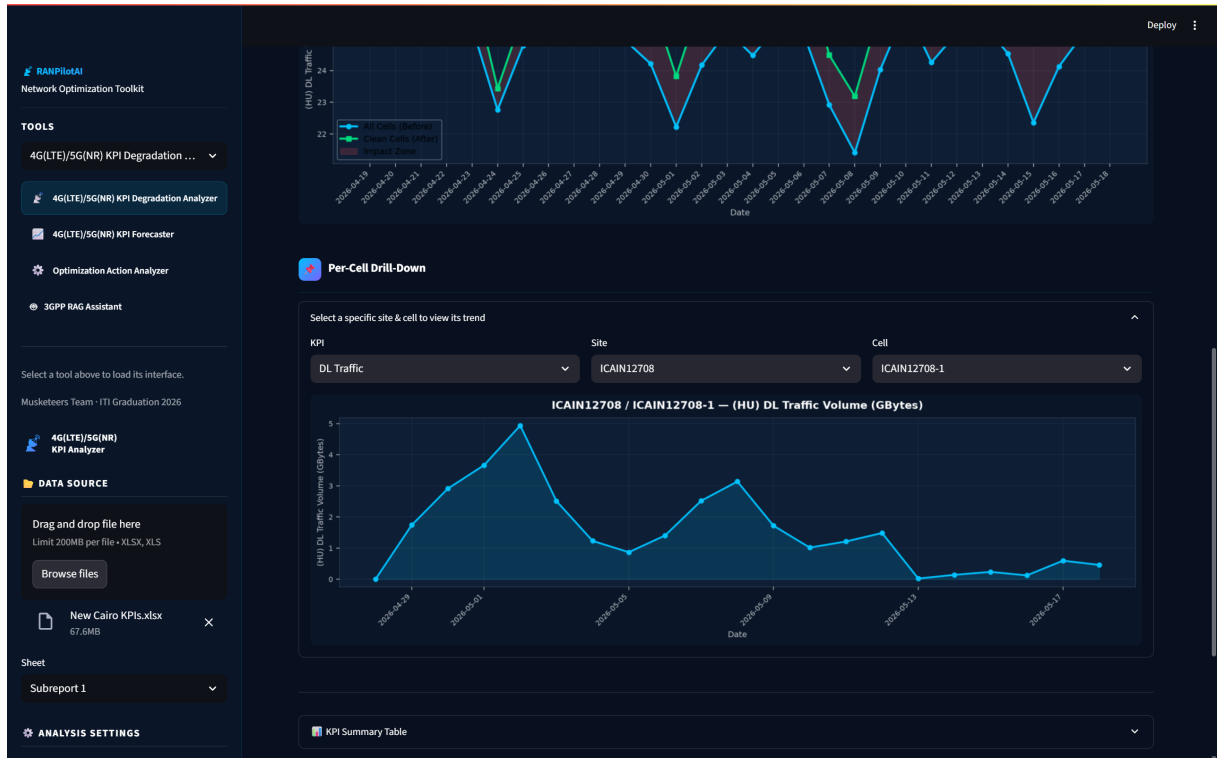


Figure 4.9: Additional trend analysis drill-down showing multi-day KPI trajectory comparison between the baseline and recent periods for a second degraded cell, demonstrating the enhancement potential and before-after overlay visualisation capability.

4.10 Anomaly Detection

In addition to threshold-based degradation detection, the KPI Degradation Analyzer implements a statistical anomaly detection function that identifies zero-value and spike anomalies in the KPI time series using a 4-week rolling baseline. This function complements the main degradation detection pipeline by flagging transient anomalies that may not cross the threshold-based degradation criterion but nonetheless indicate operational issues requiring investigation.

Figure 4.10 presents the anomaly detection output panel.

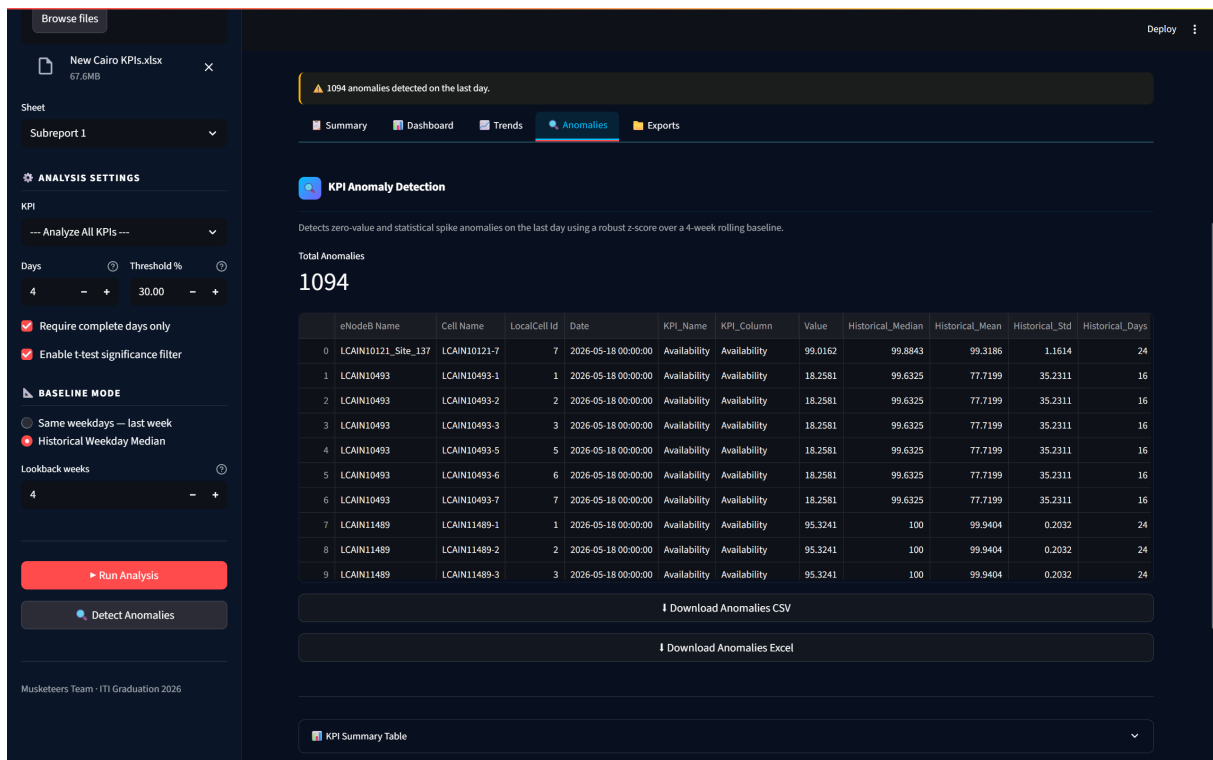


Figure 4.10: Anomaly detection panel showing zero-value and spike anomalies identified in the KPI time series across the analysed cell set, using a 4-week rolling baseline for comparison.

The anomaly detection engine applies the following classification criteria:

- **Zero-value anomaly:** A counter reporting a value of zero on a day when the rolling mean is non-zero and no scheduled maintenance is recorded. Zero-value events for traffic or performance counters typically indicate a cell outage or data collection fault.
- **Positive spike anomaly:** A counter value exceeding a configurable number of standard deviations above the rolling mean, indicating an abnormal peak that may correspond to an interference event, a counter reset artefact, or a genuine traffic surge.
- **Negative spike anomaly:** A counter value falling a configurable number of standard deviations below the rolling mean, indicating an abrupt drop that may precede or accompany a sustained degradation.

4.11 Export Options

The Exports tab provides multiple output format options enabling engineers to incorporate the analysis results into existing operational workflows and reporting templates. Figure 4.11 presents the primary export panel.

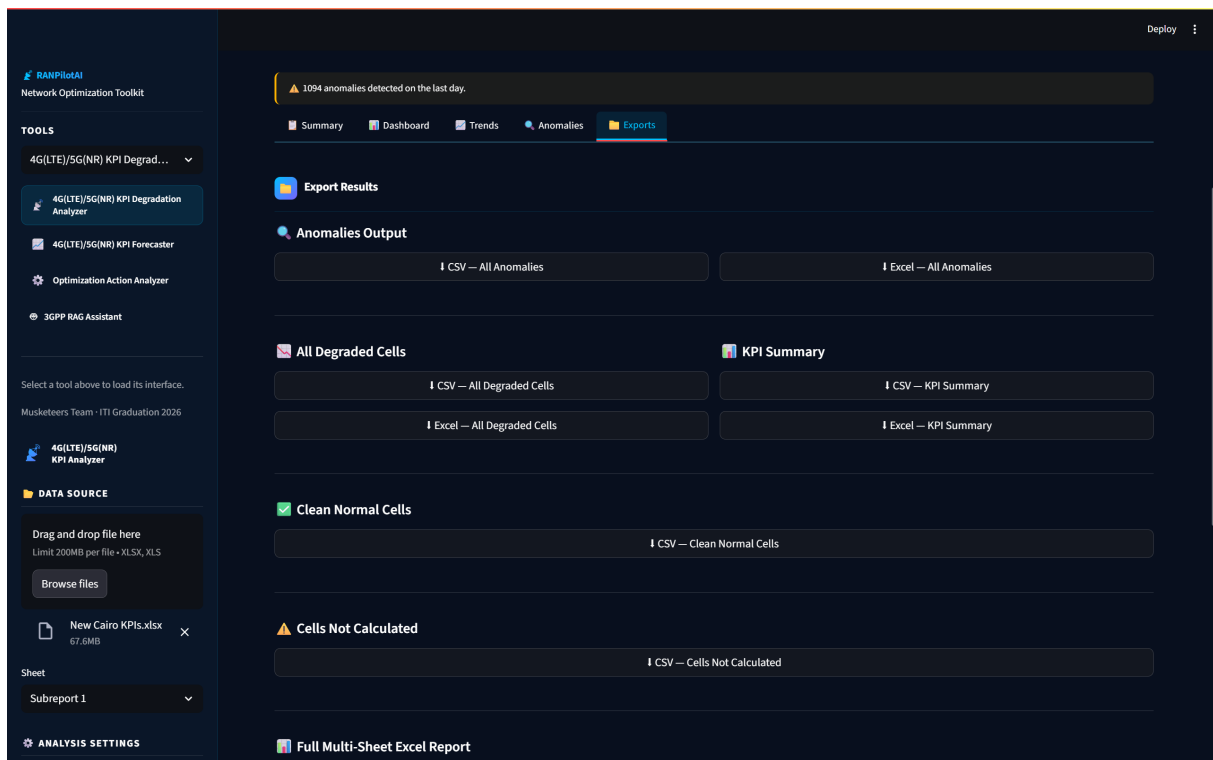


Figure 4.11: Primary export options panel of the KPI Degradation Analyzer, providing download buttons for Excel and Word format exports of the degradation analysis results.

Figure 4.12 presents the additional export options including CSV format and chart export capabilities.

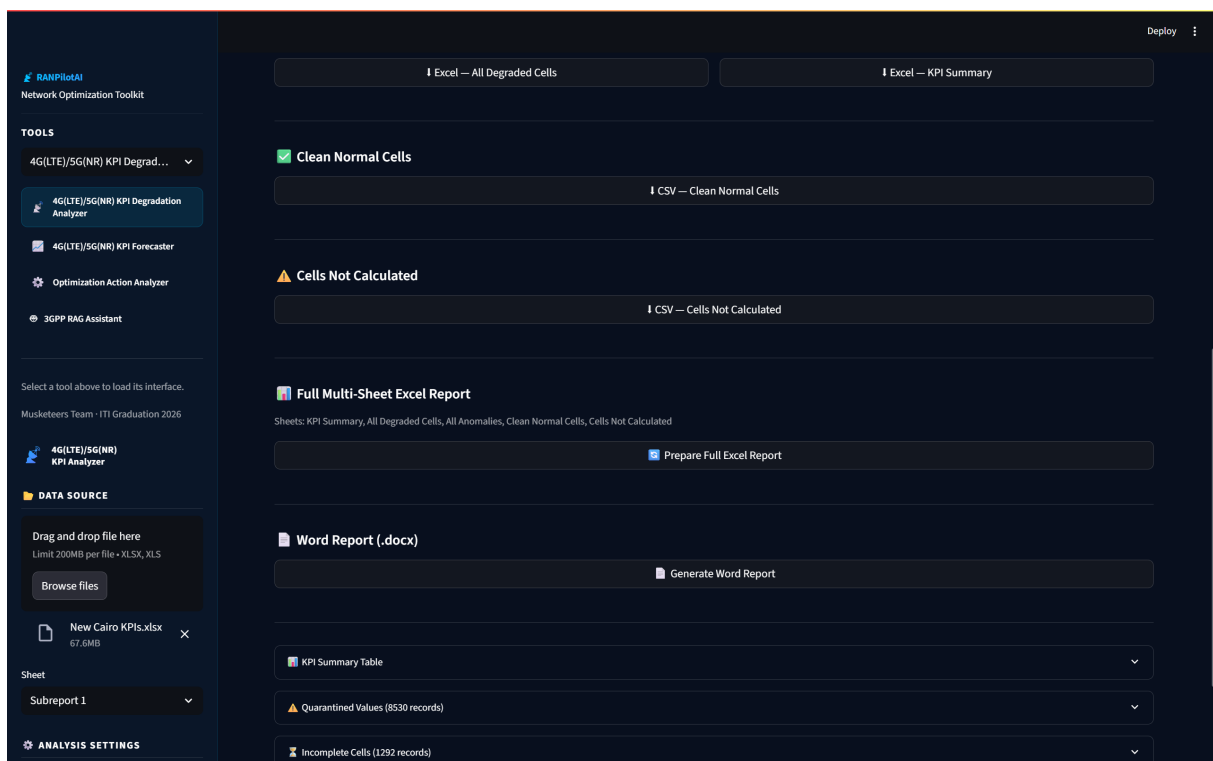


Figure 4.12: Additional export options providing CSV download and individual chart export capabilities for integration with external reporting and ticketing systems.

The following export formats are supported:

- **Excel (XLSX)**: A formatted multi-sheet workbook containing the KPI summary table, the degraded cells detail table with all RCA output columns, and per-KPI result sheets. Cell values are conditionally formatted using red-green colour coding based on severity classification.
- **Word (DOCX)**: A formatted report document suitable for inclusion in network performance reports or incident analysis documentation. The Word export includes summary statistics, the KPI summary table, and the top degraded cells with root cause classifications.
- **CSV**: A flat comma-separated values file containing the complete degraded cells detail table, suitable for import into external analysis tools or network management systems.

4.12 Summary

This chapter has presented the KPI Degradation Analyzer module of the RANPilotAI platform in detail. The module implements a three-layer analytical pipeline comprising a data quality engine, a statistical degradation detection engine, and an RF-aware root cause analysis engine. The pipeline processes raw KPI counter exports from LTE and 5G NR networks and produces structured degradation reports including severity classifications, confidence scores, root cause pattern identifications, and recommended next investigation steps.

The experimental validation of this module on the New Cairo LTE dataset, demonstrating the detection of 784 degraded cells across 13 KPI categories from a dataset of 37,684 rows and 308 columns, is presented in Chapter 9. The module successfully replaces a manual cell-by-cell analysis workflow with an automated, RF-aware analysis pipeline that runs to completion in seconds, producing consistent and interpretable output across all monitored KPI categories.

Chapter 5

KPI Forecasting & Predictive Analytics Module

5.1 Introduction & Motivation

Modern LTE network operations have traditionally relied on reactive maintenance paradigms, wherein performance degradation is addressed only after subscriber-facing quality-of-service (QoS) incidents have already occurred. This post-facto approach incurs tangible costs in terms of churn, regulatory penalties, and emergency truck rolls. A paradigm shift toward *proactive maintenance* requires the ability to anticipate Key Performance Indicator (KPI) trajectories before they breach contractual thresholds.

This chapter presents the **KPI Forecasting & Predictive Analytics Module**, a cell-level, multi-horizon forecasting engine designed to predict LTE KPI values seven days into the future. The module is architected around three complementary modeling strategies: (i) naive statistical baselines that serve as sanity checks, (ii) classical Holt-Winters exponential smoothing to capture dominant seasonal patterns, and (iii) a gradient-boosted tree regressor (XGBoost) that leverages cross-KPI lag features to model non-linear, multi-variate temporal dependencies. A unified evaluation framework compares all candidates against hold-out data and automatically recommends the most reliable predictor on a per-cell, per-KPI basis.

The remainder of this chapter is organized as follows. Section 5.2 reviews the theoretical foundations of time-series forecasting, exponential smoothing, and residual diagnostics. Section 5.3 defines the module’s functional requirements and its interfaces with the degradation analyzer (Chapter 4) and the action analyzer (Chapter 6). Section 5.4 details the data pipeline, seasonality detection, and leakage-safe feature engineering. Section 5.5 derives the three forecasting models and the fallback selection logic. Section 5.6 establishes the evaluation protocol, residual diagnostics, and feature-importance interpretation. Section 5.7 concludes with key findings.

5.2 Theoretical Background

5.2.1 Time-Series Forecasting in Telecom Networks

A univariate time series $\{y_t\}_{t=1}^T$ observed for a single cell-KPI pair can be decomposed into four additive components:

$$y_t = T_t + S_t + C_t + \varepsilon_t, \quad (5.1)$$

where T_t denotes the long-term trend, S_t the seasonal oscillation (e.g., weekly traffic cycles), C_t cyclical variations, and ε_t an irregular residual. In cellular networks, S_t typically dominates because human mobility and usage patterns exhibit strong daily and weekly periodicities. Cyclical components are often negligible at the 30-day observation windows typical of operational dashboards, leaving the forecaster to model $T_t + S_t$ while ensuring that ε_t is approximately white noise.

Telecom KPIs pose distinct challenges: they are often non-stationary (mean and variance shift with network upgrades or load spikes), heteroscedastic (variance scales with traffic volume), and subject to abrupt structural breaks (hardware failures, neighbor-cell interference). These properties motivate a multi-model strategy rather than reliance on a single inductive bias.

5.2.2 Holt-Winters Exponential Smoothing

Holt-Winters extends simple exponential smoothing by explicitly modeling level, trend, and seasonality. For a series with period m , the additive formulation updates three smoothed states:

$$\ell_t = \alpha (y_t - s_{t-m}) + (1 - \alpha) (\ell_{t-1} + b_{t-1}), \quad (5.2)$$

$$b_t = \beta (\ell_t - \ell_{t-1}) + (1 - \beta) b_{t-1}, \quad (5.3)$$

$$s_t = \gamma (y_t - \ell_t) + (1 - \gamma) s_{t-m}, \quad (5.4)$$

where ℓ_t is the smoothed level, b_t the trend, s_t the seasonal component, and $\alpha, \beta, \gamma \in (0, 1)$ are smoothing parameters optimized via maximum likelihood. A multiplicative variant replaces additive seasonality with ratio-based updates when seasonal amplitude scales with the level. In this module, both trend and seasonal component types are configurable (additive, multiplicative, or disabled), allowing the model to adapt to KPIs with weak or strong seasonality.

5.2.3 Gradient Boosting for Time-Series (XGBoost)

While XGBoost is not inherently a sequential model, it can be adapted to time-series forecasting through careful feature engineering that encodes temporal structure into a tabular regression task. Given a target KPI y_t , the model learns a mapping

$$\hat{y}_t = f(\mathbf{x}_t) + \varepsilon_t, \quad (5.5)$$

where the feature vector $\mathbf{x}_t$ comprises lagged values, rolling statistics, and cross-KPI covariates, all strictly constructed from observations at times $< t$ to prevent data leakage. XGBoost approximates f via an ensemble of K regression trees:

$$\hat{y}_t = \sum_{k=1}^K \eta \cdot h_k(\mathbf{x}_t), \quad (5.6)$$

with learning rate η and where each h_k is fitted to the negative gradient (pseudo-residuals) of a differentiable loss function. Regularization terms

$$\Omega(h_k) = \gamma T + \frac{1}{2}\lambda \sum_{j=1}^T w_j^2 + \alpha \sum_{j=1}^T |w_j| \quad (5.7)$$

penalize tree complexity (T leaves), leaf weights w_j , and enforce sparsity via L_1 (α) and L_2 (λ) coefficients. These controls are essential given the limited sample size (~ 30 days) relative to the engineered feature dimensionality.

5.2.4 Residual Diagnostics & Model Validation

A well-specified forecasting model should extract all systematic information from the series, leaving residuals that resemble white noise. Two standard tests are employed:

Durbin-Watson statistic. For residuals $\{e_t\}$, the statistic

$$DW = \frac{\sum_{t=2}^T (e_t - e_{t-1})^2}{\sum_{t=1}^T e_t^2} \quad (5.8)$$

tests for first-order autocorrelation. Values near 2 indicate no autocorrelation; values below 1.5 or above 2.5 suggest positive or negative serial correlation, respectively, implying the model has missed a temporal pattern.

Ljung-Box test. A portmanteau test evaluating the null hypothesis that the first h autocorrelations are jointly zero:

$$Q = T(T+2) \sum_{k=1}^h \frac{\hat{\rho}_k^2}{T-k}, \quad (5.9)$$

where $\hat{\rho}_k$ is the sample autocorrelation at lag k . Under the null, $Q \sim \chi^2(h)$. A p -value exceeding 0.05 supports the white-noise hypothesis.

In addition, the Autocorrelation Function (ACF) of residuals is inspected visually; significant spikes outside the 95% confidence band ($\pm 1.96/\sqrt{T}$) at non-zero lags indicate unmodeled seasonality or trend.

5.3 System Requirements & Design

5.3.1 Functional Requirements

The forecasting module satisfies the following functional requirements derived from the broader proactive maintenance pipeline:

- FR-1. **Multi-model execution:** Support at least one statistical benchmark (Holt-Winters), one machine-learning model (XGBoost), and naive baselines.
- FR-2. **Leakage-safe feature engineering:** All predictive features must be computable using only past observations relative to the forecast origin.
- FR-3. **Automatic model selection:** Compare each candidate against a naive baseline on a hold-out window and recommend the best performer.

- FR-4. **Seasonality quantification:** Compute a seasonality-strength score per cell-KPI pair to inform model selection and UI ranking.
- FR-5. **Residual diagnostics:** Report Durbin-Watson, Ljung-Box, and ACF plots for every XGBoost run.
- FR-6. **Recursive multi-step forecasting:** Generate a 7-day ahead trajectory by iteratively appending predictions to history and re-deriving features.
- FR-7. **LLM-ready export:** Emit a structured textual summary of forecasts, trends, and alerts for downstream natural-language reasoning modules.

5.3.2 Module Architecture & Data Flow

Figure 6.1 illustrates the internal data flow. The uploaded CSV passes through a loader that coerces percentages and renames raw counters to canonical short names. The seasonality analyzer then decomposes each cell-KPI series via STL and computes a strength score. The feature engineer constructs a leakage-safe design matrix, which is split into training and hold-out sets. The forecasting engine trains all candidate models, evaluates them on the hold-out window, and triggers residual diagnostics for XGBoost. Finally, a report generator assembles an LLM-ready prompt summarizing the results.

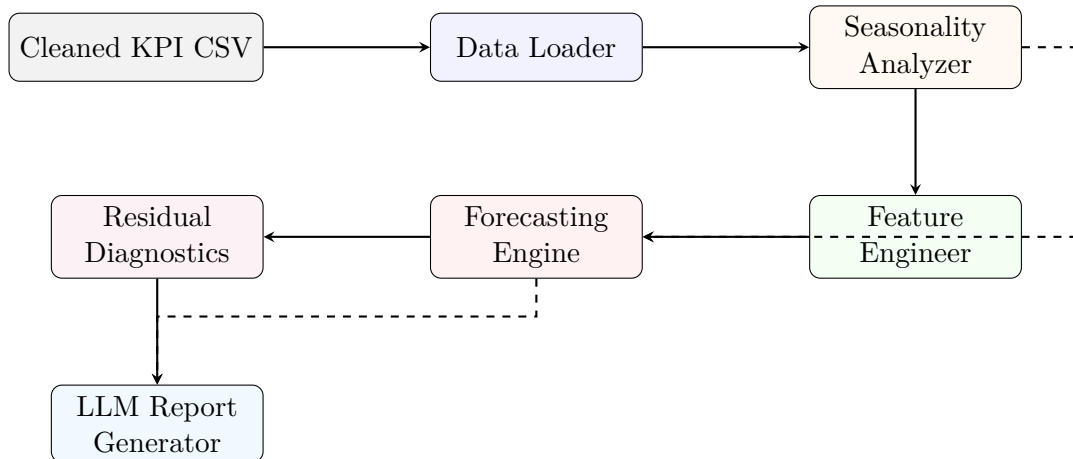


Figure 5.1: High-level data-flow diagram of the KPI Forecasting Module. Solid arrows indicate primary data flow; dashed arrows indicate metadata or control signals.

5.3.3 Interface with Adjacent Modules

The forecasting module occupies a central position in the proactive maintenance pipeline:

- **Upstream (Chapter 4 – Degradation Analyzer):** The degradation analyzer flags anomalous cells and KPIs exhibiting recent downward trends or threshold breaches. These flagged entities are prioritized in the forecasting UI, ensuring that operator attention and computational resources are directed toward the most critical network elements.
- **Downstream (Chapter 6 – Action Analyzer):** The 7-day ahead forecasts produced by this module serve as primary inputs to the action analyzer. Predicted

traffic spikes, for example, trigger preemptive capacity scaling recommendations, while forecasted drops in handover success rates may prompt neighbor-list audits before subscriber impact materializes.

- **Lateral (Chapter 7 – Telecom RAG Assistant):** The LLM-ready report generator outputs structured prompts that the Retrieval-Augmented Generation (RAG) assistant consumes to produce natural-language root-cause hypotheses and remediation strategies.

5.4 Data Pipeline & Preprocessing

5.4.1 Input Data Format

The module ingests a cleaned CSV export containing daily cell-level observations. Each row corresponds to a single cell-date pair, with columns mapped from raw counter names to canonical short identifiers. Table 5.1 enumerates the supported KPIs grouped by the standard 3GPP performance pillar.

Table 5.1: Supported LTE KPIs with canonical internal names and performance categories.

Category	Display Name	Internal Name
Traffic	DL Traffic Volume (GBytes)	DL_Traffic
Traffic	DL Average Throughput (Mbps)	DL_Throughput
Traffic	Average UE Number	Avg_UE_Number
Traffic	Active Users	Active_Users
Accessibility	RRC Setup Success Rate (%)	RRC_Setup_SR
Retainability	E-RAB Drop Rate (%)	ERAB_Drop_Rate
Mobility	Intra-Freq HO Success Rate (%)	Intra_HO_SR
Mobility	Inter-Freq HO Success Rate (%)	Inter_HO_SR
Integrity	DL Average CQI	DL_CQI
Integrity	DL IBLER (%)	DL_IBLER
Utilization	DL PRB Utilization (%)	DL_PRB_Util
Experience	User DL Avg Throughput (Mbps)	User_DL_Throughput

Percentage and rate columns are coerced from string representations (e.g., “98.5%”) to floating-point values during ingestion to ensure numeric compatibility across the pipeline.

5.4.2 Seasonality Detection

Before model training, each cell-KPI series is decomposed using Seasonal and Trend decomposition using Loess (STL). The decomposition separates the observed series into trend, seasonal, and residual components. To quantify the prominence of the seasonal pattern, the seasonality strength score proposed by Hyndman and Athanasopoulos is computed:

$$F_{\text{seasonal}} = \max \left(0, 1 - \frac{\text{Var}(R_t)}{\text{Var}(S_t + R_t)} \right), \quad (5.10)$$

where S_t is the seasonal component and R_t the residual. A score approaching 1.0 indicates that most variation is explained by seasonality, whereas a score near 0.0 implies a weak or noisy seasonal signal. This score is used to sort the cell-selection dropdown (strongest seasonality first) and to display contextual warnings when lag-7 or weekly-mean features may be unreliable.

5.4.3 Cross-KPI Feature Engineering

A leakage-safe feature matrix is constructed for each target KPI by drawing exclusively from past observations. Table 5.2 categorizes the engineered features and their operational rationale.

Table 5.2: Leakage-safe feature engineering taxonomy for the XGBoost regressor.

Feature Category	Examples	Rationale
Lag features	$\text{lag}_1 \dots \text{lag}_7, \text{lag}_{14}, \text{lag}_{21}$	Capture recent historical levels and weekly periodicity.
Rolling statistics	$\text{rolling_mean}_7, \text{rolling_std}_7, \text{rolling_mean}_3, \text{rolling_mean}_{14}, \text{rolling_mean}_{21}$	Smooth local trends and characterize volatility over multiple horizons.
Normalized lags	$\text{lag7_norm} = \text{lag}_7 / \text{rolling_mean}_7$	Express deviation relative to the recent weekly average.
Momentum	$\text{momentum}_1 = \text{lag}_1 - \text{lag}_2, \text{momentum}_2 = \text{lag}_2 - \text{lag}_3$	Encode rate of change and local acceleration.
Week-over-week diff	$\text{wow_diff} = \text{lag}_1 - \text{lag}_8$	Isolate week-to-week drift.
Shape descriptors	$\text{weekly_slope} = \text{lag}_1 - \text{lag}_7$	Capture intra-week trajectory.
Cross-KPI lags	$\text{ERAB_Drop_Rate_lag1}, \text{Inter_H0_SR_lag1}$	Leverage inter-KPI correlations (e.g., throughput vs. PRB utilization).
Lag interactions	$\text{lag1_x_lag7} = \text{lag}_1 \times \text{lag}_7$	Model non-linear interactions between recent and weekly history.

Notably, NaN values arising from insufficient history (e.g., lag_{21} on day 5) are intentionally preserved rather than imputed. XGBoost handles missing values natively via its split-based decision logic, learning optimal directions for absent features without introducing synthetic bias.

5.5 Forecasting Models

5.5.1 Baseline Models

Two naive strategies provide lower-bound benchmarks against which any sophisticated model must justify its complexity:

1. **Naive (last-value) baseline:** $\hat{y}_{T+h} = y_T$ for all $h \in \{1, \dots, 7\}$. This assumes the most recent observation is the best predictor of the immediate future.
2. **Weekly-mean baseline:** If at least seven historical observations exist, $\hat{y}_{T+h} = \frac{1}{7} \sum_{i=0}^6 y_{T-i}$. This captures a crude local average and is particularly competitive when the series is stable.

The baseline with the lowest Mean Absolute Error (MAE) on the hold-out window is designated the “best baseline.” If a candidate model fails to outperform this baseline, the system flags the cell as one where added model complexity does not translate to predictive value.

5.5.2 Holt-Winters Exponential Smoothing

The Holt-Winters implementation uses `statsmodels.tsa.holtwinters.ExponentialSmoothing`. Both trend and seasonal components are configurable via the user interface: `trend` $\in \{\text{add, mul, None}\}$ and `seasonality` $\in \{\text{add, mul, None}\}$, with the seasonal period defaulting to 7 days to match weekly usage cycles. The model is fit via maximum-likelihood optimization of the smoothing parameters and then used to generate both in-sample fitted values and out-of-sample forecasts for the hold-out window and the future 7-day horizon.

Holt-Winters serves as the classical counterpart to XGBoost: it excels when the series exhibits a clear, stable seasonal structure and requires no manual feature engineering. Its primary limitation is the assumption that seasonality is regular and that trend behavior does not shift abruptly—conditions that are occasionally violated in cellular networks during special events or hardware upgrades.

5.5.3 XGBoost Regressor with Recursive Multi-Step Forecasting

The XGBoost model is trained on the feature matrix described in Section 5.4 using a configurable hold-out window (default: the most recent 4 days). Hyperparameters exposed through the interface include:

- `n_estimators` $\in [50, 300]$, default 100
- `learning_rate` $\in \{0.01, 0.03, 0.05, 0.1, 0.2\}$, default 0.05
- `max_depth` $\in [2, 6]$, default 3
- `subsample` $\in [0.5, 1.0]$, default 0.8

Fixed regularization constants are `colsample_bytree`=0.8, `min_child_weight`=3, `reg_alpha`=0.1, and `reg_lambda`=1.0, chosen to combat overfitting given the limited

temporal depth of the dataset. Figure 5.2 shows the Streamlit sidebar through which operators adjust these parameters interactively.

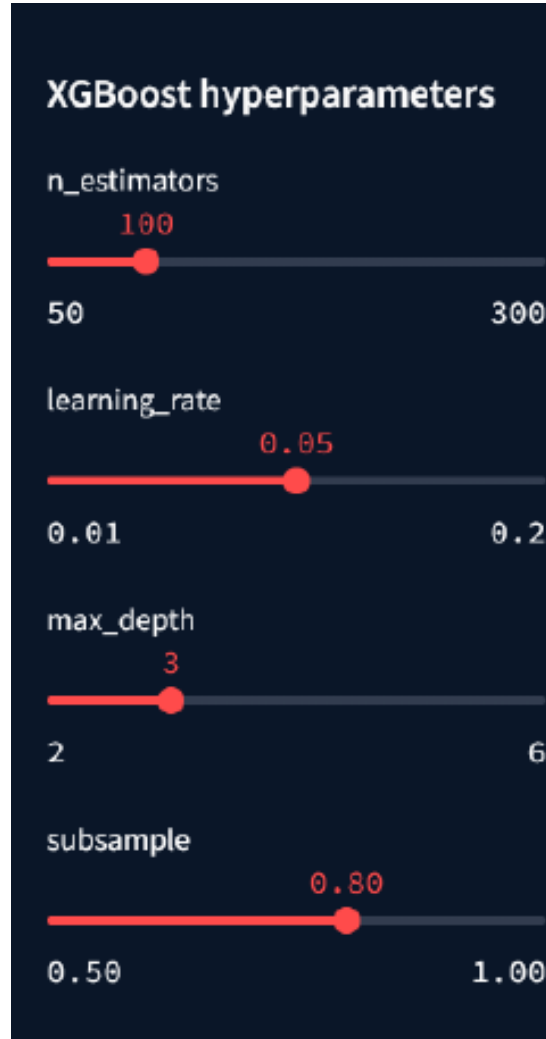


Figure 5.2: Streamlit sidebar exposing XGBoost hyperparameters: `n_estimators`=100, `learning_rate`=0.05, `max_depth`=3, and `subsample`=0.80.

For multi-step forecasting, a recursive strategy is employed. Rather than training seven separate one-step models, a single regressor is retrained on the full available history and then iteratively forecasts one day at a time:

Algorithm 1 Recursive 7-Day Ahead Forecast

Require: Full history $\mathcal{H} = \{y_1, \dots, y_T\}$, feature builder $\phi(\cdot)$, trained model f

- 1: $\mathcal{H}^{(0)} \leftarrow \mathcal{H}$
 - 2: **for** $h = 1$ to 7 **do**
 - 3: $\mathbf{x}_{T+h} \leftarrow \phi(\mathcal{H}^{(h-1)})$
 - 4: $\hat{y}_{T+h} \leftarrow f(\mathbf{x}_{T+h})$
 - 5: Append $\hat{y}_{T+h}$ to $\mathcal{H}^{(h-1)}$ to form $\mathcal{H}^{(h)}$
 - 6: **end for**
 - 7: **return** $\{\hat{y}_{T+1}, \dots, \hat{y}_{T+7}\}$
-

This approach preserves cross-feature and cross-KPI relationships at each step but accumulates prediction error as the horizon extends. The 7-day window was selected to balance operational relevance (typical maintenance scheduling cadence) with forecast fidelity.

5.5.4 Model Selection & Fallback Logic

After all candidates are evaluated on the hold-out window, the system applies the following selection protocol:

1. Compute MAE for XGBoost, Holt-Winters, and the best baseline.
2. Identify the model with the lowest MAE.
3. If the best baseline outperforms both XGBoost and Holt-Winters, emit a *fallback recommendation*: “Model is not adding real value for this cell; use baseline.”
4. Otherwise, promote the winning model and display its metrics alongside the baseline for transparency.

This protocol ensures that the system does not overstate the value of machine learning when the underlying series is inherently unpredictable or stable. Figure 5.3 shows the UI presentation of this comparison for cell LCAIN10022-6.

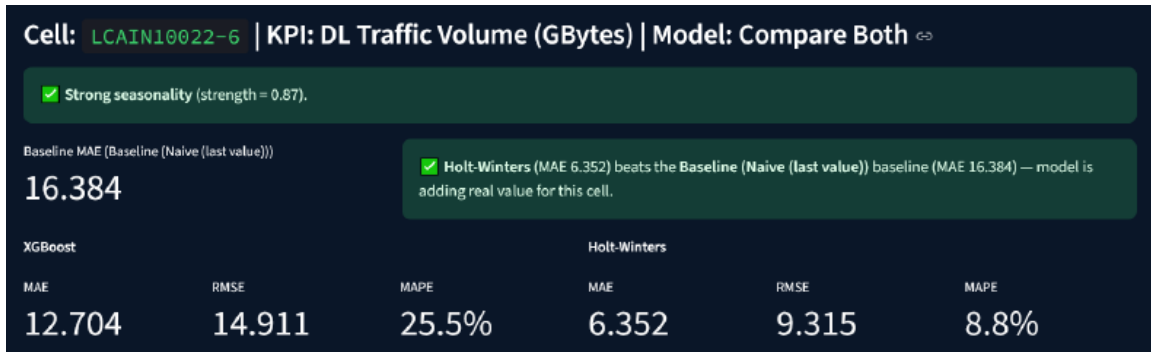


Figure 5.3: Dashboard header for cell LCAIN10022-6 forecasting DL Traffic Volume. The seasonality strength badge ($F_{\text{seasonal}} = 0.87$) and baseline comparison panel confirm that Holt-Winters (MAE 6.352) substantially outperforms the naive baseline (MAE 16.384), validating model deployment for this cell.

5.6 Model Evaluation & Diagnostics

5.6.1 Accuracy Metrics

Three standard metrics quantify hold-out predictive accuracy:

$$\text{MAE} = \frac{1}{H} \sum_{h=1}^H |y_{T+h} - \hat{y}_{T+h}|, \quad (5.11)$$

$$\text{RMSE} = \sqrt{\frac{1}{H} \sum_{h=1}^H (y_{T+h} - \hat{y}_{T+h})^2}, \quad (5.12)$$

$$\text{MAPE} = \frac{100\%}{H} \sum_{h=1}^H \left| \frac{y_{T+h} - \hat{y}_{T+h}}{y_{T+h}} \right|. \quad (5.13)$$

MAE is robust to outliers and directly interpretable in the original KPI units (e.g., GBytes). RMSE penalizes large deviations more heavily, making it sensitive to occasional miss-predictions during anomalous days. MAPE provides a scale-independent percentage error but is undefined for zero-valued observations and unstable for series near zero; it is reported primarily for relative comparison across KPIs with different magnitudes.

5.6.2 Baseline Comparison

Figure 5.4 presents a representative forecast visualization. The vertical dashed line separates the training region from the 4-day hold-out test window. Both XGBoost and Holt-Winters are plotted against actuals and the naive baseline, enabling visual inspection of phase alignment, amplitude capture, and bias.



Figure 5.4: Forecast chart for DL Traffic Volume (GBytes) on cell LCAIN10022-6. The strong weekly seasonality is accurately tracked by Holt-Winters (purple), while XGBoost (red) exhibits larger deviations during the hold-out period. The naive baseline (gray dotted) flatlines at the last training value.

5.6.3 Residual Analysis

For the XGBoost model, in-sample training residuals are subjected to a four-panel diagnostic suite illustrated in Figures 5.5 and 5.6. The diagnostic criteria are:

- **Mean residual:** Should be near zero, indicating unbiased predictions.

- **Durbin-Watson:** Acceptable range $[1.5, 2.5]$. Values outside this interval signal autocorrelated errors.
- **Ljung-Box p -value:** Exceeding 0.05 confirms that residual autocorrelations up to lag 10 are jointly insignificant.
- **ACF plot:** All lags should reside within the 95% confidence band.
- **Residuals vs. fitted:** No funnel or systematic pattern should be visible, confirming homoscedasticity.

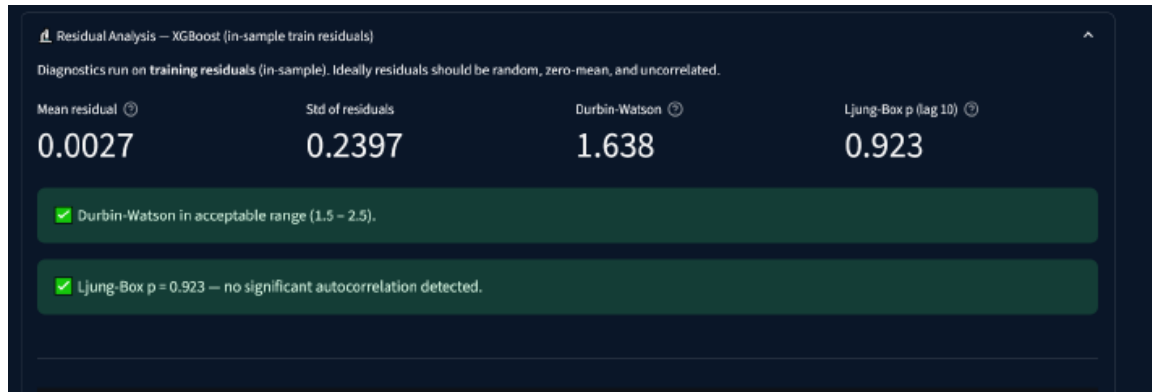


Figure 5.5: Residual summary statistics for XGBoost on the training set. Mean residual = 0.0027, standard deviation = 0.2397, Durbin-Watson = 1.638 (within acceptable range), and Ljung-Box p -value = 0.923 (no significant autocorrelation).

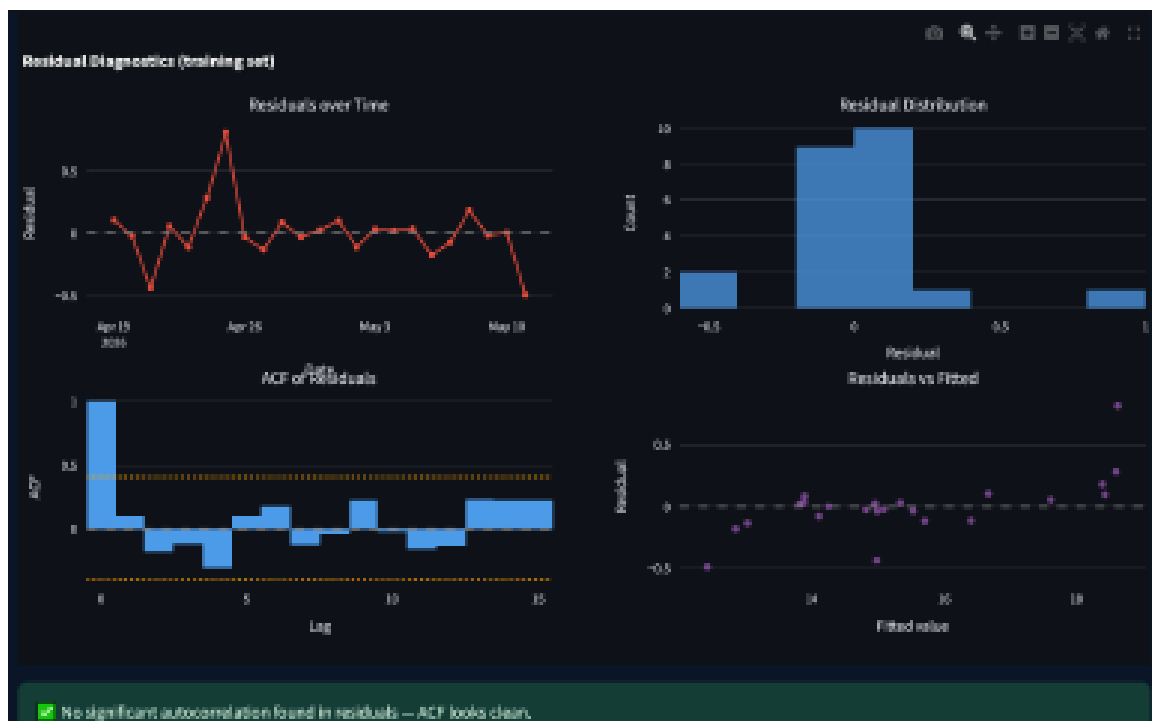


Figure 5.6: Four-panel residual diagnostic plot for XGBoost. Top-left: residuals over time with zero reference; top-right: residual histogram; bottom-left: ACF with 95% confidence bands; bottom-right: residuals versus fitted values. The absence of structure supports the white-noise hypothesis.

5.6.4 Feature Importance Interpretation

Gain-based feature importance from XGBoost reveals which engineered signals drive predictions. Figure 5.7 displays the top-15 features for the DL Traffic Volume example. The dominance of `lag7_norm` and `lag_14` confirms that weekly seasonality is the primary predictive signal, while `momentum_1` and cross-KPI features such as `Inter_HO_SR` capture secondary dynamics. This interpretability is operationally valuable: if a degradation in handover success rate consistently appears as a top feature for throughput forecasting, it suggests a causal coupling that network engineers can investigate.

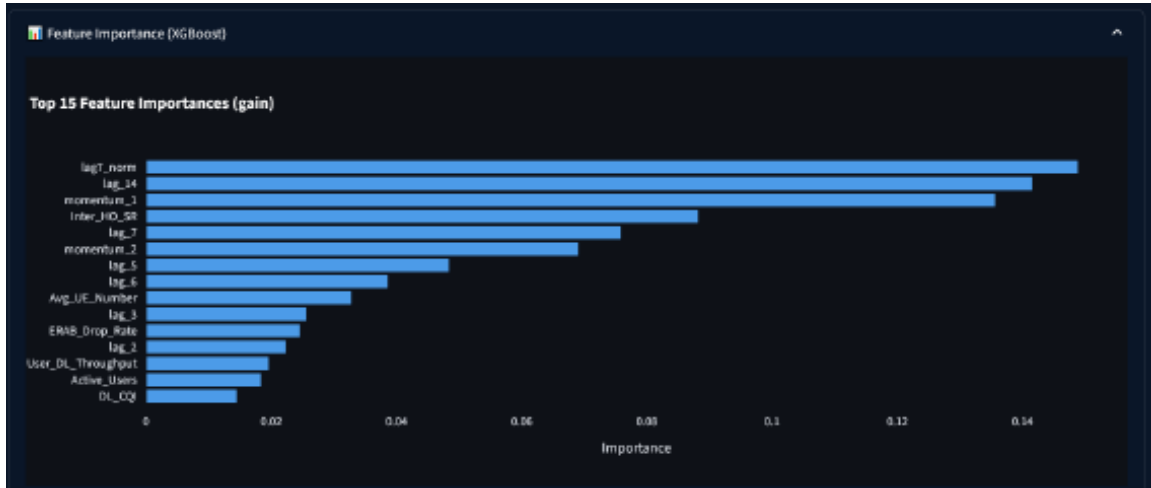


Figure 5.7: Top-15 XGBoost feature importance by gain for DL Traffic Volume forecasting. `lag7_norm` and `lag_14` dominate, confirming strong weekly periodicity. Cross-KPI features (`Inter_HO_SR`, `Avg_UE_Number`) and momentum terms contribute non-linear explanatory power.

5.7 Chapter Summary

This chapter presented the KPI Forecasting & Predictive Analytics Module, a multi-model engine for cell-level LTE KPI prediction. The key contributions and findings are:

- A **leakage-safe feature engineering pipeline** was designed, incorporating lag features, rolling statistics, normalized shape descriptors, and cross-KPI covariates to encode temporal structure for gradient-boosted regression.
- **Three modeling tiers**—naive baselines, Holt-Winters exponential smoothing, and XGBoost—were implemented and evaluated under a unified protocol. An automatic fallback mechanism prevents over-reliance on complex models when simple baselines suffice.
- **Seasonality detection** via STL decomposition quantifies weekly pattern strength per cell-KPI pair, informing both UI ranking and model-selection expectations.
- **Residual diagnostics** (Durbin-Watson, Ljung-Box, ACF) confirmed that the XGBoost model captures the majority of systematic signal, leaving approximately white-noise residuals on the training set.

- The module exposes its outputs through an interactive Streamlit dashboard and generates **LLM-ready reports** for downstream natural-language reasoning, bridging quantitative forecasting with qualitative network optimization.

Chapter 6

Optimization Action Analyzer

6.1 Introduction

Radio Access Network (RAN) optimization teams routinely push configuration changes to live network elements — antenna tilt adjustments, transmit power tuning, handover parameter updates, carrier-aggregation activation, and similar tuning actions. A recurring, largely manual problem faced by these teams is answering a simple but operationally critical question:

“Did the change we just made actually improve the network, and by how much?”

Answering this question correctly requires (i) knowing precisely **what** changed and **when**, (ii) selecting a fair **baseline** period against which to compare performance, and (iii) quantifying the impact across dozens of Key Performance Indicators (KPIs) while accounting for the strong diurnal and weekly seasonality that characterises telecom traffic.

The **Optimization Action Analyzer** (internally named the *NetOps Impact Analyzer*) is a web-based analytical tool built to automate this workflow end-to-end. It couples a version-controlled database of network configuration changes with a KPI time-series warehouse, and produces an interactive before/after impact analysis for any configuration change — without requiring the analyst to manually export data, align time windows, or build charts in a spreadsheet.

6.2 Motivation and Problem Statement

Traditional post-optimization impact analysis in telecom operations suffers from several recurring issues:

- **No single source of truth for “what changed”.** Configuration changes are often tracked informally (tickets, spreadsheets, chat messages), making it difficult to reconstruct an accurate timeline of network actions.
- **Naive before/after comparisons are biased.** Comparing a Monday after a change to the previous day (Sunday) ignores weekly traffic seasonality and produces misleading conclusions.
- **Overlapping changes confound results.** If a second optimization action occurs shortly after the first, naive analysis windows will blend the effects of both changes.

- **Manual, non-repeatable analysis.** Without tooling, every impact assessment is a bespoke, one-off spreadsheet exercise that is slow and error-prone.

The Optimization Action Analyzer directly addresses each of these issues by combining a **version-controlled configuration database (Dolt)** with a **weekday-matched statistical comparison engine** and an **interactive visualization layer**.

6.3 System Architecture

The application follows a clean layered architecture that separates data access, data processing, visualization, configuration persistence, and UI orchestration. This separation of concerns keeps each module independently testable and allows the underlying database or charting library to be replaced with minimal impact on the rest of the system.

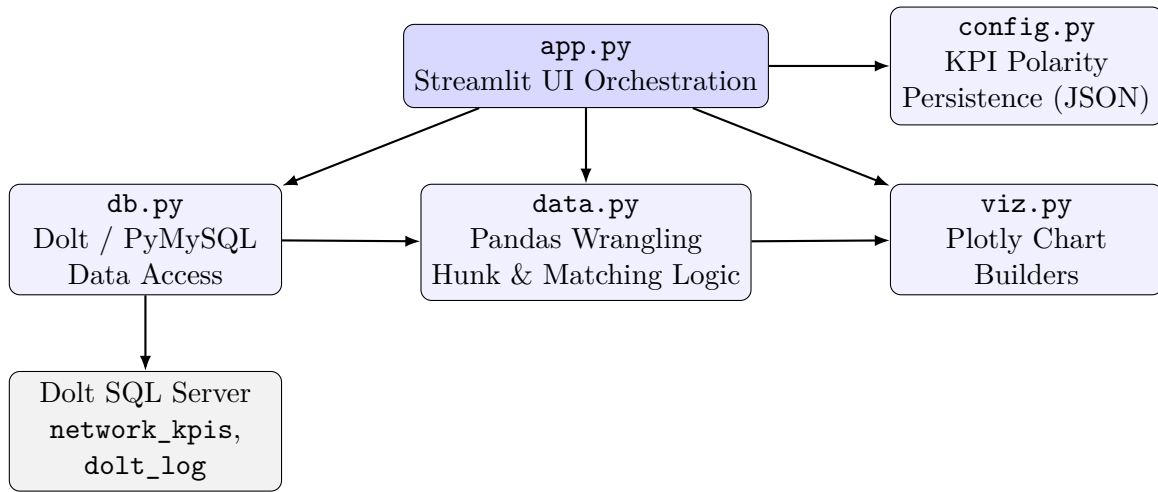


Figure 6.1: High-level module architecture of the Optimization Action Analyzer. The Streamlit application (`app.py`) orchestrates calls into four independent layers: database access, data wrangling, visualization, and configuration persistence.

Table 6.1 summarises the responsibility of each module.

6.4 Data Model and Sources

The analyzer consumes two logical data sources, both served through a **Dolt** SQL server (a version-controlled, Git-like relational database) via **PyMySQL**:

1. **network_kpis** — a time-series table of network performance counters at fixed granularity (15-minute intervals in the reference deployment), including metrics such as downlink/uplink throughput, drop rate, RACH success rate, CQI, BLER, latency, PRB utilization, handover success rate, and SINR.
2. **dolt_log** — Dolt’s built-in commit history system table, giving a full audit trail of every configuration change: commit hash, committer, e-mail, timestamp, and commit message. Because Dolt versions the configuration tables themselves (e.g. `cell_config`, `rf_config`), the per-commit **diff** (which columns changed, from what value to what value) can be retrieved directly through Dolt’s `dolt_diff_<table>` system views.

Table 6.1: Module responsibilities

Module	Responsibility
<code>db.py</code>	Establishes the connection to the Dolt SQL server (or falls back to a synthetic mock data generator for development/demo), discovers KPI columns from the schema, and exposes query functions for the commit log, commit diffs, and KPI time-series.
<code>data.py</code>	Pure-pandas transformation layer: groups raw commits into daily <i>Action Hunks</i> , resolves the rolling evaluation window, and builds weekday-matched Before/After KPI profiles (aggregated and per-day).
<code>viz.py</code>	Builds all Plotly figures — the action timeline, the Before/After trend chart, the interval delta bar chart — and constructs the tabular KPI impact summary.
<code>config.py</code>	Persists the user-defined optimization polarity (“higher is better” vs. “lower is better”) for every KPI to <code>kpi_config.json</code> , inferring sensible defaults from column-name keywords on first run.
<code>app.py</code>	Streamlit front-end: wires the other four modules together, renders the sidebar, controls, charts, and summary table, and manages Streamlit’s caching layer.

This design means the tool requires *no separate change-management system*: the database’s own version history *is* the action log, guaranteeing that the recorded timeline of changes is always accurate and tamper-evident.

For development and demonstration purposes, `db.py` includes a `USE MOCK DATA` switch that generates realistic synthetic KPI data (diurnal traffic patterns plus an injected post-action uplift) and a fabricated five-hunk commit log, allowing the full pipeline to be exercised without a live Dolt instance.

6.5 Core Algorithms

6.5.1 Action Hunks: Grouping Commits by Calendar Day

Individual commits are rarely meaningful in isolation — an optimization action typically spans several related configuration commits performed within the same maintenance window. `build_action_hunks()` groups all commits sharing the same calendar date into a single *Action Hunk*, producing, for each hunk, a human-readable label (e.g. “2024-06-10 (Mon) · 3 commits”) and the list of underlying commit records. This gives the analyst one clean, selectable unit of change per optimization event rather than a noisy list of individual commits.

6.5.2 Rolling Evaluation Window

Once a hunk is selected, `resolve_eval_window()` determines the “After” period over which impact will be measured:

- The window always starts on the hunk’s date.
- By default, the window ends either *today* or the day before the *next* recorded hunk — whichever is earlier — so that the effect of a later change is never mixed into the current analysis.
- An “**Ignore Next Action Hunk**” toggle lets the analyst override this cap and extend the window through to the present day when a later change is known to be unrelated or negligible.

6.5.3 Weekday-Matched Baseline Construction

Naively comparing the day after a change to the day before it is statistically unsound because telecom traffic exhibits strong weekly seasonality (weekday vs. weekend, business-hours peaks, etc.). The analyzer instead uses `collect_matching_weekdays()`, which:

1. For every date in the After window, computes the exact same weekday exactly **seven days earlier** as its matching Before date.
2. Averages KPI values across all After-window dates sharing the same minute-of-day index (0–1439), and separately averages the corresponding Before dates.
3. Aligns both resulting profiles to a common minute-of-day index so that Before and After curves are directly comparable.

This produces statistically fair, seasonality-normalised Before/After profiles even for multi-week evaluation windows. A companion function, `collect_individual_days()`, preserves the same matching logic without averaging, grouping every individual After day (and its matched Before day) by weekday name, so an analyst can drill into a specific day’s behaviour rather than only viewing the aggregate.

6.5.4 Optimization Polarity

Not every KPI improves by increasing: a rise in throughput is good, while a rise in drop rate or latency is bad. `config.py` maintains a persistent **polarity map** (“higher” or “lower” is better) for every discovered KPI. On first run, polarity is inferred automatically from column-name keywords (e.g. `drop`, `error`, `latency`, `bler`, `retry` $\Rightarrow$ lower is better); the analyst can override and persist any KPI’s polarity from the sidebar, and the choice is written to `kpi_config.json` so it survives application restarts. This polarity map drives the colour-coding (improved / degraded) used throughout the delta chart and the summary table.

6.6 Visualization Layer

All charts are built with Plotly and share a consistent dark, telecom-operations-console visual theme (navy background, cyan/green/red accent palette, monospace font) defined centrally in `viz.py`.

Action Timeline (`plot_action_timeline`) — a horizontal HUD-style timeline where each marker represents one Action Hunk; marker size and colour encode the number of commits, and hovering reveals full commit metadata (hash, committer, e-mail, timestamp, message) for every commit in that hunk.

KPI Trend — Aggregated Mode (`plot_kpi_trend`) — a dual-line overlay of the averaged Before and After minute-of-day profiles for the selected KPI, with optional vertical markers annotating the exact commit timestamps inside the evaluation window.

KPI Trend — Individual-Day Mode (`plot_kpi_trend_individual`) — one figure per weekday, overlaying every individual After day against its matched Before day, with an optional bold mean-overlay trace per weekday group.

Interval Delta Bars (`plot_deltaBars`) — a polarity-aware bar chart of (After — Before) at every minute-of-day interval, coloured green where the change is an improvement and red where it is a degradation, based on the KPI's configured polarity.

KPI Impact Summary Table (`build_summary_table`) — a single table covering *every* discovered KPI in one view: Before average, After average, absolute change, percentage change, and a colour-coded Improved / Degraded / No-Change status, giving the analyst an at-a-glance verdict on the overall health impact of the optimization action.

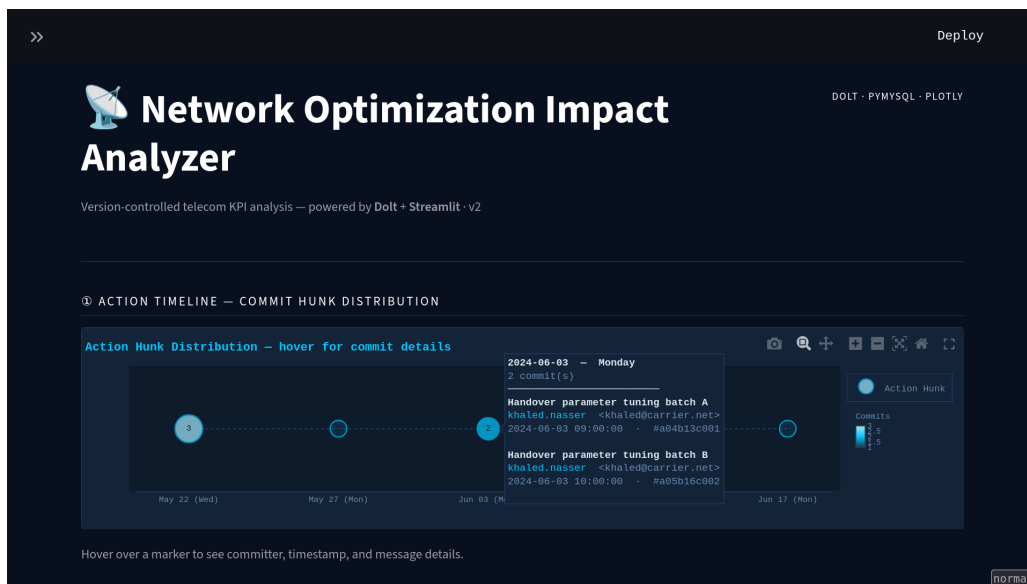


Figure 6.2: Action Timeline HUD showing five Action Hunks with per-commit hover detail.

6.7 User Workflow

The Streamlit application (`app.py`) guides the analyst through a linear, five-section workflow, illustrated in Figure 6.6 and summarised below.



Figure 6.3: Before/After KPI trend comparison for a selected metric, with commit annotations.

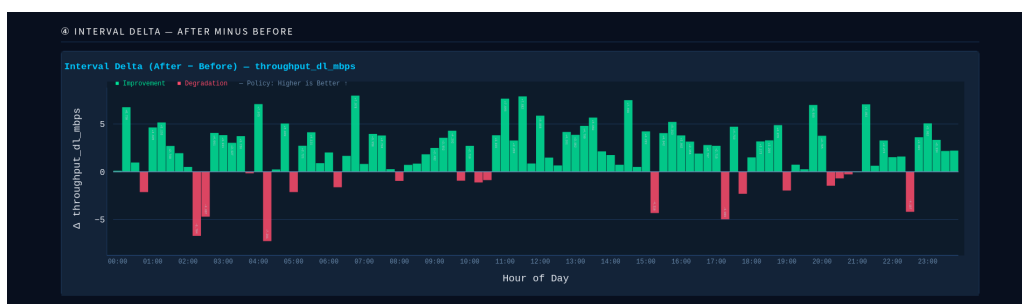


Figure 6.4: Polarity-aware interval delta bar chart.

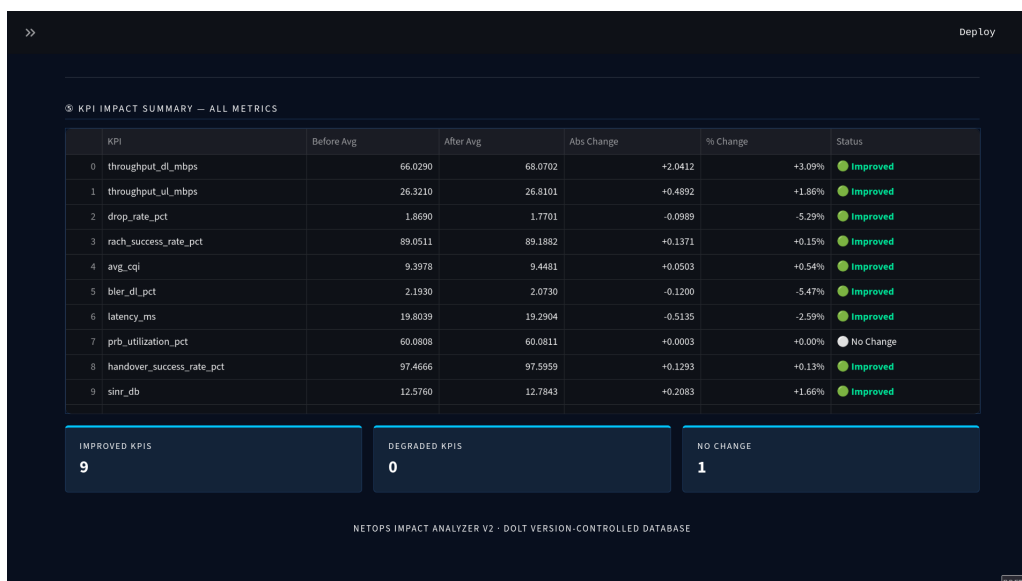


Figure 6.5: KPI impact summary table with Improved/Degraded/No-Change status counts.

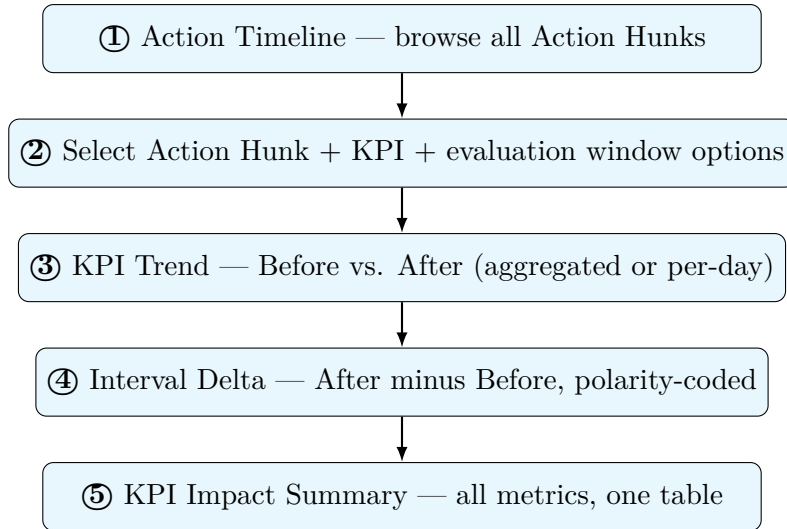


Figure 6.6: The five-step analyst workflow implemented in `app.py`.

1. **Action Timeline.** The analyst first sees every Action Hunk plotted on an interactive timeline, giving immediate visual context on how frequently and when optimization actions have occurred.
2. **Hunk & Window Selection.** The analyst picks a specific hunk, a KPI to analyse, and optionally toggles *Ignore Next Action Hunk* to extend the evaluation window past a later change. Metric tiles immediately display the resolved Action Date, After-window end date, next-hunk boundary (if any), and matched weekday. An expandable panel additionally surfaces the exact configuration diff (table, column, from-value, to-value) for every commit in the hunk, sourced live from Dolt's `dolt_diff_*` system views.
3. **KPI Trend.** The Before/After profiles are rendered either as a single aggregated pair of curves or, when multiple After days exist, split per weekday with each individual day plotted against its matched baseline. A time-of-day slider lets the analyst zoom into a specific part of the day (e.g. business hours only).
4. **Interval Delta.** A bar chart quantifies the exact magnitude and direction of change at every time-of-day interval, automatically colour-coded using the KPI's configured polarity.
5. **Impact Summary.** A single sortable table reports Before/After averages, absolute and percentage change, and an Improved/Degraded/No-Change verdict for *every* KPI in the dataset simultaneously, with roll-up counts displayed as summary tiles.

6.8 Technology Stack

Table 6.2: Technologies used in the Optimization Action Analyzer

Layer	Technology
Database	Dolt (version-controlled MySQL-compatible SQL server)
Database connector	PyMySQL
Data processing	pandas, NumPy
Visualization	Plotly (<code>plotly.graph_objects</code>)
Web application / UI	Streamlit
Configuration persistence	JSON (<code>kpi_config.json</code>)

6.9 Design Highlights

- **Version control as the audit trail.** By building on Dolt rather than a conventional relational database, every configuration change is inherently versioned, attributable, and diffable — eliminating the need for a bespoke change-log table.
- **Seasonality-aware statistics.** The weekday-matching baseline logic (Section 6, Core Algorithms) avoids the common pitfall of comparing traffic across mismatched days of the week.
- **Automatic boundary detection.** The rolling evaluation window automatically caps itself at the next recorded change, preventing the effects of two overlapping optimization actions from being conflated, while still giving the analyst an explicit override.
- **Zero-configuration KPI discovery.** New KPI columns added to the `network_kpis` table are automatically discovered via `INFORMATION_SCHEMA` introspection and assigned a sensible default polarity, requiring no code changes to support new metrics.
- **Graceful degradation.** A mock-data mode (`USE MOCK DATA`) allows the entire application — UI, charts, and analysis logic — to be demonstrated and developed without a live Dolt deployment.

6.10 Limitations and Future Work

- The current workflow depends on the dolt version-control system which is yet to be adopted in the workflow of the filed optimization.
- Statistical significance testing (e.g. confidence intervals or hypothesis tests on the Before/After delta) is not yet implemented; the summary table currently reports point estimates only.
- The tool currently analyses one cell/site’s KPI stream at a time; extending the schema and queries to support multi-cell, multi-site comparative analysis is a natural next step.

- Anomaly detection could be layered on top of the delta bar chart to automatically flag intervals with statistically unusual deviations, rather than relying solely on the polarity sign convention.

6.11 Summary

The Optimization Action Analyzer demonstrates how pairing a version-controlled database with a seasonality-aware statistical comparison engine and an interactive visualization layer can turn a traditionally manual, error-prone impact-assessment workflow into a repeatable, auditable, and self-service analytical tool. By automatically reconstructing the timeline of network changes from the database’s own commit history and comparing performance against weekday-matched baselines, the system gives network optimization teams a fast, reliable answer to the central question of their work: *did this change actually help?*

Chapter 7

Telecom RAG Assistant

The Telecom Retrieval-Augmented Generation (RAG) Assistant is one of the four intelligent modules developed within the RANPilotAI platform. It is designed to assist Radio Access Network (RAN) engineers in efficiently retrieving and understanding information from official Third Generation Partnership Project (3GPP) Technical Specifications. Unlike conventional AI chatbots that rely solely on the internal knowledge of Large Language Models (LLMs), the proposed assistant integrates semantic retrieval with language generation, ensuring that every generated response is grounded in authentic engineering documentation.

The assistant combines document preprocessing, semantic chunking, dense embedding generation, vector similarity search, neural re-ranking, prompt engineering, and Large Language Models into a unified Retrieval-Augmented Generation pipeline. Consequently, engineers can ask technical questions using natural language while receiving structured responses supported by direct citations from the corresponding 3GPP specifications.

7.1 Introduction

Modern LTE and 5G New Radio (NR) networks are standardized by hundreds of Technical Specifications published by the Third Generation Partnership Project (3GPP). These specifications define every aspect of the cellular system, including Radio Resource Control (RRC), Packet Data Convergence Protocol (PDCP), Radio Link Control (RLC), Medium Access Control (MAC), mobility management, measurement procedures, bearer configuration, security mechanisms, and radio resource management.

Although these documents serve as the authoritative engineering reference, they collectively contain thousands of pages distributed across multiple releases and specification documents. Locating a specific protocol definition, signaling procedure, configuration parameter, or ASN.1 structure often requires manually searching several PDF files and navigating numerous sections before identifying the required information. Traditional keyword-based searching is inefficient because engineering concepts are frequently distributed across different chapters, tables, annexes, and protocol descriptions.

Recent advances in Large Language Models (LLMs) have significantly improved natural language interaction with technical information. However, general-purpose language models cannot be considered authoritative engineering references because they generate responses solely from learned knowledge and may therefore produce inaccurate or hallucinated technical information. Such inaccuracies are unacceptable in professional telecommunication environments where engineering decisions must comply with official

standards.

To address these challenges, the proposed Telecom RAG Assistant employs a Retrieval-Augmented Generation architecture. Rather than generating answers directly from the language model, the system first retrieves the most relevant sections from locally indexed 3GPP specifications before constructing the prompt supplied to the LLM. This approach combines the reasoning capabilities of modern language models with the reliability of official technical documentation, producing responses that are both accurate and traceable.

The Telecom RAG Assistant is fully integrated into the RANPilotAI platform alongside the KPI Degradation Analyzer, KPI Forecaster, and Optimization Action Analyzer. The platform provides a unified environment in which engineers can perform network analysis and standards consultation without switching between multiple software applications.

Figure 7.1 presents the main interface of the Telecom RAG Assistant. The interface enables engineers to select the RAG module, optionally restrict the search to a specific Technical Specification, submit engineering questions using natural language, and receive structured answers supported by citations extracted from the original specifications.

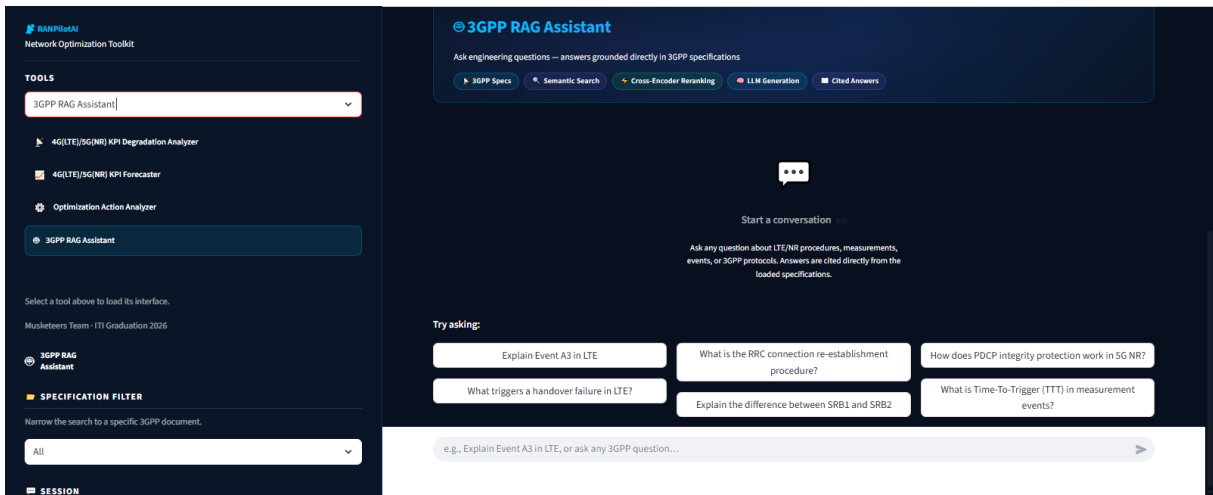


Figure 7.1: Main interface of the Telecom RAG Assistant integrated into the RANPilotAI platform. The interface provides access to the Telecom RAG module, allows filtering by specific 3GPP Technical Specifications, and supports interactive engineering question answering using Retrieval-Augmented Generation.

Unlike conventional document search systems, the developed assistant performs semantic understanding of both the user’s query and the indexed specification content. Consequently, relevant information can be retrieved even when the wording used by the engineer differs from the terminology employed within the official specification.

Furthermore, the generated responses are presented in a structured and readable format that includes hierarchical headings, explanatory paragraphs, engineering tables, bullet lists, and expandable citation panels. This presentation enables engineers not only to obtain concise explanations but also to verify every statement against the corresponding sections of the official 3GPP documentation.

7.2 Motivation

Radio Access Network engineers routinely consult 3GPP Technical Specifications during network optimization, protocol analysis, interoperability testing, feature implementation, and troubleshooting activities. As LTE and 5G technologies continue to evolve, the complexity of these specifications has increased substantially, introducing thousands of configurable parameters, signaling procedures, optional features, and release-specific enhancements.

In traditional engineering workflows, locating the required information often involves opening multiple PDF documents, performing repetitive keyword searches, comparing several specification releases, and manually interpreting protocol descriptions before obtaining a complete understanding of the desired concept. This process is both time-consuming and inefficient, particularly when information is distributed across multiple sections or Technical Specifications.

Although modern Large Language Models provide an intuitive conversational interface, they cannot guarantee engineering accuracy because they do not inherently reference official standards during inference. Consequently, generated responses may contain incomplete explanations, outdated information, or fabricated protocol behavior. Such limitations significantly reduce their reliability for standards-based engineering applications.

The primary motivation behind developing the Telecom RAG Assistant was therefore to combine the natural language understanding capabilities of modern LLMs with the reliability of official 3GPP specifications. By integrating semantic retrieval with language generation, the assistant ensures that every response is generated using authentic engineering context retrieved directly from the indexed documents.

Another important objective was to reduce the time required for standards consultation. Instead of manually navigating hundreds of pages across multiple specifications, engineers can simply submit questions in natural language and immediately receive concise technical explanations accompanied by explicit references to the corresponding Technical Specification, release version, section number, and page number.

The developed graphical interface further enhances usability by allowing users to restrict searches to specific Technical Specifications through the Specification Filter, observe the internal execution stages of the Retrieval-Augmented Generation pipeline, and review supporting citations for every generated response. This transforms static technical documentation into an interactive engineering assistant capable of supporting daily telecommunication engineering activities.

7.3 Problem Statement

The exponential growth of LTE and 5G New Radio (NR) standards has significantly increased the complexity of engineering documentation. Modern 3GPP Technical Specifications consist of thousands of pages describing protocol procedures, signaling messages, configuration parameters, ASN.1 definitions, state machines, timers, and implementation guidelines. Although these documents serve as the official reference for telecommunication systems, efficiently locating the required information remains a major challenge for network engineers.

Traditional document retrieval techniques rely primarily on keyword-based searches within Portable Document Format (PDF) files. While these methods are suitable for

locating exact phrases, they frequently fail when users employ different terminology or when the required information is distributed across multiple specification sections. For example, an engineer searching for "Why is the HO-Conn field required?" may not retrieve the relevant specification if the document describes the same concept using formal protocol terminology rather than the user's wording.

General-purpose Large Language Models provide an alternative approach by enabling natural language interaction with technical knowledge. However, these models generate responses based on their internal parameters rather than official engineering references. Consequently, they may produce inaccurate explanations, outdated information, or fabricated protocol descriptions. Such hallucinations are unacceptable in professional telecommunication environments where engineering decisions must strictly comply with standardized specifications.

Therefore, the primary challenge addressed in this work is the development of an intelligent question-answering system capable of combining semantic understanding with authoritative document retrieval. The system should allow engineers to interact with 3GPP specifications using natural language while ensuring that every generated response remains grounded in authentic engineering documentation.

To address this challenge, the proposed Telecom RAG Assistant employs a Retrieval-Augmented Generation (RAG) architecture that retrieves the most relevant specification sections before generating the final response. This approach significantly improves factual accuracy while preserving complete traceability to the original Technical Specifications.

7.4 Background

The Telecom RAG Assistant integrates several modern Artificial Intelligence and Information Retrieval technologies to transform static engineering documentation into an interactive knowledge assistant. This section introduces the fundamental concepts that form the basis of the proposed system.

7.4.1 Large Language Models (LLMs)

Large Language Models (LLMs) are deep neural networks trained on massive textual datasets to predict the next token in a sequence [14]. Recent models such as GPT, LLaMA, Claude, Gemini, and DeepSeek have demonstrated remarkable capabilities in natural language understanding, reasoning, summarization, translation, and conversational question answering. These capabilities enable users to interact with complex technical information using natural language instead of predefined commands or structured queries.

Despite their impressive performance, LLMs suffer from several limitations when applied to engineering applications. Since their knowledge is acquired during training, newly published standards or updated technical documents cannot be accessed unless the model is retrained. Furthermore, the generated responses may contain hallucinations, where the model produces fluent but factually incorrect information. In standards-based domains such as telecommunications, such inaccuracies may lead to incorrect engineering decisions.

Consequently, rather than relying exclusively on an LLM, the proposed system augments the language model with external knowledge retrieved directly from official 3GPP specifications.

7.4.2 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is a hybrid Artificial Intelligence framework that combines document retrieval with neural language generation. Instead of generating responses solely from the model's internal knowledge, a RAG system first retrieves the most relevant documents from an external knowledge base and then incorporates these documents into the prompt supplied to the language model [9].

The general RAG workflow consists of four sequential stages:

1. User query processing.
2. Semantic retrieval of relevant document chunks.
3. Prompt construction using the retrieved context.
4. Response generation by the Large Language Model.

This architecture provides several important advantages over standalone language models. First, it allows the knowledge base to be updated without retraining the language model. Second, it significantly reduces hallucinations because responses are generated using authentic supporting documents. Finally, it enables explicit citation of the retrieved sources, thereby improving transparency and user confidence.

Within the proposed Telecom RAG Assistant, Retrieval-Augmented Generation serves as the core framework that connects semantic search with natural language generation, enabling accurate engineering question answering over official 3GPP Technical Specifications.

7.4.3 Embedding Models

Semantic retrieval requires transforming textual information into numerical representations that preserve contextual meaning. This transformation is performed using embedding models, which encode sentences or document passages into dense high-dimensional vectors.

Unlike traditional keyword matching, embedding models capture semantic similarity between different textual expressions. Consequently, two sentences describing the same engineering concept using different terminology are represented by nearby vectors in the embedding space.

For example, the following engineering questions express similar meanings despite using different wording:

- What is Event A3?
- Explain the A3 measurement event.
- How does LTE Event A3 trigger handover?

A high-quality embedding model generates similar vector representations for these queries, enabling the retrieval engine to identify the same specification sections regardless of the exact wording used by the engineer.

The Telecom RAG Assistant employs a transformer-based embedding model to encode both the indexed specification chunks and incoming user queries into a shared semantic vector space.

7.4.4 Vector Databases

After generating dense embeddings for every document chunk, an efficient mechanism is required to store and retrieve these vectors. Traditional relational databases are not designed for similarity search in high-dimensional vector spaces. Instead, specialized vector databases perform Approximate Nearest Neighbor (ANN) search to efficiently retrieve semantically similar vectors [8].

The proposed system utilizes Qdrant as the vector database responsible for storing all embedded document chunks together with their associated metadata. Besides the dense embeddings, each indexed record contains additional information including:

- Technical Specification number.
- 3GPP Release version.
- Section identifier.
- Page number.
- Document title.

This metadata enables the system to filter search results according to user-selected specifications while preserving complete traceability to the original engineering documents.

7.4.5 Cross Encoder Re-ranking

Although dense vector retrieval effectively identifies semantically related document chunks, the retrieved results may still include partially relevant passages. To improve retrieval precision, the proposed system performs a second-stage neural re-ranking process using a Cross Encoder model.

Unlike embedding models that independently encode queries and documents, Cross Encoders jointly process both inputs and compute a direct relevance score for each query-document pair. This enables the model to perform deeper semantic analysis and produce a more accurate ranking of candidate document chunks.

After the initial vector search retrieves the top candidate passages, the Cross Encoder evaluates each retrieved chunk, assigns a relevance score, and reorders the results accordingly. Only the highest-ranked chunks are subsequently included in the final prompt supplied to the language model.

This two-stage retrieval strategy combines the efficiency of vector similarity search with the accuracy of neural relevance estimation, significantly improving the quality of the generated engineering responses while minimizing irrelevant contextual information.

7.5 Overall System Architecture

The Telecom RAG Assistant follows a modular Retrieval-Augmented Generation (RAG) architecture specifically designed for querying official 3GPP Technical Specifications. The architecture integrates document preprocessing, semantic indexing, vector retrieval, neural re-ranking, prompt construction, and Large Language Model (LLM) inference into a unified pipeline. Each module performs a dedicated task while interacting with adjacent

modules through well-defined interfaces, resulting in a scalable and maintainable software architecture.

Unlike conventional document search systems, the proposed architecture performs semantic retrieval instead of keyword matching. Both the indexed specification documents and incoming user queries are represented as dense embedding vectors. Similarity search is then performed within the embedding space, enabling the retrieval of conceptually related information even when different terminology is used.

The overall architecture of the proposed Telecom RAG Assistant is illustrated in Figure 7.2.

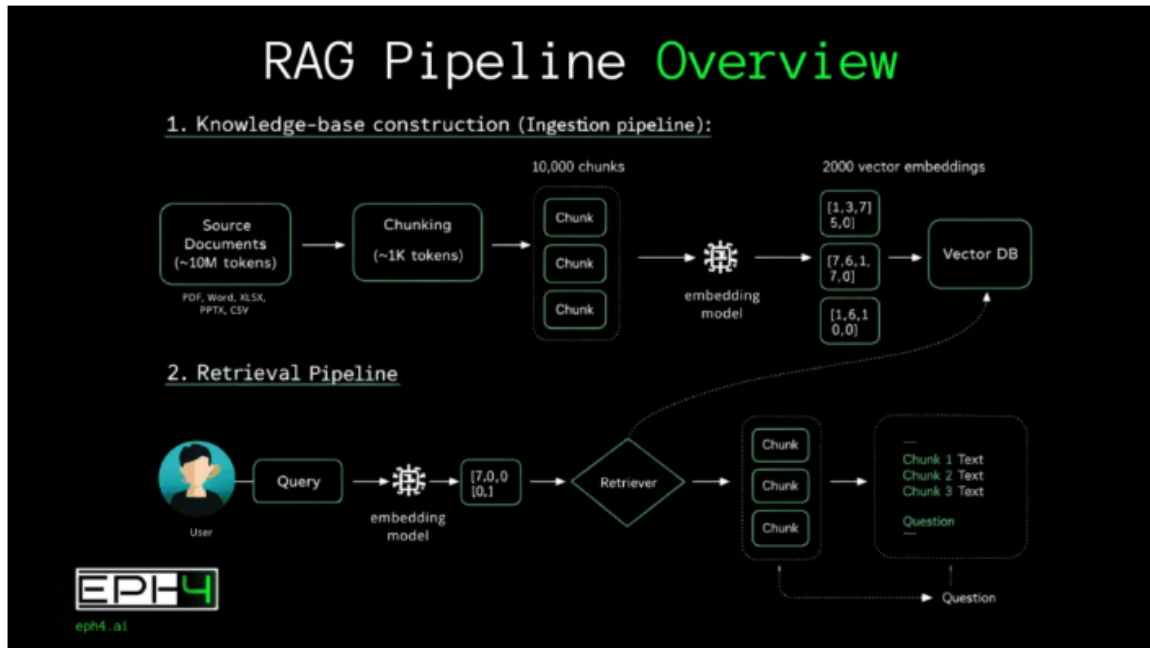


Figure 7.2: Overall architecture of the proposed Telecom RAG Assistant.

The complete system consists of the following processing stages:

1. PDF Processing
2. Section-aware Chunking
3. Embedding Generation
4. Vector Storage using Qdrant
5. Metadata Filtering
6. Dense Vector Retrieval
7. Cross Encoder Re-ranking
8. Prompt Construction
9. LLM Response Generation
10. Response Rendering with Source Citations

Each module contributes to improving retrieval accuracy, reducing hallucination, and maintaining traceability to the original engineering documentation.

7.6 Software Architecture

The Telecom RAG Assistant was implemented using a modular software architecture in which every component is responsible for a specific stage of the Retrieval-Augmented Generation pipeline. This modular design simplifies maintenance, facilitates future extensions, and allows individual components to be optimized independently.

The software architecture is composed of the following modules:

- **PDF Processing Module**

Extracts textual content and metadata from official 3GPP Technical Specifications.

- **Section-aware Chunking Module**

Splits the extracted documents into semantically meaningful chunks while preserving the hierarchical structure of the specifications.

- **Embedding Module**

Generates dense vector representations for every document chunk using the selected embedding model.

- **Vector Database Module**

Stores all embedding vectors together with their associated metadata using the Qdrant vector database.

- **Metadata Filtering Module**

Restricts document retrieval according to user-selected Technical Specifications.

- **Retrieval Module**

Performs semantic similarity search to retrieve the Top-K most relevant document chunks.

- **Cross Encoder Module**

Re-ranks the retrieved chunks according to their semantic relevance to the user query.

- **Prompt Construction Module**

Builds the final prompt by combining the retrieved specification context with the user's question.

- **Large Language Model Module**

Generates the final engineering response using the constructed prompt.

- **Streamlit User Interface**

Provides the graphical interface through which users interact with the complete system.

The modular organization allows each subsystem to operate independently while contributing to the overall Retrieval-Augmented Generation workflow.

7.7 Query Processing Workflow

The complete query processing workflow begins when the engineer submits a natural-language question through the Streamlit interface. Instead of directly forwarding the request to the Large Language Model, the proposed system executes a sequence of retrieval and validation stages to ensure that the generated response is based exclusively on relevant sections extracted from the indexed 3GPP specifications.

Figure 7.3 illustrates the execution stages displayed by the system while processing a user query.

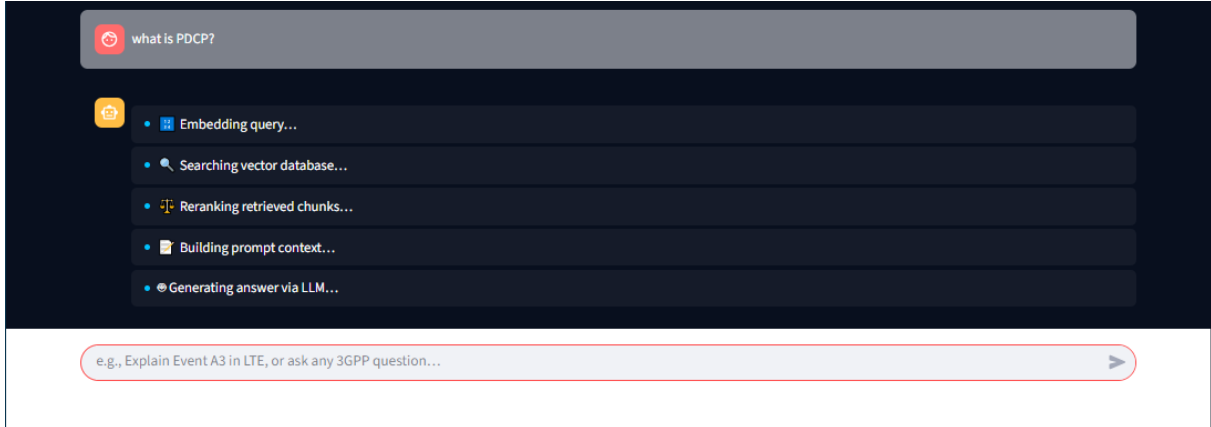


Figure 7.3: Real-time execution stages of the Retrieval-Augmented Generation pipeline during query processing.

After receiving a user query, the system performs the following sequence of operations:

1. The user submits a technical question through the graphical interface.
2. The query is converted into a dense semantic embedding using the embedding model.
3. The generated embedding is submitted to the Qdrant vector database to retrieve the Top-K semantically similar document chunks.
4. Metadata filtering is applied whenever the user restricts the search to a specific Technical Specification.
5. Duplicate chunks are removed to maximize contextual diversity.
6. A Cross Encoder model evaluates each retrieved chunk and computes semantic relevance scores.
7. The highest-ranked chunks are selected to construct the final prompt.
8. The prompt is transmitted to the selected Large Language Model through the Open-Router API.
9. The language model generates a structured engineering response based on the retrieved specification context.
10. Supporting citations are attached to the generated response, enabling engineers to verify every statement directly from the corresponding 3GPP Technical Specification.

The graphical interface provides users with real-time feedback regarding the execution status of every processing stage, including embedding generation, vector retrieval, neural re-ranking, prompt construction, and response generation, thereby improving transparency and user confidence in the retrieval process.

7.8 PDF Processing

The first stage of the proposed Telecom RAG Assistant consists of processing the official 3GPP Technical Specification documents. Since these specifications are distributed as Portable Document Format (PDF) files, their contents must first be transformed into a machine-readable representation before they can be indexed and queried.

Unlike conventional document parsers that simply extract plain text, the proposed system performs structured document analysis in order to preserve the hierarchical organization of the specifications. This is particularly important because 3GPP documents contain numbered sections, subsections, engineering tables, ASN.1 definitions, mathematical expressions, figures, and annexes, all of which contribute to the semantic meaning of the document.

The PDF processing module is responsible for extracting textual content, removing unnecessary formatting artifacts, identifying section boundaries, extracting engineering tables, and collecting document metadata such as the Technical Specification number, release version, page number, and document title. These metadata elements are preserved throughout the indexing pipeline and later attached to every retrieved chunk, allowing engineers to trace generated responses back to their original source.

The implementation utilizes the PyMuPDF library to efficiently parse PDF documents while preserving page-level information. During preprocessing, repetitive page headers, footers, and formatting artifacts are removed in order to improve retrieval quality. Engineering tables are detected and converted into Markdown representations, enabling the language model to interpret structured information more accurately than raw PDF text. This preprocessing stage significantly improves the readability and semantic consistency of the indexed documents.

The processed output generated by this stage serves as the input to the Section-aware Chunking module, which further organizes the extracted content into semantically meaningful units suitable for embedding generation.

7.9 Section-aware Chunking

One of the most important contributions of the proposed Telecom RAG Assistant is the implementation of a section-aware chunking strategy specifically designed for 3GPP Technical Specifications.

Traditional Retrieval-Augmented Generation systems generally divide documents into fixed-size chunks based on a predefined number of tokens or characters. Although computationally simple, this approach frequently separates logically connected engineering concepts across multiple chunks. As a consequence, important contextual information may be lost, leading to incomplete retrieval results and reduced response quality.

To overcome this limitation, the developed system performs hierarchical chunking based on the structural organization of 3GPP specifications. Instead of splitting documents at arbitrary token boundaries, the chunking algorithm detects section headers,

subsection identifiers, engineering tables, figure descriptions, annexes, ASN.1 definitions, and mathematical expressions. Each chunk therefore represents a complete engineering concept rather than an arbitrary text fragment.

- The textual content extracted from the specification.
- Metadata describing the source Technical Specification, release version, section identifier, page number, and document type.

This structured representation enables subsequent retrieval stages to perform semantic search while simultaneously preserving complete traceability to the original engineering documentation.

7.10 Embedding Generation

After document chunking has been completed, every chunk is transformed into a dense numerical representation using a transformer-based embedding model. These dense vectors capture the semantic meaning of each engineering concept, enabling similarity search beyond simple keyword matching.

The embedding model projects both specification chunks and incoming user queries into the same high-dimensional semantic vector space. Consequently, conceptually related texts are positioned close to one another regardless of the exact wording used. This capability enables engineers to formulate questions using natural language while still retrieving the appropriate specification sections.

During the indexing process, each document chunk is independently encoded before being inserted into the vector database. Similarly, every user query submitted through the graphical interface is encoded using the same embedding model. Since both document embeddings and query embeddings share the same semantic space, similarity can be efficiently computed using vector distance metrics.

The generated embeddings constitute the primary index used by the retrieval engine and form the foundation of the semantic search capabilities implemented within the Telecom RAG Assistant.

7.11 Vector Database (Qdrant)

Following embedding generation, all dense vectors are stored inside a local Qdrant vector database. Qdrant was selected due to its efficient Approximate Nearest Neighbor (ANN) search algorithms, support for metadata filtering, lightweight deployment model, and seamless integration with Python-based machine learning frameworks.

Unlike cloud-hosted vector databases, the developed system employs Qdrant in embedded local mode. Consequently, the complete vector database resides on the local machine, eliminating external dependencies while improving data privacy and reducing deployment complexity.

Each indexed record stored within Qdrant contains both the embedding vector and its associated metadata. Typical metadata include:

- Technical Specification number
- 3GPP Release version

- Section identifier
- Page number
- Document title

The inclusion of metadata enables advanced filtering operations during retrieval while preserving direct references to the original engineering documentation.

When a user submits a query, the corresponding embedding vector is generated and compared against all indexed vectors stored in Qdrant. The database retrieves the Top-K most semantically similar chunks, which are subsequently passed to the Cross Encoder re-ranking stage for further refinement.

7.12 Metadata Filtering

Although semantic retrieval effectively identifies document chunks that are conceptually related to the user’s query, searching across all indexed specifications may introduce unnecessary or less relevant results. Since the Telecom RAG Assistant indexes multiple 3GPP Technical Specifications spanning different releases and protocol layers, restricting the search scope can significantly improve retrieval precision.

To address this challenge, the proposed system incorporates a metadata filtering stage before the final retrieval process. Every indexed document stored in the vector database contains descriptive metadata extracted during the indexing stage, including the Technical Specification (TS) number, release version, section identifier, page number, and document title. These metadata fields enable the retrieval engine to perform constrained searches according to the user’s selected criteria.

The graphical user interface provides an interactive Specification Filter that allows engineers to limit the search space to one or more Technical Specifications before submitting a query. For example, an engineer investigating Radio Resource Control (RRC) procedures may restrict the search to TS 36.331, while another engineer interested in Physical Layer procedures may instead search within TS 36.214.

Figure 7.4 illustrates the Specification Filter implemented within the Telecom RAG Assistant.

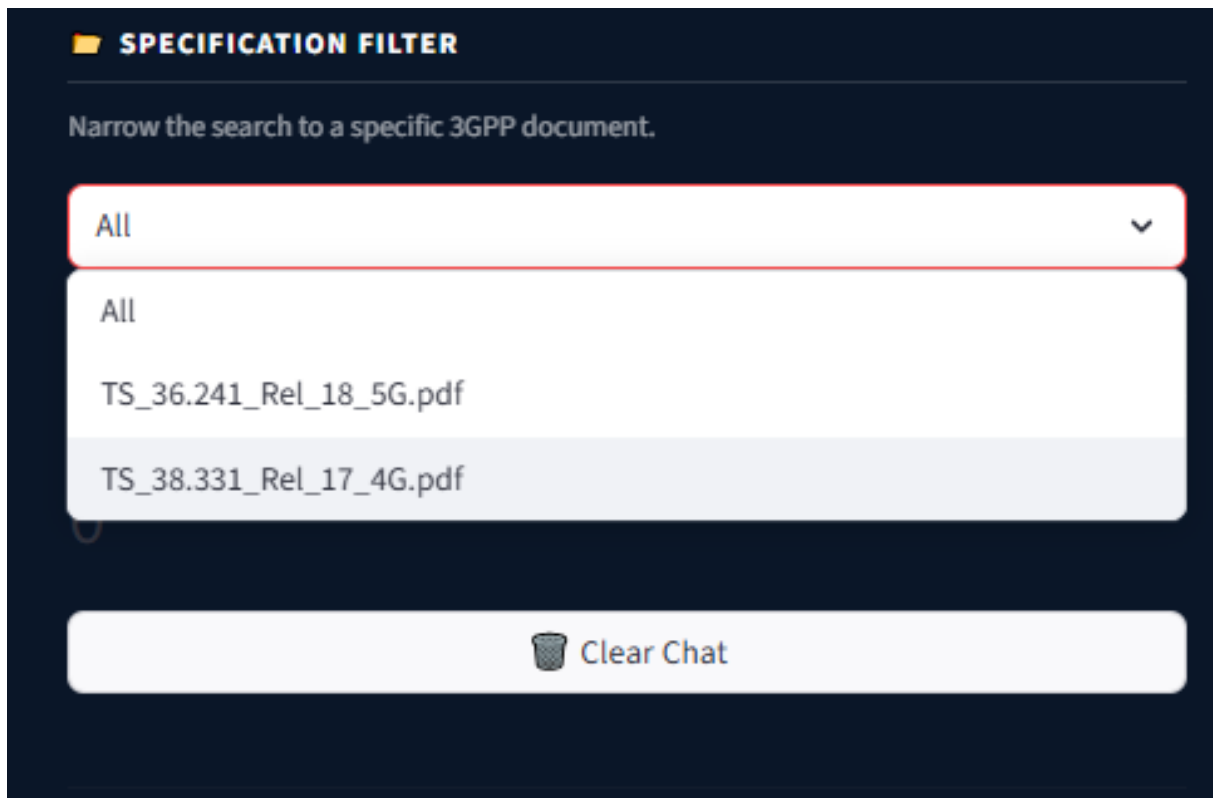


Figure 7.4: Specification Filter allowing users to restrict semantic retrieval to selected 3GPP Technical Specifications.

By eliminating unrelated specifications before semantic retrieval, metadata filtering reduces computational complexity, improves retrieval precision, and minimizes the likelihood of retrieving technically similar but contextually unrelated document sections. This filtering mechanism is particularly valuable when multiple specifications describe related procedures using similar terminology.

7.13 Hybrid Retrieval Pipeline

After applying metadata filtering, the system performs semantic retrieval using a hybrid pipeline that combines dense vector search with neural relevance estimation. This two-stage retrieval strategy balances computational efficiency with retrieval accuracy and forms one of the key engineering contributions of the developed assistant.

Initially, the user’s natural-language question is converted into a dense embedding vector using the same embedding model employed during document indexing. The generated embedding is then submitted to the Qdrant vector database, which performs Approximate Nearest Neighbor (ANN) search to retrieve the Top-K semantically similar document chunks.

Instead of directly forwarding these retrieved chunks to the language model, the proposed system performs an additional refinement stage to improve retrieval quality. This refinement reduces redundant context, removes partially relevant passages, and prioritizes document chunks that best answer the user’s question.

Figure 7.5 presents the execution stages displayed during query processing.



Figure 7.5: Execution stages of the hybrid Retrieval-Augmented Generation pipeline.

The implemented retrieval pipeline consists of the following stages:

1. Query embedding generation.
2. Metadata-aware vector retrieval using Qdrant.
3. Duplicate chunk removal.
4. Cross Encoder semantic re-ranking.
5. Selection of the highest-ranked document chunks.
6. Prompt construction.
7. Large Language Model inference.

This hybrid retrieval strategy significantly improves response quality compared to relying solely on dense vector similarity.

7.14 Cross Encoder Re-ranking

Although dense vector retrieval effectively retrieves semantically related specification sections, the retrieved candidate chunks may still contain partially relevant information. To improve retrieval precision, the Telecom RAG Assistant incorporates a Cross Encoder re-ranking stage before prompt construction.

Unlike embedding models that independently encode the query and document, a Cross Encoder jointly processes both inputs within the same neural network. This architecture enables the model to analyze complex semantic relationships between the user query and each retrieved document chunk, producing a highly accurate relevance score.

Following the initial retrieval stage, every candidate chunk retrieved from Qdrant is evaluated by the Cross Encoder. The model computes an independent relevance score for every query-document pair, after which the retrieved chunks are sorted according to their semantic relevance.

Only the highest-ranked chunks are selected for inclusion within the final prompt supplied to the language model. Consequently, the prompt contains highly relevant engineering context while excluding redundant or weakly related information.

This additional neural re-ranking stage substantially improves the factual accuracy of the generated responses by ensuring that the language model receives only the most informative specification excerpts.

7.15 Prompt Construction

Prompt construction represents the final stage before invoking the Large Language Model. Rather than forwarding only the user’s question, the system constructs a structured prompt that combines the retrieved specification context with carefully designed system instructions.

The prompt consists of three principal components:

1. A system prompt defining the assistant’s behavior and restricting responses to the retrieved engineering context.
2. The highest-ranked specification chunks obtained after semantic retrieval and Cross Encoder re-ranking.
3. The user’s original engineering question.

The system prompt explicitly instructs the language model to generate responses exclusively from the supplied specification excerpts. Whenever the retrieved context does not contain sufficient information to answer the user’s question, the model is instructed to acknowledge the absence of supporting evidence rather than generating speculative or hallucinated information.

The retrieved document chunks preserve their original metadata throughout the prompt construction process. Consequently, the generated response can include references to the corresponding Technical Specification number, release version, section identifier, and page number.

This prompt engineering strategy enables the Telecom RAG Assistant to produce responses that are both conversational and technically reliable while maintaining complete traceability to the original engineering documentation.

7.16 LLM Response Generation

After the prompt has been constructed, it is transmitted to the selected Large Language Model (LLM) through the OpenRouter API. Unlike conventional conversational AI systems that rely solely on the model’s internal knowledge, the Telecom RAG Assistant generates responses exclusively from the contextual information retrieved from the indexed 3GPP Technical Specifications.

The prompt supplied to the LLM consists of three main components:

1. System instructions that define the assistant’s role and constrain its behaviour.
2. The retrieved specification chunks selected after semantic retrieval and Cross Encoder re-ranking.
3. The user’s original engineering question.

The system prompt explicitly instructs the model to answer only using the supplied engineering context. Whenever sufficient information cannot be found within the retrieved specification excerpts, the assistant is instructed to acknowledge the absence of evidence rather than generating speculative information. This prompt engineering strategy significantly reduces hallucination and improves the reliability of generated responses.

After inference, the generated answer is returned together with the supporting references corresponding to the retrieved document chunks. Each response therefore remains directly traceable to the original 3GPP specification, allowing engineers to verify every statement against the official standard.

Figure 7.6 illustrates a generated response describing the LTE Event A3 measurement event.

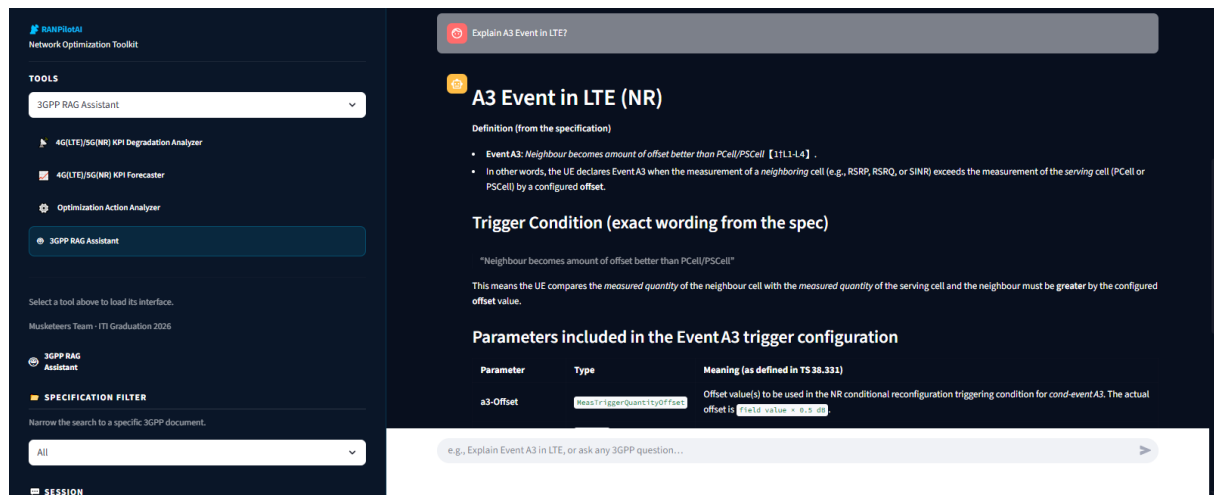


Figure 7.6: Generated engineering response for the LTE Event A3 measurement event.

The generated response contains both explanatory text and structured engineering information, allowing engineers to understand the underlying protocol behavior without manually navigating multiple specification sections.

7.17 User Interface

The Telecom RAG Assistant provides an interactive graphical user interface developed using Streamlit. The interface was designed to simplify interaction with complex engineering documentation while exposing the internal stages of the Retrieval-Augmented Generation pipeline.

The main interface consists of four principal components:

- Module navigation panel.
- Specification filtering controls.
- Conversational chat interface.
- Supporting citation viewer.

The conversational interface enables engineers to formulate technical questions using natural language instead of manually searching specification documents.

Figure 7.7 presents the main interface of the Telecom RAG Assistant.

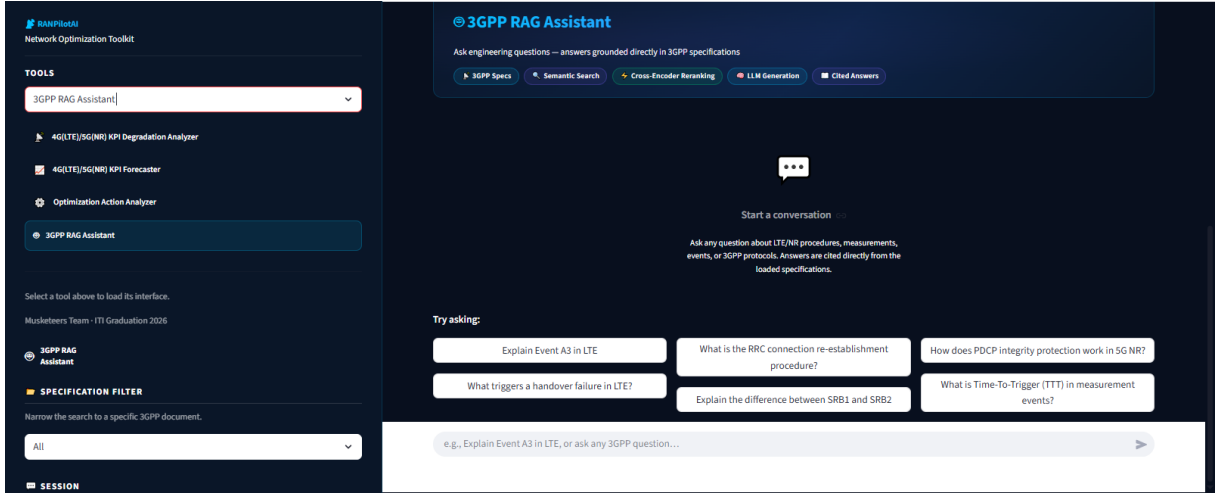


Figure 7.7: Main interface of the Telecom RAG Assistant.

One of the key capabilities of the developed Telecom RAG Assistant is its ability to maintain conversational context across successive user interactions. Rather than treating each query as an isolated request, the assistant leverages the previous questions and responses within the ongoing conversation to better interpret follow-up queries. This contextual awareness enables engineers to ask natural continuation questions without repeatedly providing the complete technical context, resulting in a more intuitive and efficient interaction. For example, after requesting an explanation of a specific LTE or NR procedure, the user may ask a follow-up question using pronouns or abbreviated terminology, and the assistant is able to infer the intended context from the conversation history before retrieving the most relevant specification excerpts. This conversational capability closely resembles the workflow of human technical discussions and significantly improves the usability of the system during troubleshooting and standards exploration. Figure 7.8

illustrates an example in which the assistant correctly interprets a follow-up question regarding the `H0-Conn` field by utilizing the context established in the previous interaction. The generated response remains grounded in the retrieved 3GPP specifications while preserving conversational continuity, allowing engineers to explore related concepts naturally without restarting the search process.

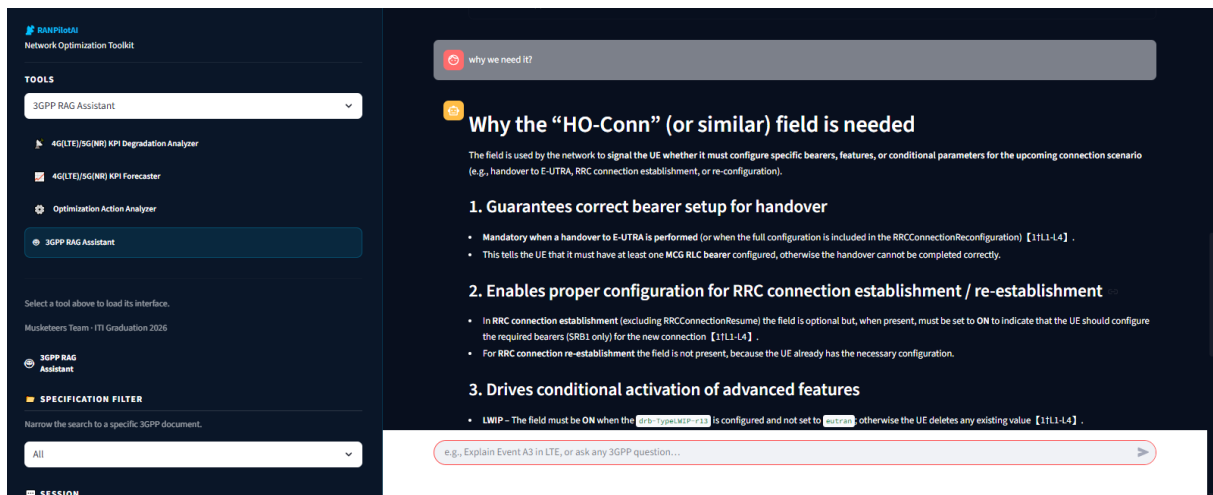


Figure 7.8: Context-aware response generated by the Telecom RAG Assistant using information from previous user interactions within the same conversation.

The assistant also produces concise engineering summaries highlighting the most important technical points extracted from the specifications.

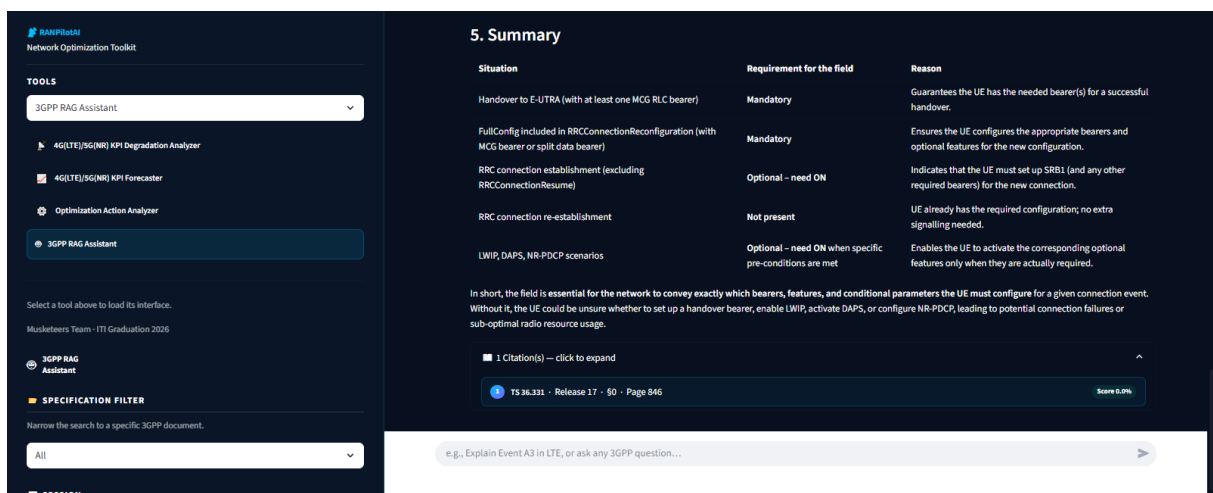


Figure 7.9: Structured engineering summary generated by the Telecom RAG Assistant.

The developed assistant is capable of interpreting engineering tables directly from the indexed specifications.

Figure 7.10 illustrates an example response describing the Packet Data Convergence Protocol (PDCP), while Figure 7.11 presents the corresponding engineering summary generated from the retrieved specification sections.

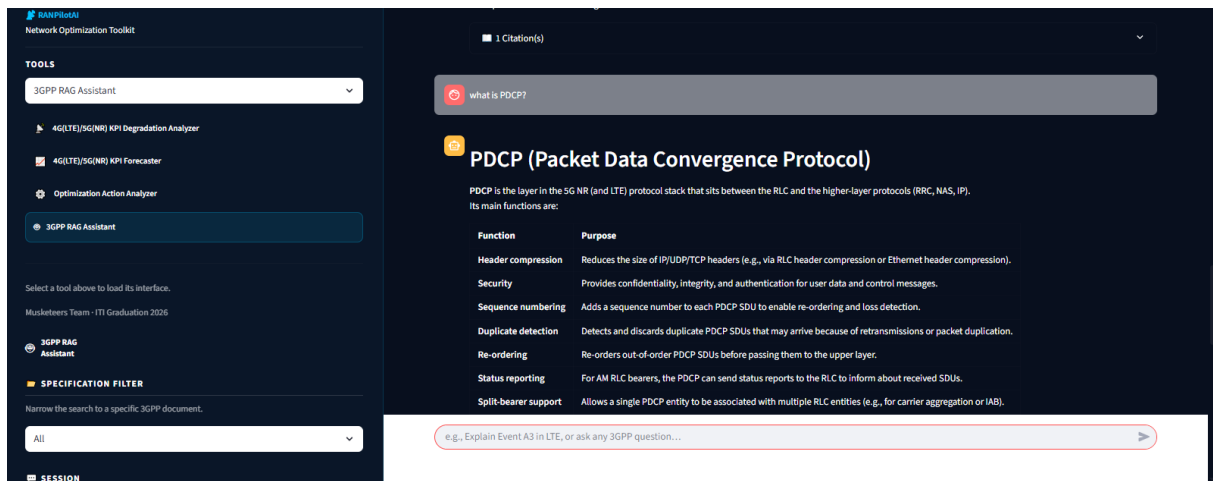


Figure 7.10: Generated response explaining the Packet Data Convergence Protocol (PDCP).

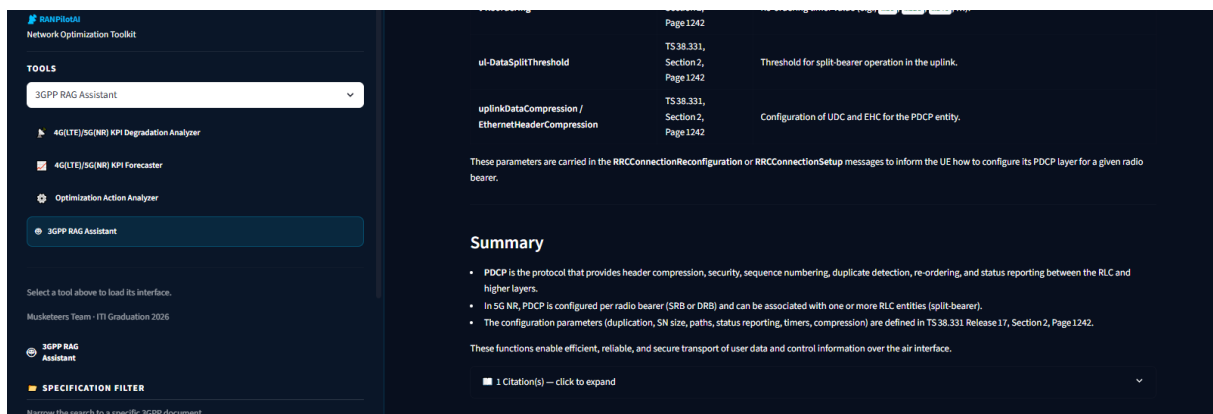


Figure 7.11: Structured engineering summary for the PDCP protocol.

The interface automatically attaches supporting citations to every generated response, allowing engineers to verify the retrieved information directly from the corresponding Technical Specification.

7.18 Performance Evaluation

The developed Telecom RAG Assistant incorporates an integrated profiling mechanism that records the execution time of every stage within the Retrieval-Augmented Generation pipeline. This profiling framework assists developers in identifying computational bottlenecks while providing transparency regarding the internal processing workflow.

For every submitted query, the profiler measures:

- Query embedding time.
- Vector database retrieval time.
- Cross Encoder re-ranking time.
- Prompt construction time.

- Large Language Model inference time.
- Total response latency.

Figure 7.12 illustrates the execution statistics generated during query processing.

```
mahmo@DESKTOP-RJT2JTQ MINGW64 /d/ITI_Graduation_project (main)
Network URL: http://192.168.1.8:8501

[14:59:27] Load Embedding Model | 3915.5 ms
[14:59:27] Init Embedder | 3918.9 ms
[14:59:27] Init Qdrant | 781.5 ms | path=D:\ITI_Graduation_project\RAG_tool\vector_db\qdrant_local
Qdrant collections: ['telecom_3gpp']
Points count: 2246
[14:59:27] Init OpenRouter LLM | 0.0 ms
[14:59:30] Load Reranker Model | 2941.0 ms | device=CPU
[14:59:30] Init Reranker | 2941.3 ms
[15:00:35] Query Embedding | 2149.2 ms
[15:00:35] Embedding Query | 2149.5 ms
[15:00:36] Qdrant Search | 28.7 ms | fetched=12, top_k=6
[15:00:36] Qdrant Retrieval | 29.0 ms | fetched=12
[15:00:36] Metadata Boost | 0.1 ms | query_terms_matched=1
[15:00:36] Remove Duplicates | 0.0 ms | input=12, output=3, removed=9
[15:00:36] Deduplicate | 0.3 ms | input=12, output=3
[15:00:36] Filter Chunks | 0.6 ms | original=3, filtered=2, removed=1
[15:00:36] Filter Context | 0.8 ms
[15:00:38] Reranking | 2720.7 ms | input=2, returned=2, predict_ms=2720.7, sort_ms=0.0, device=CPU
[15:00:38] Context Budget | 0.0 ms | selected=1, total_tokens=2516
[15:00:47] LLM Generate | 8343.6 ms
```

Figure 7.12: Pipeline profiling and execution statistics generated during query processing.

The collected execution statistics provide valuable insight into the computational cost of each pipeline stage and facilitate future optimization of the retrieval and inference process.

7.19 Discussion

The developed Telecom RAG Assistant successfully demonstrates how Retrieval-Augmented Generation can be adapted to the telecommunications domain through the integration of semantic retrieval, metadata-aware filtering, neural re-ranking, and conversational language models. By grounding every generated response in official 3GPP Technical Specifications, the system significantly reduces hallucination while maintaining complete traceability to the original engineering documentation.

Compared with traditional PDF searching, the proposed assistant provides a more intuitive interaction model that enables engineers to obtain accurate technical explanations using natural language. The modular architecture further simplifies future extensions, allowing additional specifications, embedding models, or language models to be incorporated with minimal modifications to the overall system.

7.20 Limitations

Although the developed Telecom RAG Assistant provides accurate and traceable engineering responses, several limitations remain.

The quality of generated responses depends directly on the availability and completeness of the indexed Technical Specifications. Information that has not been indexed cannot be retrieved by the system.

Additionally, the overall response latency is influenced by the selected embedding model, vector database search, Cross Encoder inference, and external Large Language Model response time. Processing very large document collections may therefore require additional optimization or distributed deployment strategies.

Finally, although Retrieval-Augmented Generation substantially reduces hallucination, complete elimination of incorrect responses cannot be guaranteed under all operating conditions.

7.21 Future Improvements

Several enhancements can further improve the proposed Telecom RAG Assistant.

Future work includes supporting additional telecommunication standards beyond LTE and 5G NR, integrating multimodal retrieval capable of interpreting engineering figures and diagrams, incorporating continuous indexing of newly released 3GPP specifications, and evaluating larger domain-specific embedding models.

Future versions may also integrate local Large Language Models to reduce dependency on external inference services while improving privacy and deployment flexibility.

7.22 Chapter Summary

This chapter presented the design and implementation of the Telecom RAG Assistant developed as part of the RANPilotAI platform. Beginning with the motivation and theoretical foundations of Retrieval-Augmented Generation, the chapter described the complete processing pipeline including PDF processing, section-aware chunking, embedding generation, vector storage, metadata filtering, hybrid semantic retrieval, Cross Encoder re-ranking, prompt construction, and language model inference. The implementation demonstrates how modern Artificial Intelligence techniques can be combined with official 3GPP Technical Specifications to provide an intelligent engineering assistant capable of answering complex telecommunication questions with high accuracy and complete source traceability. The presented architecture establishes a practical foundation for AI-assisted standards consultation and represents an important step toward intelligent engineering support systems for future mobile communication networks.

Chapter 8

System Integration

8.1 Introduction

The RANPilotAI platform represents the integration of the intelligent software modules developed throughout this project into a unified engineering environment for Radio Access Network (RAN) optimization. Instead of operating as independent applications, the developed modules are deployed within a single web-based platform that provides centralized navigation, shared software services, and a consistent graphical user interface.

The integrated platform enables RF engineers to perform multiple optimization tasks through a common environment. Network performance can be analyzed, future KPI behaviour can be predicted, optimization recommendations can be obtained, and official 3GPP Technical Specifications can be consulted without leaving the platform. This unified workflow improves engineering productivity while reducing the complexity associated with switching between multiple software tools.

From a software engineering perspective, the platform follows a modular architecture in which each analytical module encapsulates its own processing logic while sharing common infrastructure services. This design simplifies maintenance, improves scalability, and allows future analytical modules to be integrated with minimal architectural modifications.

This chapter describes the integration architecture adopted in the RANPilotAI platform, presents the overall organization of the developed modules, and explains the software engineering principles that guided the implementation of the integrated system.

8.2 Overall Platform Architecture

The RANPilotAI platform adopts a modular architecture in which the Telecom Hub functions as the central access point to all developed analytical modules. Rather than interacting directly with the individual processing engines, users communicate exclusively with the Telecom Hub, which manages application routing, session handling, and interaction with the underlying analytical components.

Figure [8.1](#) illustrates the overall architecture of the integrated platform.

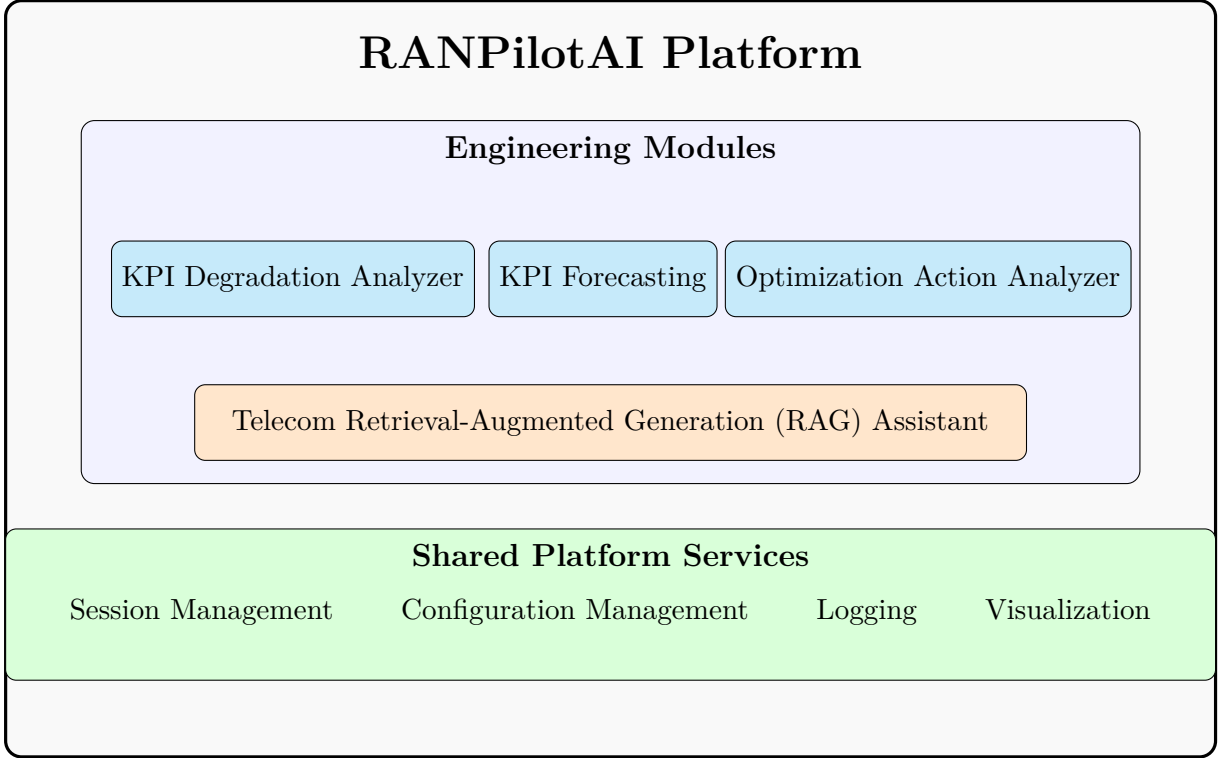


Figure 8.1: Overall architecture of the integrated RANPilotAI platform. The platform groups all analytical modules within a unified engineering environment while providing common platform services shared across the developed applications.

As illustrated in Figure 8.1, the RANPilotAI platform is organized around two principal components. The first component contains the engineering modules responsible for implementing the intelligent analytical capabilities of the platform, whereas the second component provides the shared platform services required by all modules. This organization separates domain-specific processing from common software infrastructure, thereby improving maintainability and facilitating future extensions.

Each analytical module executes independently while sharing common services such as session management, configuration handling, logging, and visualization. Consequently, software duplication is minimized and a consistent user experience is maintained across the entire platform.

8.3 Integration Objectives

The integration strategy adopted in the development of the RANPilotAI platform was guided by several software engineering objectives intended to improve the usability, maintainability, scalability, and extensibility of the system. Rather than focusing solely on integrating multiple analytical tools into a single application, the objective was to establish a modular software architecture capable of supporting future enhancements while maintaining a consistent engineering workflow [10].

The first objective was to provide a unified engineering environment that consolidates the complete RAN optimization workflow into a single platform. Instead of requiring engineers to execute separate applications for KPI analysis, forecasting, optimization planning, and standards consultation, all functionalities are accessible through the Telecom

Hub.

The second objective was to maximize software modularity. Every analytical component was developed as an independent subsystem responsible for a specific engineering task. This separation of responsibilities minimizes software dependencies, simplifies debugging, and enables individual modules to evolve independently without affecting the remaining components.

Another important objective was the reuse of common software services. Instead of duplicating functionalities such as session management, configuration loading, logging, visualization, and navigation within every module, these services are centralized and shared across the entire platform. This approach reduces software complexity while improving maintainability and consistency.

Finally, the architecture was designed with extensibility as a primary consideration. Future analytical modules, artificial intelligence models, or optimization algorithms can be incorporated into the existing framework by integrating them into the Engineering Layer without requiring modifications to the overall platform architecture.

8.4 Layered Software Architecture

From a software engineering perspective, the RANPilotAI platform follows a layered architecture that separates user interaction, application logic, engineering modules, and data management into distinct software layers. Such separation improves maintainability by ensuring that each layer is responsible for a well-defined set of functionalities while communicating only with adjacent layers.

Figure 8.2 presents the layered software architecture adopted in the implementation of the platform.

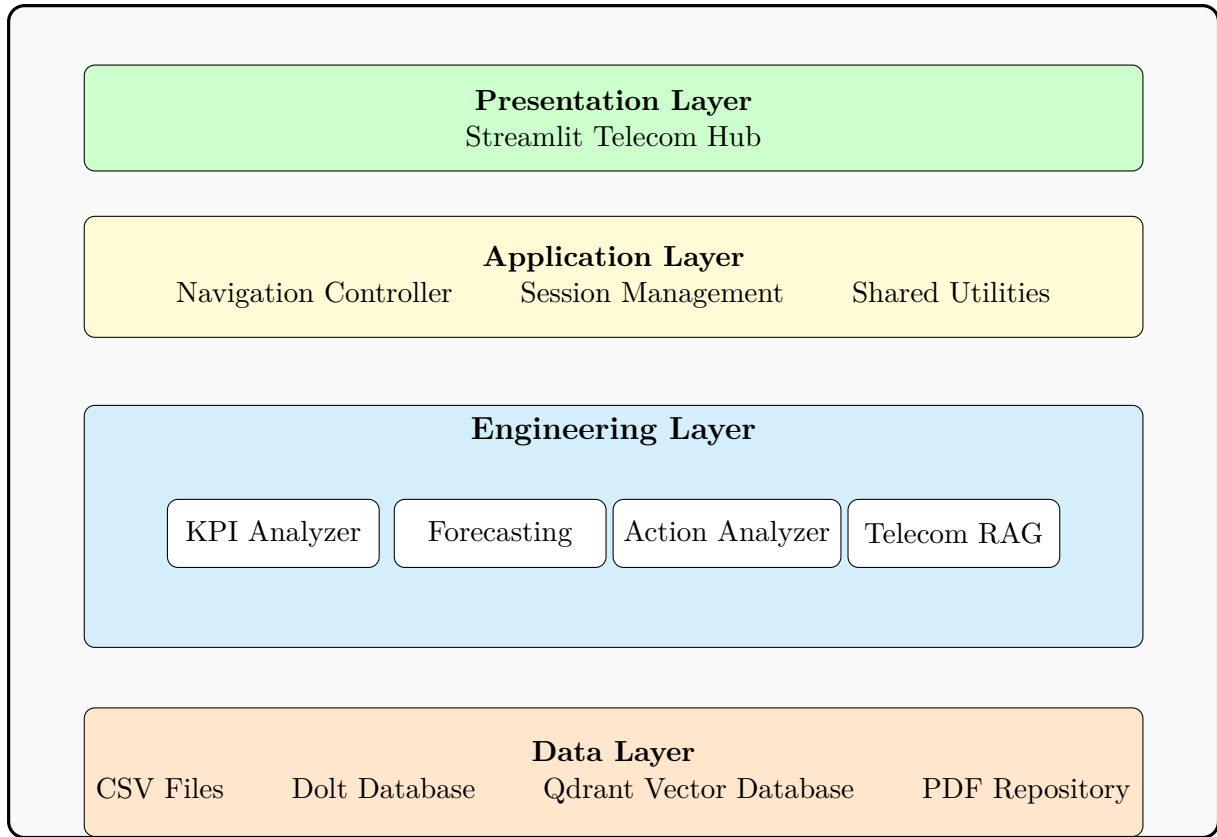


Figure 8.2: Layered software architecture of the RANPilotAI platform. The platform is organized into four logical layers separating presentation, application logic, engineering modules, and data management.

The layered architecture shown in Figure 8.2 separates the software into four logical layers, each responsible for a specific aspect of the platform. The Presentation Layer provides the graphical user interface through which engineers interact with the platform. The Application Layer manages navigation, user sessions, and shared software utilities that are reused across all analytical modules.

The Engineering Layer contains the four intelligent modules developed throughout this project. Each module executes independently while relying on the common services provided by the Application Layer. This separation ensures that modifications to one analytical component do not affect the remaining modules.

Finally, the Data Layer provides persistent storage and external resources required by the engineering modules. Structured datasets are maintained in CSV files, version-controlled engineering data are stored using Dolt, semantic document embeddings are managed by the Qdrant vector database, and the official 3GPP Technical Specifications are maintained within the PDF repository. This organization establishes a clear separation between data management and analytical processing while simplifying future maintenance and platform expansion [9].

8.5 Module Integration

The RANPilotAI platform integrates four independent analytical modules into a unified engineering environment while preserving the autonomy of each subsystem. Rather

than tightly coupling the modules, the integration strategy follows a modular design in which every component performs a dedicated engineering task and communicates with the Telecom Hub through standardized interfaces.

Each analytical module executes its own processing pipeline and maintains its own internal implementation. Consequently, modifications to one subsystem do not require changes to the remaining modules. This loose coupling improves software maintainability, facilitates debugging, and enables independent development of future platform extensions.

The KPI Degradation Analyzer is responsible for detecting abnormal network behaviour through KPI analysis. The KPI Forecasting module predicts future network performance using machine learning models. The Optimization Action Analyzer provides optimization recommendations based on historical engineering knowledge, while the Telecom Retrieval-Augmented Generation (RAG) Assistant enables engineers to interact with official 3GPP Technical Specifications using natural language.

Although these modules implement different analytical functionalities, they all follow the same execution model. Every request originates from the Telecom Hub, which identifies the requested service, forwards the request to the corresponding analytical module, receives the generated results, and presents them using a consistent graphical interface.

Figure 8.3 illustrates the interaction between the Telecom Hub and the developed analytical modules.

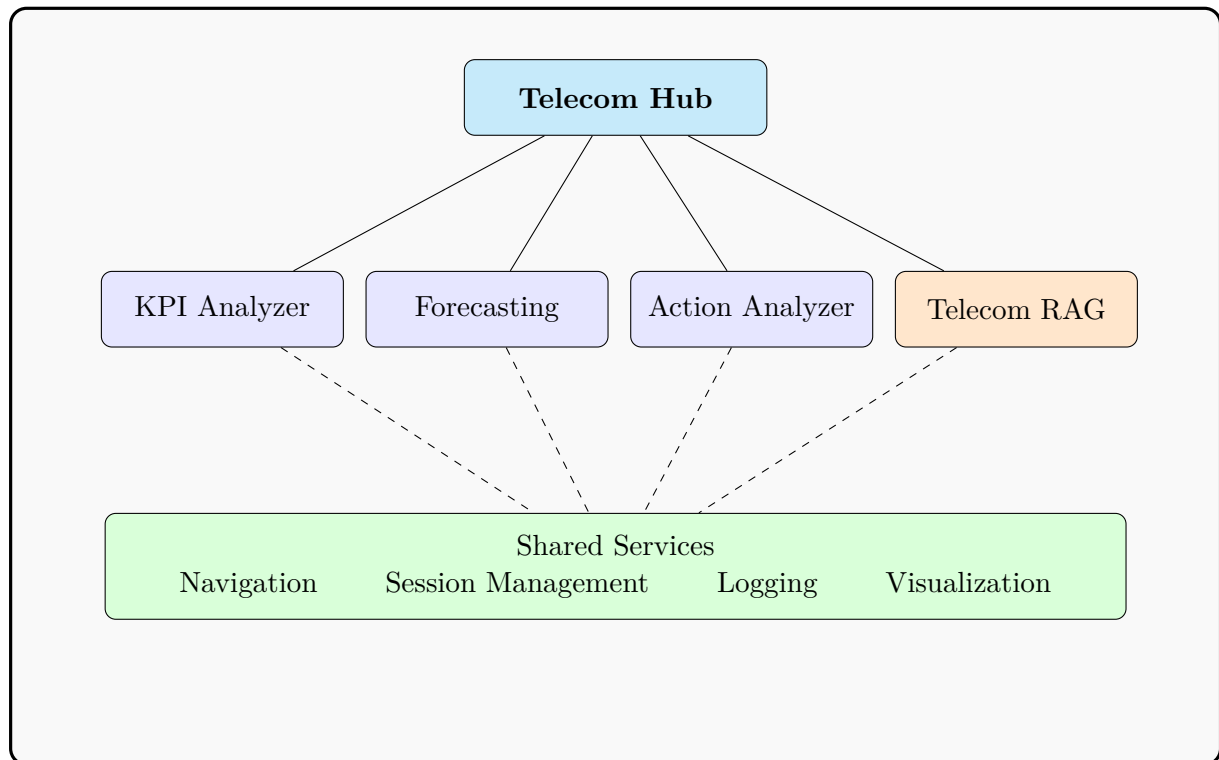


Figure 8.3: Module interaction within the RANPilotAI platform. The Telecom Hub coordinates user requests while common software services are shared among all analytical modules.

As illustrated in Figure 8.3, all user requests are processed through the Telecom Hub before reaching the corresponding analytical module. This centralized coordination simplifies application navigation while preserving complete independence between the engineering subsystems.

The shared service layer provides reusable software functionality required by every module, including session management, application navigation, logging, and visualization. Consequently, common software components are implemented only once, reducing redundancy and improving maintainability.

8.6 Shared Components

Although each analytical module implements a different engineering task, several software components are shared throughout the platform. Centralizing these functionalities simplifies software maintenance while ensuring a consistent user experience.

The principal shared components include:

- **Navigation Manager**

Responsible for routing users between the integrated analytical modules through the Telecom Hub.

- **Session Manager**

Maintains user selections, application state, and module-specific parameters throughout the execution session.

- **Configuration Manager**

Loads configuration files, model parameters, database connections, and application settings required by the analytical modules.

- **Logging Service**

Records application events and execution information to simplify debugging and future maintenance.

- **Visualization Engine**

Provides reusable plotting utilities and standardized graphical components that ensure consistent visualization across the platform.

Figure 8.4 presents the logical organization of the shared software components.

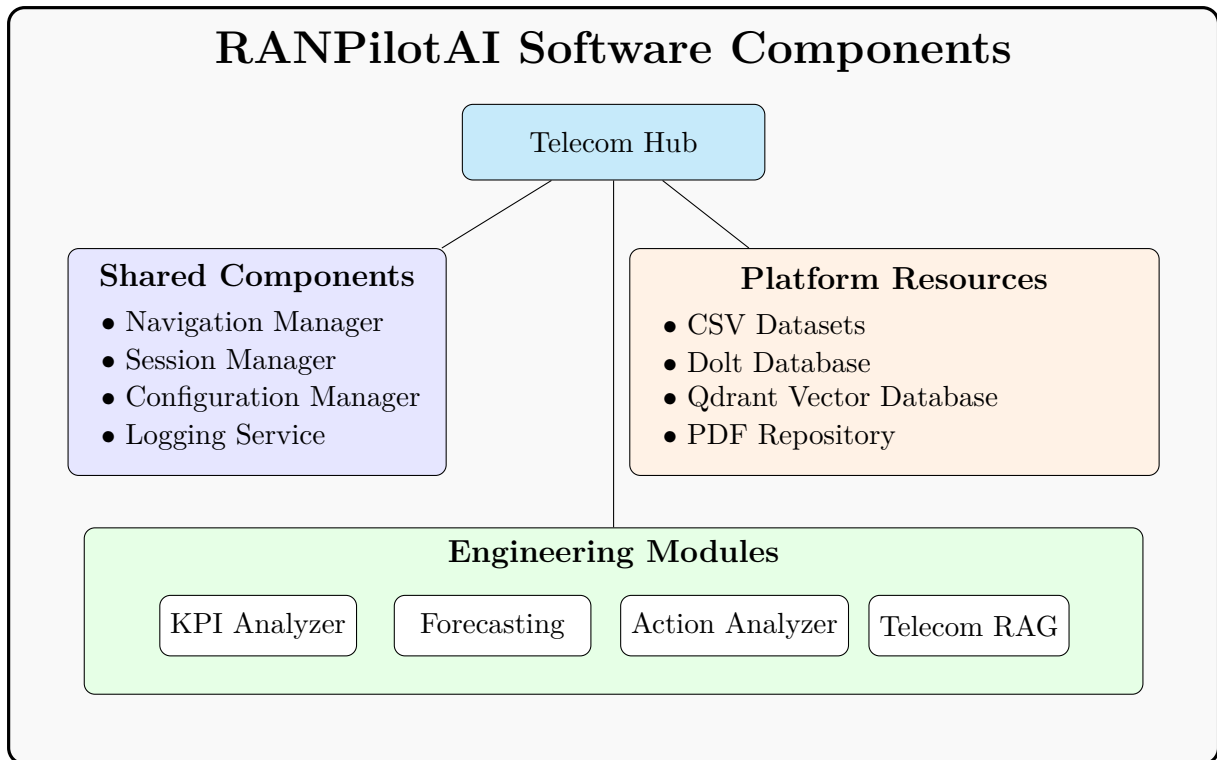


Figure 8.4: Component organization of the RANPilotAI platform showing the relationship between the Telecom Hub, shared software components, platform resources, and engineering modules.

Figure 8.4 illustrates the logical organization of the software components comprising the RANPilotAI platform. The Telecom Hub coordinates the interaction between users, shared software services, engineering modules, and platform resources. Shared components provide common functionality reused throughout the application, while the Engineering Modules implement the domain-specific analytical capabilities presented in the previous chapters.

Separating reusable software services from domain-specific modules significantly improves software maintainability by reducing duplication and establishing clear boundaries between infrastructure components and analytical functionality. Likewise, the explicit separation of platform resources enables individual modules to access common datasets, repositories, and vector databases without introducing unnecessary dependencies between the analytical subsystems.

8.7 Data Flow Between Modules

The analytical modules integrated within the RANPilotAI platform exchange information through a centralized execution model coordinated by the Telecom Hub. Instead of allowing direct communication between individual modules, every user request is first processed by the Telecom Hub, which determines the selected analytical service and forwards the request to the corresponding processing engine. This centralized data flow simplifies software maintenance, reduces coupling between modules, and establishes a consistent execution model throughout the platform.

Although each analytical module implements a different processing pipeline, all mod-

ules follow the same execution sequence. User inputs are collected through the graphical interface, processed by the selected analytical engine, and finally returned to the Telecom Hub for visualization. Depending on the selected module, additional resources such as CSV datasets, the Dolt database, the Qdrant vector database, or the PDF repository may be accessed during execution.

Figure 8.5 illustrates the overall data flow within the RANPilotAI platform.

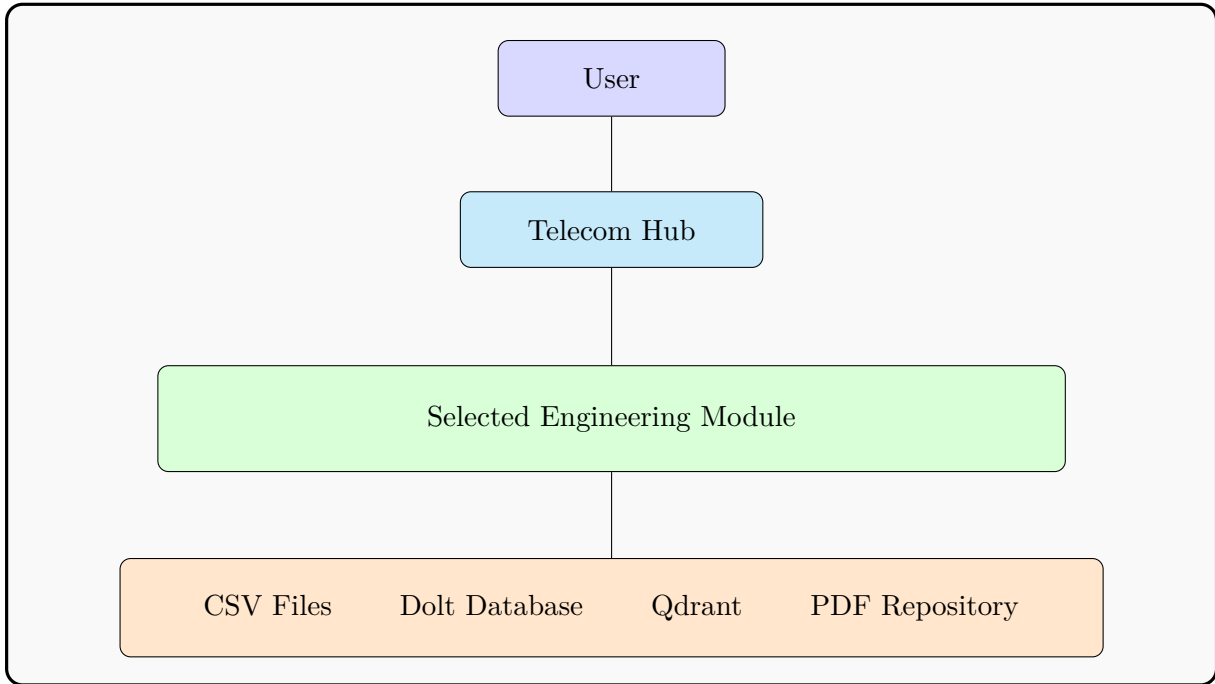


Figure 8.5: Overall data flow within the RANPilotAI platform. All requests are coordinated by the Telecom Hub before reaching the selected engineering module.

As shown in Figure 8.5, the Telecom Hub acts as the intermediary between the user and the analytical modules. Every engineering request follows the same execution model regardless of the selected module. This unified data flow simplifies the software architecture and enables the platform to incorporate additional analytical modules without modifying the existing execution framework.

8.8 User Navigation Workflow

The graphical user interface of RANPilotAI was designed to provide a simple and intuitive navigation experience. Rather than exposing the internal complexity of the analytical modules, users interact only with the Telecom Hub, which presents all available engineering tools through a unified interface.

Figure 8.6 illustrates the workflow followed by a typical user during interaction with the platform.

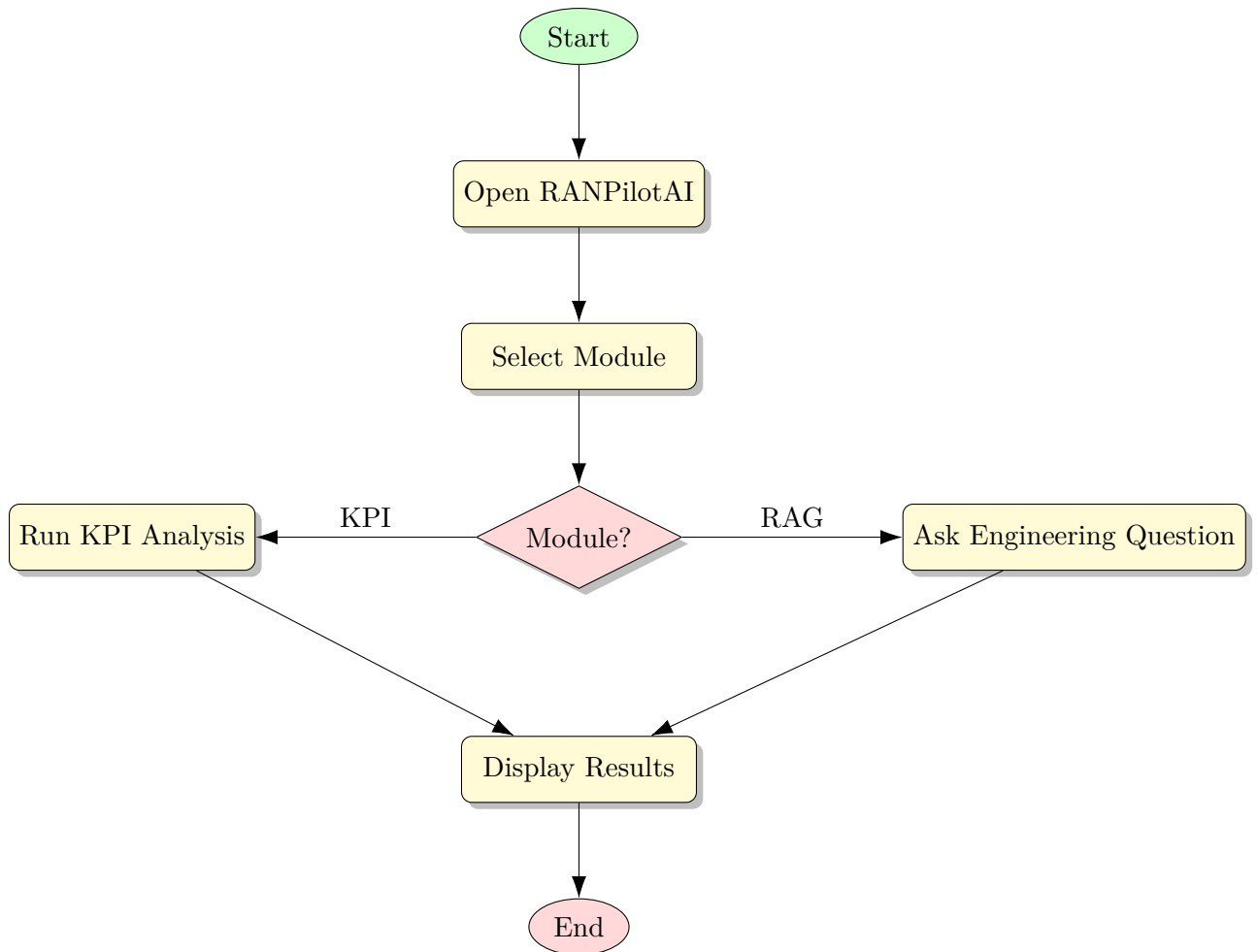


Figure 8.6: Typical user navigation workflow within the RANPilotAI platform.

The workflow begins when the user launches the platform and selects one of the available analytical modules. The Telecom Hub then forwards the request to the selected subsystem, waits for the analytical results, and presents them through the graphical interface. This interaction model remains identical regardless of the selected engineering module, thereby ensuring a consistent user experience throughout the platform.

8.9 Deployment Environment

The RANPilotAI platform was designed as a lightweight web-based application that can be deployed on a single workstation without requiring a distributed computing infrastructure. The deployment strategy prioritizes simplicity, portability, and ease of installation while providing sufficient computational resources for the analytical tasks implemented throughout the platform.

The Streamlit framework serves as the presentation layer of the application, providing an interactive graphical user interface through which users access the different engineering modules. Since Streamlit executes as a local web application, users can interact with the platform through a standard web browser without installing additional client software [13].

All engineering modules execute within the same Python runtime environment. This unified execution model simplifies dependency management, facilitates communication

between modules, and enables the reuse of common software components. Furthermore, executing the analytical modules locally reduces network latency and allows the platform to remain operational even in environments with limited Internet connectivity.

The platform relies on several local data resources. Structured engineering datasets are maintained as CSV files, historical engineering information is managed using the Dolt database, semantic document embeddings are stored within the Qdrant vector database, and official 3GPP Technical Specifications are maintained locally as PDF documents. These resources collectively provide the information required by the different analytical modules while remaining independent from external services.

External communication is required only during the execution of the Telecom Retrieval-Augmented Generation (RAG) Assistant. Once relevant engineering information has been retrieved from the local vector database, the generated prompt is forwarded to a Large Language Model (LLM) through the OpenRouter API to produce the final engineering response. Consequently, the majority of platform operations remain local while external communication is limited to natural language generation.

This deployment strategy provides an effective balance between computational efficiency and intelligent reasoning capabilities. Local execution ensures rapid analytical processing and secure management of engineering datasets, whereas cloud-based language models provide advanced natural language understanding without increasing the complexity of the local deployment.

8.10 Platform Scalability

Scalability was considered as one of the principal design objectives during the development of the RANPilotAI platform. Rather than implementing a monolithic software application, the platform adopts a modular architecture in which every analytical component operates as an independent subsystem connected through the Telecom Hub.

This modular organization enables future engineering modules to be incorporated without modifying the existing analytical components. New machine learning models, optimization algorithms, or AI assistants can be integrated by implementing the required processing logic and registering the module within the Telecom Hub navigation framework.

The shared software infrastructure further improves scalability by centralizing common functionalities such as session management, configuration loading, logging, visualization, and application routing. Since these services are reused by all analytical modules, the integration of additional functionalities does not introduce unnecessary software duplication.

The platform architecture also supports future migration toward distributed deployment environments. Although the current implementation executes on a single workstation, the modular software organization allows individual analytical modules to be deployed independently as web services or containerized applications. Such an architecture would enable computational workloads to be distributed across multiple processing nodes while preserving the unified user experience provided by the Telecom Hub.

Finally, the data management layer was designed to accommodate continuous expansion. Additional engineering datasets, telecommunications standards, optimization knowledge bases, and vector collections can be incorporated without requiring structural modifications to the software architecture. This capability ensures that the platform can evolve alongside future mobile communication technologies and emerging artificial intelligence techniques.

8.11 Integration Challenges

The integration of multiple analytical modules into a unified engineering platform introduced several software engineering challenges throughout the development process. Although each module was developed independently, integrating them into a single application required careful architectural decisions to ensure consistency, maintainability, and reliable execution.

One of the primary challenges involved coordinating heterogeneous analytical components. The developed modules employ different computational approaches, including traditional machine learning models, time-series forecasting algorithms, retrieval-based information systems, and Large Language Models. Establishing a unified execution model capable of supporting these diverse computational paradigms required the implementation of a common interaction framework through the Telecom Hub.

Another challenge concerned software dependency management. Since each analytical module relies on specialized Python libraries and external packages, maintaining compatibility between software components required careful version management and standardized project organization. Consolidating all dependencies into a common execution environment significantly simplified deployment while reducing software conflicts.

Maintaining a consistent graphical user interface also represented an important engineering consideration. Despite the diversity of analytical tasks performed by the individual modules, users should experience a unified interaction model throughout the platform. Standardizing visualization components, navigation mechanisms, and application layouts therefore became an essential aspect of the integration strategy.

Managing local and external resources presented an additional challenge. While most analytical operations rely exclusively on locally available datasets and vector databases, the Telecom RAG Assistant communicates with external language models through the OpenRouter API. Coordinating local retrieval with cloud-based response generation required careful management of network communication while preserving application responsiveness.

Finally, ensuring extensibility without sacrificing maintainability influenced many architectural decisions throughout the project. The adoption of modular software components, reusable utilities, and standardized interfaces enables future platform expansion while minimizing the impact of modifications on the existing system.

8.12 Future Extensions

Although the current implementation of the RANPilotAI platform provides a comprehensive intelligent environment for Radio Access Network (RAN) optimization, its modular software architecture enables several opportunities for future enhancement. The separation between the analytical modules and the Telecom Hub allows new functionalities to be incorporated without significant architectural modifications, ensuring that the platform can evolve alongside future telecommunications technologies.

One potential extension involves the integration of real-time network monitoring capabilities. Instead of relying solely on offline engineering datasets, future versions of the platform could establish direct communication with Operations Support Systems (OSS) or Network Management Systems (NMS) to analyze live network measurements. Such integration would enable continuous KPI monitoring and provide engineers with immediate analytical feedback based on current network conditions.

Another promising enhancement concerns the deployment architecture. Although the current implementation executes on a single workstation, the platform can be migrated to a cloud-native environment using containerization technologies such as Docker and orchestration platforms such as Kubernetes. This deployment model would improve scalability, simplify software maintenance, and enable concurrent access by multiple engineers.

The analytical capabilities of the platform can also be expanded by incorporating additional machine learning and artificial intelligence models. Future versions may include anomaly detection algorithms, reinforcement learning techniques for automatic optimization, predictive maintenance models, and advanced traffic forecasting approaches. Since each analytical component operates independently, these capabilities can be integrated while preserving the existing software architecture.

The Telecom Retrieval-Augmented Generation (RAG) Assistant can also be enhanced by integrating domain-specific language models trained exclusively on telecommunications literature and vendor documentation. Such models would improve the accuracy of technical responses and provide more specialized engineering recommendations. Furthermore, expanding the document repository to include vendor-specific implementation guides, operational manuals, and optimization procedures would significantly increase the knowledge available to the assistant.

Finally, future versions of RANPilotAI may provide collaborative engineering capabilities through multi-user authentication, centralized project management, shared engineering workspaces, and automated report generation. These features would transform the platform from an individual engineering assistant into a collaborative decision-support system capable of supporting larger operational teams.

8.13 Chapter Summary

This chapter presented the system integration strategy adopted during the development of the RANPilotAI platform. Unlike the previous chapters, which focused on the implementation of individual analytical modules, this chapter described how these independent components were combined into a unified engineering environment capable of supporting multiple Radio Access Network optimization tasks.

The chapter introduced the overall platform architecture, the layered software organization, the interaction between analytical modules, and the shared software components that provide common functionality across the platform. Furthermore, the deployment environment, scalability considerations, and software engineering challenges encountered during system integration were discussed in detail.

The modular architecture adopted throughout the development process establishes a flexible and extensible software foundation that simplifies future platform evolution while maintaining a consistent user experience. By integrating machine learning models, Retrieval-Augmented Generation techniques, optimization algorithms, and engineering knowledge within a single software environment, RANPilotAI provides a comprehensive intelligent platform for supporting modern Radio Access Network optimization activities.

The following chapter presents the overall conclusions of this work, summarizes the principal contributions of the project, and outlines potential directions for future research and development.

Chapter 9

Testing Strategy

9.1 Introduction

9.1.1 Purpose

This document defines the complete testing strategy for the LTE RAN KPI Analysis Tool. The objective is to verify that every module performs according to its functional requirements while ensuring correctness of KPI degradation calculations, root cause analysis (RCA), anomaly detection, dashboard preparation, and report generation.

- **The test plan validate:**
- Functional correctness
- Statistical correctness
- Data quality handling
- Robustness against invalid datasets
- Performance on large LTE datasets
- Integration between analysis modules

9.2 Scope

The following modules are included in the test scope:

9.3 Testing Levels

9.3.1 Unit Testing

Each function is tested independently .

Module	Tested
Excel Import	✓
Data Cleaning	✓
KPI Configuration	✓
Baseline Calculation	✓
Day-by-Day Degradation Engine	✓
Statistical Significance Test	✓
RCA Engine	✓
Cell Outage Detection	✓
Multi-KPI Analysis	✓
Dashboard Dataset Generator	✓
Degraded Cell Removal	✓
Word Report Generator	✓
Visualization Module	✓
Logging	✓
Error Handling	✓

Examples:

- `calculate_degradation()`
- `compute_day_by_day_degradation()`
- `perform_ttest()`
- `validate_columns()`
- `detect_cell_outage()`
- `generate_outage_rca()`

9.3.2 Integration Testing

Validates interaction between: Excel Reader → Cleaning → Baseline → KPI Analysis → RCA → Dashboard → Word Report.

9.3.3 System Testing

Complete execution from loading Excel until generating Dashboard, Word report, Summary, and RCA.

9.4 Functional Test Cases

TC ID	Requirement	Test Scenario	Test Input	Expected Result
TC-001	Import LTE KPI dataset	Upload a valid Excel file	Valid LTE KPI Excel	Dataset imported successfully
TC-002	Validate input file	Upload corrupted Excel file	Corrupted file	Validation error displayed
TC-003	Validate required columns	File missing mandatory columns	Missing KPI column	Missing-column error generated
TC-004	Clean column names	Import dataset with extra spaces	Inconsistent headers	Column names standardized
TC-005	Convert numeric values	Dataset contains numeric strings	Mixed numeric formats	Numeric values converted correctly
TC-006	Parse dates	Dataset contains valid dates	Date column	Dates converted successfully
TC-007	Remove invalid dates	Dataset contains invalid dates	Invalid date values	Invalid rows excluded
TC-008	Load KPI configuration	Initialize KPI configuration	Configuration file	All KPI definitions loaded
TC-009	Select KPI	Choose one KPI	Accessibility KPI	KPI selected correctly
TC-010	Invalid KPI selection	Select undefined KPI	Invalid KPI name	Value Error generated
TC-011	Last Week baseline	Select Last Week mode	Historical dataset	Same weekday baseline calculated
TC-012	Rolling baseline	Select 4-week mode	Historical dataset	4-week median baseline calculated
TC-013	Historical fallback	Missing baseline values	Incomplete history	Historical fallback applied
TC-014	Minimum baseline	No historical baseline	Empty baseline	Minimum baseline assigned
TC-015	Prevent imputation	Missing recent values	Recent window missing	Recent values remain unchanged
TC-016	Day-by-day degradation	Execute degradation calculation	Recent and baseline	Median degradation calculated
TC-017	Ratio KPI calculation	Analyze ratio KPI	Success rate KPI	Difference degradation calculated
TC-018	Volume KPI calculation	Analyze traffic KPI	Throughput KPI	Percentage degradation calculated
TC-019	Zero baseline handling	Baseline equals zero	KPI values	No divide-by-zero error
TC-020	Near-zero baseline	Extremely small baseline	Small KPI values	Winsorized degradation applied
TC-021	Spike-day robustness	One abnormal day	Multi-day history	Median unaffected by spike
TC-022	Missing comparison day	Missing recent day	Partial dataset	Cell marked incomplete
TC-023	Baseline recovery	Missing baseline days	Sparse history	Baseline resolved using fallback
TC-024	Threshold evaluation	KPI exceeds threshold	High degradation	Status = Degraded
TC-025	Normal classification	KPI below threshold	Low degradation	Status = Normal
TC-026	Statistical significance	Enable t-test	KPI observations	p-value calculated

TC ID	Requirement	Test Scenario	Test Input	Expected Result
TC-028	Invalid counter	Negative KPI values	Invalid counters	Values quarantined
TC-029	Sentinel value	Sentinel values exist	Sentinel inputs	Converted to NaN
TC-030	Extreme value	Unrealistic KPI values	Outliers	Recorded in quarantine
TC-031	Incomplete data report	Missing baseline/recent days	Partial dataset	Incomplete report generated
TC-032	Single KPI analysis	Analyze selected KPI	Accessibility KPI	Analysis completed successfully
TC-033	Analyze all KPIs	Run batch analysis	Complete dataset	All KPIs analyzed
TC-034	KPI failure isolation	Force one KPI failure	Invalid KPI	Remaining KPIs continue processing
TC-035	KPI summary gen.	Complete batch analysis	Analysis output	Summary table generated
TC-036	Combine degraded	Multiple degraded KPIs	Analysis results	Combined cell list created
TC-037	Cell outage detection	DL=0 and UL=0	Traffic counters	Cell classified as outage
TC-038	Normal traffic val.	DL>0 or UL>0	Traffic counters	No outage detected
TC-039	Root Cause Analysis	Related counters available	Degraded KPI	Root cause identified
TC-040	Multiples cause	Multiple degraded counters	KPI dataset	Multi-cause analysis generated
TC-041	Missing counters	Related counters absent	KPI dataset	Manual investigation rec.
TC-042	RCA recommendation	RCA completed	RCA output	Correct recommendations generated
TC-043	Remove degraded	Remove all degraded cells	Combined list	Clean dataset produced
TC-044	Remove recent cells	Recent-only removal	Recent degradation	Recent records removed
TC-045	Remove cells by KPI	KPI filter enabled	Selected KPI	Only filtered cells removed
TC-046	Empty degraded list	No degraded cells	Normal dataset	Original dataset returned
TC-047	Dashboard gen.	Generate dashboard data	Clean dataset	Dashboard-ready data produced
TC-048	Generate Word report	Export analysis	Completed analysis	Report generated successfully
TC-049	Include RCA in report	Export report	RCA results	RCA section included
TC-050	Enhancement potential	Generate report	Original & cleaned	Enhancement potential calculated

9.5 Boundary Testing

Test	Expected
Empty Excel	Error
One row	Process
One day	Incomplete
Zero threshold	All degraded
Very high threshold	None degraded
Maximum degradation	Clipped
Large KPI values	Stable

9.6 Negative Testing

- Missing KPI column / Cell ID / Site Name / Cell Name / Local Cell ID
- Wrong date format
- Duplicate rows
- Non-numeric counters, Infinite values, NaN everywhere
- Empty baseline or empty recent window
- Invalid KPI name

9.7 Regression Testing

Every software update must verify:

1. KPI calculations
2. RCA accuracy
3. Baseline calculation
4. Dashboard output
5. Report generation
6. Threshold handling
7. Data cleaning
8. Statistical significance
9. Cell removal

9.8 Acceptance Criteria

The project is accepted when :

- All KPIs are analyzed successfully .
- Day-by-day degradation calculations are correct .
- Statistical tests are executed correctly .
- RCA identifies valid degradation causes .
- Cell outage detection functions correctly .
- Dashboard excludes degraded cells correctly .
- Reports are generated without errors .
- Invalid data is quarantined rather than causing failures .
- Application remains stable if a single KPI analysis fails .
- Performance meets expected runtime for large LTE datasets .

9.8.1 Test Coverage Summary

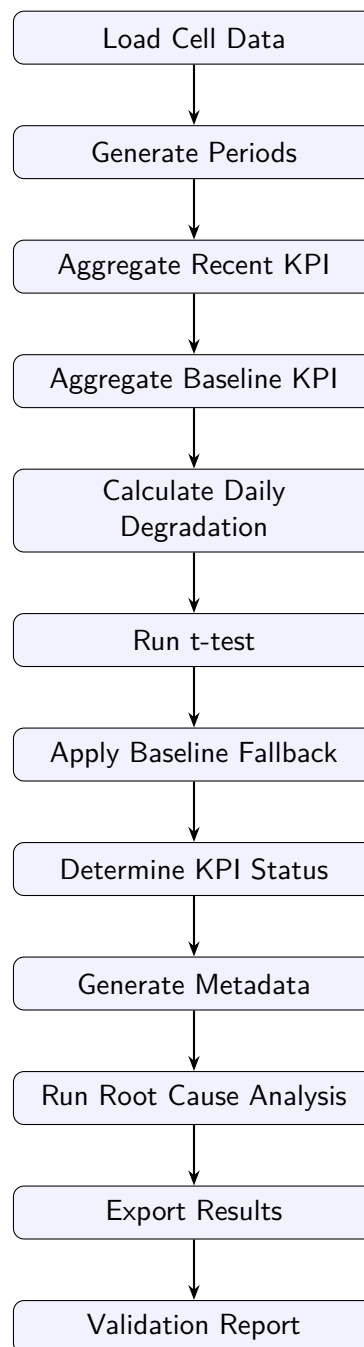
Area	Coverage
Excel Import, Data Cleaning, KPI Configuration	100%
Baseline Resolution, Day-by-Day Degradation	100%
Statistical Testing, Data Quality Validation	100%
RCA Engine, Cell Outage Detection, Multi-KPI	100%
Degraded Cell Removal, Dashboard Prep, Reports	100%
Logging, Error Handling, Boundary & Negative Testing	100%

9.9 Testing Framework Architecture

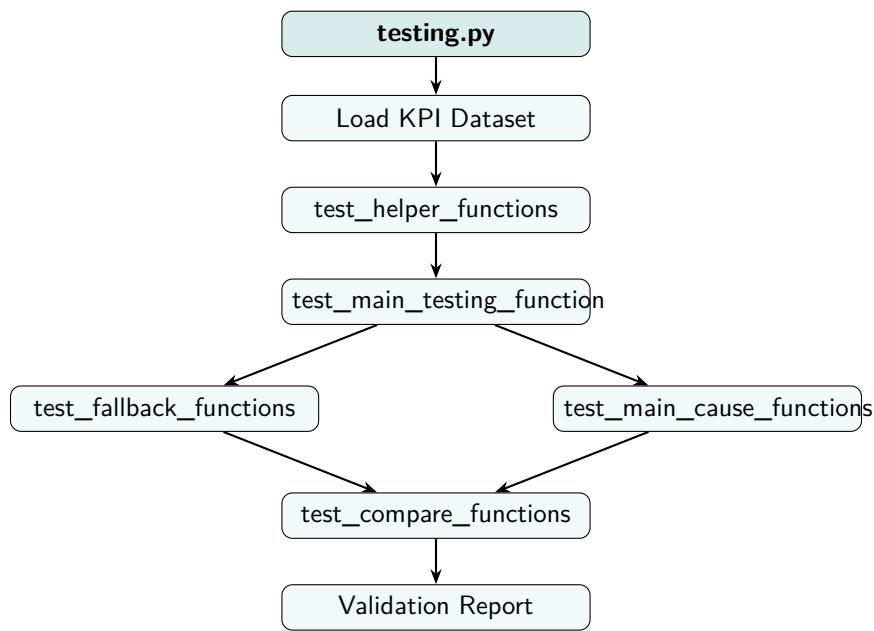
The framework uses a modular architecture where each module validates one part of the pipeline independently :

```
testing.py
|- test_helper_functions.py      (Helper and utility function testing)
|- test_fallback_functions.py    (Baseline fallback algorithm testing)
|- test_main_cause_functions.py  (Root cause analysis testing)
|- test_main_testing_function.py (Independent KPI analysis)
\-- test_compare_functions.py    (Comparison against production output)
```


9.9.1 Workflow Diagram



9.9.2 Modular Subsystem Map



9.10 Test Execution Observations

9.10.1 Test Case: TC-001

Title: Verify KPI Analysis Using Custom Range Baseline

- **Expected Result:** Analysis completes successfully; outputs match main implementation; exact number of degraded cells detected .
- **Actual Observation:** Executed without error; 32 common cells matched with 0 differences

Running analysis for KPI: DL Traffic , Baseline Mode: custom_range , start c

Missing rows for KPI: DL Traffic, Days: 7, Baseline Mode: custom_range:

```
Total common rows: 32
Rows only in test_output: 0
Rows only in output_df: 0
Rows with differences: 0
Empty DataFrame
Columns: [LocalCell Id, recent_avg_kpi, baseline_avg_kpi, kpi_degradation_r
Index: []
Number of significant results in test output: 32
Number of significant results in main output: 32
```

9.10.2 Test Case: TC-002

Title: Verify KPI Analysis Using Last Week Baseline

- **Expected Result:** KPI calculations and RCA match exactly .
- **Actual Observation:** Executed successfully; identified 34 significant degraded cells identically in both implementations .

Missing rows for KPI: DL Traffic , Days: 4, Baseline Mode: last_week:
Empty DataFrame

Columns: [LocalCell Id, recent_avg_kpi, baseline_avg_kpi, kpi_degradation_r

Index: []

Number of significant results in test output: 34

Number of significant results in main output: 34

~~~~~

# Chapter 10

## Conclusion and Future Work

### 10.1 Conclusion

This graduation project presented **RANPilotAI**, an integrated, AI-powered platform designed to automate and augment the daily workflows of Radio Frequency optimization engineers working in 4G LTE and 5G New Radio mobile networks. The platform addresses a fundamental gap in existing tooling: the absence of a unified, intelligent environment capable of covering the complete RAN optimization cycle — from performance degradation detection and trend forecasting, through configuration change impact assessment, to authoritative standards consultation.

The work delivered four specialized analytical modules, all integrated into a single web-based engineering environment called the **Telecom Hub**.

#### KPI Degradation Analyzer

The KPI Degradation Analyzer demonstrated that statistically rigorous automated detection of network performance degradation is achievable across a broad set of KPI categories without requiring manual threshold tuning or domain-specific scripting for each indicator. The three-layer pipeline — combining Welch’s t-test for statistical confidence scoring, RF-aware root cause inference with configurable severity weighting, and a data quality framework supporting sentinel-value filtering, unit-aware validation, and same-weekday baseline imputation — produced consistent and interpretable degradation reports across 13 KPI families including throughput, accessibility, retainability, mobility, availability, VoLTE, CSFB, and RRC re-establishment. Engineers can upload operator performance data and receive a structured root-cause summary within seconds, replacing a workflow that previously required significant manual effort.

#### KPI Forecasting Engine

The KPI Forecasting Engine demonstrated that combining multiple complementary forecasting models within a single interactive interface provides engineers with both predictive power and interpretability. The XGBoost model, built with leakage-safe cross-KPI feature engineering, captures non-linear relationships and inter-KPI dependencies that classical time-series methods cannot exploit. The Holt-Winters exponential smoothing model provides a transparent seasonal decomposition baseline whose assumptions are easier to audit in operational settings. Together with STL-based seasonality analysis, backtesting

metrics (MAE, RMSE, MAPE), recursive 7-day forward forecasts, and residual diagnostics, the engine enables engineers to anticipate network degradation before it manifests operationally, shifting the optimization workflow from reactive troubleshooting toward proactive planning.

## **Optimization Action Analyzer**

The Optimization Action Analyzer addressed a long-standing challenge in RF optimization practice: the difficulty of establishing a causal connection between specific configuration changes and observed KPI impacts. By integrating Dolt, a version-controlled SQL database, the module provides a reproducible and auditable record of RF configuration history. Grouping same-day commits into unified action hunks, aligning configuration events with KPI time windows using weekday-matched baselines, and presenting results through interactive before/after overlay charts, delta bar plots, timeline views, and summary impact tables gave engineers a reliable, visual mechanism for attributing performance changes to individual optimization actions. This traceability significantly improves the quality of engineering decisions and supports knowledge transfer across operations teams.

## **Telecom RAG Assistant**

The Telecom RAG Assistant demonstrated that Retrieval-Augmented Generation can be applied effectively to highly technical, standards-based engineering domains. The pipeline — comprising section-aware PDF chunking, BGE-large dense embeddings, local Qdrant vector search, cross-encoder re-ranking, and a strict anti-hallucination prompt — consistently produced traceable, citation-backed answers grounded directly in official 3GPP Technical Specifications. By returning the Technical Specification identifier, release version, section number, and page reference alongside every generated response, the assistant preserved the traceability that is essential in professional telecommunications engineering. Engineers were able to query hundreds of pages of protocol specifications in natural language and receive structured, verifiable answers within a fraction of the time required by traditional manual search.

## **System Integration**

The integration of the four modules within the Telecom Hub validated the modular architecture adopted throughout the project. By separating domain-specific analytical logic from shared platform services — including session management, configuration handling, logging, and visualization — the platform achieved a consistent user experience across all modules while minimizing code duplication and simplifying maintenance. The Streamlit framework proved well-suited to rapid development of data-intensive engineering dashboards, providing a consistent layout model and interactive widget library that could be applied uniformly across all four analytical modules. The complete platform is implemented in Python 3.11 and is accessible through a standard web browser without client-side installation.

## Overall Assessment

RANPilotAI successfully demonstrated that a small, cohesive software engineering team can design, implement, and integrate a production-quality AI platform for a specialized engineering domain within a single academic project cycle. The platform reduced the time required for the core activities of KPI performance analysis, trend forecasting, optimization impact assessment, and standards consultation, while maintaining engineering accuracy and full result traceability. The combination of statistical rigor, machine learning, retrieval-augmented generation, and version-controlled data management within a unified interface represents a meaningful step toward fully data-driven RAN optimization practice.

## 10.2 Limitations

Despite the results achieved, several limitations of the current implementation should be acknowledged.

- **Offline-only operation.** The platform currently operates on data uploaded manually by the engineer. There is no live integration with network management systems, Operations Support System (OSS) interfaces, or streaming data pipelines. Consequently, the platform supplements rather than replaces the data export step in the existing engineering workflow.
- **KPI scope.** The KPI Degradation Analyzer and Forecasting Engine were designed and validated against LTE performance data. While the analytical pipeline is applicable to 5G NR KPIs in principle, extended validation against 5G-specific indicators — including gNB-level throughput, PDCP layer metrics, and beam management KPIs — was not completed within the project timeline.
- **RAG knowledge base.** The Telecom RAG Assistant operates on a pre-indexed collection of 3GPP specifications. The accuracy and completeness of answers depend on which specifications have been indexed. Specifications not included in the local index cannot be referenced, and the knowledge base does not update automatically when new 3GPP releases are published.
- **LLM dependency.** The RAG assistant relies on an external Large Language Model for answer generation. The quality, latency, and availability of responses are therefore subject to the limitations and service conditions of the chosen LLM provider.
- **Forecasting horizon.** The KPI Forecasting Engine generates recursive 7-day forward forecasts. Forecast accuracy degrades over longer horizons due to error accumulation in the recursive multi-step prediction strategy, limiting the module's utility for long-term capacity planning.

## 10.3 Future Work

The current platform establishes a strong foundation for several meaningful extensions that would increase its operational value.

- **Live OSS integration.** Connecting the platform to a vendor OSS API or a streaming KPI data pipeline would eliminate the manual data upload step and enable near-real-time degradation detection. This would allow the KPI Degradation Analyzer to function as a continuous monitoring service rather than a batch-analysis tool.
- **5G NR KPI extension.** Extending the KPI schema, root-cause inference rules, and severity weighting system to cover 5G-specific KPIs — including gNB, DU, and CU layer indicators, beam management metrics, and network slicing performance dimensions — would broaden the platform’s applicability to next-generation network operations.
- **Automated action recommendation.** The Optimization Action Analyzer currently reports the measured impact of configuration changes but does not generate recommendations. Integrating a reinforcement learning or optimization model trained on the accumulated action-impact history stored in Dolt could produce proactive, data-driven optimization suggestions tailored to each cell’s historical behavior.
- **RAG knowledge base automation.** Building an automated pipeline that monitors 3GPP publication channels, downloads newly released or updated Technical Specifications, processes them through the chunking and embedding pipeline, and updates the Qdrant vector index would keep the RAG assistant current without manual intervention.
- **Multi-modal forecasting.** Incorporating external contextual features — such as public holiday calendars, major events, and weather data — as exogenous inputs to the XGBoost forecasting model could improve forecast accuracy for KPIs that exhibit strong event-driven demand patterns.
- **Role-based access and multi-tenancy.** Adding authentication, role-based access control, and multi-tenant data isolation would allow the platform to be deployed in a shared engineering environment where multiple optimization teams work concurrently without interference.
- **Exportable reports.** Extending the platform to generate structured PDF or HTML reports summarizing the outputs of each analytical module would integrate it more naturally into existing documentation and change management workflows used by operations teams.

## 10.4 Closing Remarks

The RANPilotAI project demonstrated that the convergence of classical statistical methods, modern machine learning, retrieval-augmented generation, and version-controlled data management — within a carefully integrated engineering platform — can substantially reduce the analytical burden placed on RF optimization engineers. As mobile networks grow in scale and complexity with the continued rollout of 5G New Radio and the emergence of Open RAN architectures, the need for intelligent, automated tooling will only increase.

The platform developed in this project is a practical step in that direction: a working, browser-accessible system that an engineer can deploy today to improve the speed,

consistency, and traceability of network optimization decisions. The modular architecture ensures that as new AI techniques and data sources become available, they can be incorporated without redesigning the platform from the ground up.

It is the hope of the development team that RANPilotAI, in its current form or as the foundation for future extensions, contributes meaningfully to the ongoing effort to make RAN optimization faster, smarter, and more accessible to the engineers who keep mobile networks running.



# Chapter 11

## User Manual

This chapter provides a complete user manual for the RANPilotAI platform. It covers system requirements, installation, launching the Telecom Hub, and step-by-step operating instructions for each of the four analytical modules.

### 11.1 System Requirements

- **Operating System:** Windows 10/11, Linux, or macOS
- **Python:** Version 3.11 or higher
- **RAM:** 8 GB minimum (16 GB recommended for large KPI datasets)
- **Disk Space:** At least 5 GB free (additional space required for 3GPP PDF files and the local vector database)
- **Internet Connection:** Required for the RAG Assistant (LLM API calls via Open-Router); all other modules operate fully offline
- **Web Browser:** Any modern browser (Chrome, Firefox, Edge) for the Streamlit interface

### 11.2 Installation

#### 11.2.1 Step 1 — Install Python

Ensure Python 3.11 or higher is installed. Verify by running:

```
python --version
```

#### 11.2.2 Step 2 — Install Dependencies

From the project root directory, install all required packages with a single command:

```
pip install -r requirements.txt
```

This installs all dependencies for all four tools, including Streamlit, Pandas, NumPy, SciPy, XGBoost, Statsmodels, Sentence Transformers, Qdrant, PyMuPDF, and python-docx.

### 11.2.3 Step 3 — Configure the RAG Assistant (First Time Only)

If you intend to use the Telecom RAG Assistant, an OpenRouter API key is required:

1. Copy the environment template:

```
cp RAG_tool/.env.example RAG_tool/.env
```

2. Open `RAG_tool/.env` and enter your OpenRouter API key:

```
OPENROUTER_API_KEY=your_api_key_here
```

3. Place your 3GPP specification PDF files in `RAG_tool/data/` using the naming convention:

```
TS_<NUMBER>_Rel_<RELEASE>.pdf
```

For example: `TS_36.331_Rel_17.pdf`

4. Run the ingestion pipeline to index the PDFs into the local vector database:

```
python -m app.ingest --all
```

Note: The first run downloads the BGE-large embedding model ( $\approx 1.3$  GB).

### 11.2.4 Step 4 — Configure the Optimization Action Analyzer (If Required)

The Optimization Action Analyzer requires a running Dolt SQL server with two tables: `network_kpis` and `network_configs`. Install Dolt from <https://docs.dolthub.com/introduction/installation> and start the server on the default port before launching the platform.

## 11.3 Launching the Platform

The entire platform is launched through a single entry point from the project root directory:

```
streamlit run telecom_hub.py
```

After a few seconds, Streamlit will display a local URL (typically `http://localhost:8501`). Open this URL in your web browser. The Telecom Hub landing page will appear, displaying cards for all four analytical tools.

## 11.4 Navigating the Telecom Hub

The Telecom Hub interface consists of two areas:

- **Left Sidebar:** A dropdown menu listing all four tools. Select any tool to load it in the main area.
- **Main Content Area:** Displays the landing page or the selected tool's full interface.

To switch between tools, use the sidebar dropdown at any time. No page reload or server restart is required.

## 11.5 Tool 1 — KPI Degradation Analyzer

### 11.5.1 Purpose

Automatically detects performance degradation in LTE and 5G NR cells by comparing a recent KPI window against a historical baseline, ranking root causes, and generating exportable reports.

### 11.5.2 Input Data Format

The tool accepts Excel files (.xlsx or .xls) containing daily KPI exports. The file must include:

- An **eNodeB Name** column and a **Cell Name** (or **LocalCell Id**) column for cell identification
- A **Date** column in a standard date format
- One or more KPI counter columns (column names are matched flexibly using fuzzy matching)

### 11.5.3 Step-by-Step Usage

1. Select **KPI Degradation Analyzer** from the sidebar dropdown.
2. Click **Upload File** and select your Excel KPI export.
3. Select the appropriate worksheet from the sheet dropdown.
4. Choose the **KPI** to analyze from the KPI selector.
5. Set the **Comparison Days** (the number of recent days to compare against the baseline; default is 4).
6. Set the **Degradation Threshold** (the minimum degradation percentage to flag a cell; default is 30%).
7. Select the **Baseline Mode**:

- *Last Week* — compares the recent window against the same days in the previous week. Best for detecting sudden incidents.
  - *4-Week Rolling Median* — uses the median of the same weekdays over the past four weeks. Best for smoothing recurring weekly patterns.
8. Click **Run Selected KPI** to analyze one KPI, or **Analyze All KPIs** to process all 13 categories at once.
  9. Review the results table displaying degraded cells, degradation percentages, root cause categories, and recommended actions.
  10. Use the **Charts** tab to view bar charts of degraded cells per KPI and root cause distribution.
  11. Use the **Trends** tab to view before/after line chart overlays.
  12. Export results using the **Save CSV** button or generate a formatted Word report with **Generate Report**.

#### 11.5.4 Understanding the Output

- **rf\_severity**: Operational RF impact label — Normal, Medium, High, or Critical.
- **rca\_pattern**: Root cause pattern — Outage, Congestion, Radio Quality, Coverage, Interference, Mobility, Demand, or Unknown.
- **main\_root\_cause\_category**: The highest-scored root cause category.
- **main\_recommended\_action**: The engineering action recommended for the top cause.
- **analysis\_confidence**: Confidence label based on statistical significance, data completeness, and severity.

## 11.6 Tool 2 — KPI Forecasting Engine

### 11.6.1 Purpose

Forecasts future KPI trends using machine learning and statistical time-series models, enabling engineers to anticipate degradation before it occurs operationally.

### 11.6.2 Input Data Format

A CSV file with:

- A **Date** column in YYYY-MM-DD format
- A **Cell Name** column
- One column per KPI using the raw counter names defined in `config.py`

### 11.6.3 Step-by-Step Usage

1. Select **KPI Forecaster** from the sidebar dropdown.
2. Upload your cleaned KPI CSV file when prompted.
3. From the sidebar, choose a **Forecasting Method**: XGBoost, Holt-Winters, or Compare Both.
4. Select the **KPI to Forecast** from the dropdown (12 KPIs supported).
5. Select a **Cell** from the dropdown (sorted by weekly seasonality strength, strongest first).
6. Set the **Hold-out Test Days** (the number of most recent days used for backtesting; default is 4).
7. Optionally enable:
  - *Show Next-7-Day Future Forecast* — extends the prediction beyond the test window.
  - *Flag Seasonality Strength* — displays STL-based seasonality badges.
  - *Compare vs Naive Baseline* — benchmarks models against naive and weekly-mean baselines.
8. Adjust XGBoost or Holt-Winters hyperparameters from the sidebar if required.
9. Review the forecast chart, accuracy metrics (MAE, RMSE, MAPE), predictions table, and the 7-day forward forecast table.
10. Expand the **Feature Importance** panel to inspect the top-15 XGBoost predictors.
11. Expand the **Residual Analysis** panel for diagnostic statistics (Durbin-Watson, Ljung-Box).
12. Use the **LLM-Ready Report** panel to generate a structured summary suitable for pasting into an LLM for further optimization reasoning.

## 11.7 Tool 3 — Optimization Action Analyzer

### 11.7.1 Purpose

Correlates RF configuration changes (antenna tilt adjustments, power parameter updates) recorded in a Dolt version-controlled database with before/after KPI impact, enabling engineers to quantitatively evaluate the effect of optimization actions.

### 11.7.2 Prerequisites

- Dolt installed and a Dolt SQL server running on the default port (3306)
- A Dolt database containing `network_kpis` and `network_configs` tables
- Configuration changes committed to the Dolt database (each commit corresponds to one optimization action)

### 11.7.3 Step-by-Step Usage

1. Select **Optimization Action Analyzer** from the sidebar dropdown.
2. The tool connects to the Dolt server automatically and reads the commit log.
3. Select an **Action Hunk** (a date grouping of one or more Dolt commits) from the timeline to analyze.
4. For each KPI detected in the database, set the **polarity**:
  - *Higher is Better* (e.g., Throughput, Setup Success Rate)
  - *Lower is Better* (e.g., Drop Rate, Block Rate)
5. Review the results:
  - **Action Timeline** — a visual chart of all Dolt commits.
  - **Before/After Line Chart** — overlays the same weekday before and after the action.
  - **Delta Bar Chart** — shows the absolute KPI change per time interval, color-coded green (improved) or red (degraded).
  - **KPI Summary Table** — structured before/after comparison with absolute delta, percentage change, and status.

## 11.8 Tool 4 — Telecom RAG Assistant

### 11.8.1 Purpose

Answers engineering questions about LTE and 5G NR standards in natural language, with every response grounded in and cited from locally indexed 3GPP Technical Specification PDFs.

### 11.8.2 Prerequisites

Ensure that the RAG Assistant was configured as described in Step 3 of the Installation section: the `.env` file must contain a valid OpenRouter API key, 3GPP PDFs must be placed in `RAG_tool/data/`, and the ingestion pipeline must have been run at least once.

### 11.8.3 Step-by-Step Usage

1. Select **Telecom RAG Assistant** from the sidebar dropdown.
2. Optionally use the **Specification Filter** in the sidebar to restrict answers to a specific TS number (e.g., `36.331`) or 3GPP release (e.g., `17`).
3. Type your engineering question in natural language in the chat input box. Examples:
  - “What are the trigger conditions for Event A3 in LTE RRC?”
  - “What is the purpose of the RACH procedure in 5G NR?”

- *“Explain the difference between RLC AM and UM modes.”*
4. Press Enter or click Send.
  5. The assistant will display a structured technical response followed by expandable citation panels showing the exact TS number, release version, section number, and page number for each retrieved chunk.
  6. If the answer cannot be found in the indexed specifications, the assistant will explicitly state so rather than generating an unverified response.

### 11.8.4 Performance Tuning

The retrieval behaviour can be adjusted via the `RAG_tool/.env` file:

- `TOP_K` (default: 5) — number of specification chunks supplied to the language model.
- `FETCH_MULTIPLIER` (default: 4) — multiplier for chunks retrieved from Qdrant before re-ranking.
- `CHUNK_SIZE` (default: 800 tokens) — maximum size of each indexed chunk.
- `DEBUG_MODE` (default: false) — enables pipeline stage timing and prompt statistics in the sidebar.

## 11.9 Troubleshooting

**The platform does not start.** Verify that Python 3.11 or higher is installed and that all dependencies were installed with `pip install -r requirements.txt`. Check that `telecom_hub.py` is executed from the project root directory.

**The KPI Degradation Analyzer cannot find KPI columns.** Ensure your Excel file includes column headers that partially match the expected counter names. The tool uses fuzzy matching, but column names must be recognizable. Refer to the KPI configuration reference in `KPI_and_its_related_counters.md`.

**The Forecaster produces a flat or unstable forecast.** This may occur when the data window is shorter than 21 days, causing lag features to be sparsely populated. Reduce `max_depth` and `n_estimators` in the XGBoost sidebar, or switch to the Holt-Winters model.

**The Optimization Action Analyzer cannot connect to Dolt.** Verify that the Dolt SQL server is running (`dolt sql-server`) and that the `network_kpis` and `network_configs` tables exist in the active database.

**The RAG Assistant returns “I cannot find this in the supplied specifications.”** The queried concept may not be present in the indexed PDFs. Verify that the relevant 3GPP specification has been placed in `RAG_tool/data/` and re-run the ingestion pipeline (`python -m app.ingest -all`).

**The RAG Assistant is slow to respond.** Response latency depends on the OpenRouter API. Enable `DEBUG_MODE=true` in `.env` to display per-stage pipeline timing in the sidebar and identify bottlenecks.

# Bibliography

- [1] S. Sesia, I. Toufik, and M. Baker, *LTE — The UMTS Long Term Evolution: From Theory to Practice*, 2nd ed. Wiley, 2011.
- [2] H. Holma and A. Toskala, *LTE for UMTS: Evolution to LTE-Advanced*, 2nd ed. Wiley, 2011.
- [3] E. Dahlman, S. Parkvall, and J. Sköld, *5G NR: The Next Generation Wireless Access Technology*, 2nd ed. Academic Press, 2020.
- [4] A. R. Mishra, *Advanced Cellular Network Planning and Optimisation: 2G/2.5G/3G Evolution to 4G*. Wiley, 2007.
- [5] D. C. Montgomery and G. C. Runger, *Applied Statistics and Probability for Engineers*, 7th ed. Wiley, 2018.
- [6] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. OTexts, 2018.
- [7] G. James, D. Witten, T. Hastie, and R. Tibshirani, *An Introduction to Statistical Learning: With Applications in R*, 2nd ed. Springer, 2021.
- [8] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.
- [9] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. Prentice Hall, 2023.
- [10] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Addison-Wesley, 2021.
- [11] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [12] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2015.
- [13] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner’s Approach*, 8th ed. McGraw-Hill, 2014.
- [14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [15] E. Alpaydin, *Introduction to Machine Learning*, 4th ed. MIT Press, 2020.
- [16] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. Springer, 2009.



- [17] D. C. Montgomery, *Design and Analysis of Experiments*, 9th ed. Wiley, 2017.
- [18] J. W. Creswell and J. D. Creswell, *Research Design: Qualitative, Quantitative, and Mixed Methods Approaches*, 5th ed. SAGE Publications, 2017.
- [19] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.
- [20] T. S. Rappaport, R. W. H. Jr., R. C. Daniels, and J. N. Murdock, *Millimeter Wave Wireless Communications*. Pearson, 2024.